

BCA Project Report

Subject Code: BCSP-064

Title of the Project

“College Campus Lost & Found System”

Student Name: Pinki

Enrolment Number: 2352226370

Programme Code: BCA

Programme Name: Bachelor of Computer Application

Mobile Number: +91 99589 33429

Email ID: pinki.rawat205641@gmail.com

Study Centre Name & Code: PCTI — PC Training Institute / 2900

Regional Centre Name & Code: RC Delhi-2 / 29

Under the Guidance of

Guide Name: Sonu Pachuri

Qualification: B.Tech in Computer Science, Lucknow, 2012

Current Post: Technical Project Manager

Submission Date: ____ / ____ / 20____

Table of Contents

“College Campus Lost & Found System”	1
Table of Contents.....	2
PART A: PROJECT SYNOPSIS	7
1. Title and Introduction	8
1.1 Title of the Project	8
1.2 Introduction	8
2. Scope and Objectives	8
2.1 Scope of the Project.....	8
2.2 Objectives of the Project.....	9
Conclusion of Objectives.....	10
3. Project Category.....	10
3.1 Implementation Methodology	10
4. Problem Definition of the Project	11
5. Module Description and Process Logic.....	12
5.1 Module Description.....	12
5.2 Process Logic	13
6. Analysis of the Project	14
6.1 PERT Chart	14
6.2 Gantt Chart.....	14
Project Schedule (Task List).....	15
Gantt Chart Visualization.....	15
6.3 Data Flow Diagram (DFD)	15
6.4 Entity Relationship Diagram (ERD)	16
7. System Requirement Specification (SRS)	17
7.1 Software Requirements.....	17
7.2 Hardware Requirements.....	17
8. System Development Life Cycle (SDLC)	18
9. Feasibility Study	19
10. Testing of the Project	20
Expected Outcomes of Testing.....	21
11. Industry / Client Interface	21
12. Future Scope and Further Enhancements	22
Conclusion.....	23

13. Bibliography	23
PART B: PROJECT REPORT	25
CHAPTER 1: INTRODUCTION & BACKGROUND	25
1.1 Title & Introduction	25
1.1.1 Title of the Project.....	25
1.1.2 Introduction.....	25
1.2 Objectives of the System	26
1.2.1 Consolidated Statement of Purpose	27
1.3 Project Category	27
1.4 Literature Review	28
1.4.1 Review of Existing Practice.....	28
1.4.2 Review of Technical Literature	28
1.4.3 Identified Research Gap	29
1.5 QR Link of Project	29
CHAPTER 2: SYSTEM ANALYSIS	30
2.1 Problem Definition	30
2.2 Feasibility Study	31
2.2.1 Technical Feasibility.....	31
2.2.2 Economic Feasibility	31
2.2.3 Operational Feasibility	31
2.2.4 Legal Feasibility	32
2.2.5 Schedule Feasibility	32
2.2.6 Conclusion of the Feasibility Study.....	32
2.3 Methodology	32
2.3.1 Choice of Development Model.....	32
2.3.2 Implementation Phases.....	32
2.3.3 Project Schedule.....	33
2.4 System Requirement Specification (SRS).....	34
2.4.1 Software Requirements	34
2.4.2 Hardware Requirements	34
2.4.3 Functional Requirements (FR).....	35
2.4.4 Non-Functional Requirements (NFR).....	37
2.4.5 Assumptions and Constraints.....	37
CHAPTER 3: SYSTEM DESIGN & DIAGRAMS.....	39

3.1 Analysis Diagrams	39
3.1.1 Data Flow Diagrams	39
3.1.2 Entity Relationship Diagram	42
3.1.3 Data Dictionary	43
3.1.4 UML Diagrams	44
3.2 System Design	47
3.2.1 Modularization	47
3.2.2 Database Design and Normalization	49
3.2.3 User Interface Design and Navigation Flow	50
3.3 High-Level System Architecture	51
A. Visual Architecture Diagram	51
B. Architectural Layers Breakdown	52
C. End-to-End Execution Flow	52
CHAPTER 4: IMPLEMENTATION & CODING	54
4.1 Development Environment and Project Directory Structure	54
4.1.1 Development Environment	54
4.1.2 Project Directory Structure	55
4.2 Core Algorithms and Logic Design	55
4.2.1 Authentication and Password Hashing Algorithm	55
4.2.2 Role-Based Access Control Algorithm	56
4.2.3 Dynamic Multi-Criteria Search and Filter Algorithm	57
4.2.4 Secure Image Upload and Collision-Avoidance Algorithm	57
4.2.5 Ownership-Verified Status Mutation Algorithm	58
4.3 Programming Code	58
4.3.1 Database Provisioning Script — <code>create_mysql_db.py</code>	59
4.3.2 Application Configuration and ORM Models — <code>app.py</code>	59
4.3.3 Search and Feed Route — <code>/</code>	60
4.3.4 Authentication Routes — <code>/register, /login, /logout</code>	61
4.3.5 Item Management Routes — <code>/item/add, /item/<id>, /item/<id>/update_status</code>	62
4.3.6 Administrative Routes and Application Entry Point	63
4.3.7 Frontend Presentation Layer	64
4.3.8 Styling and Client-Side Script	74
4.4 Error Handling and Parameter Passing	88

4.4.1 Secure Parameter Passing.....	88
4.4.2 Error Handling Mechanisms	88
4.5 Hosting and Deployment	89
4.5.1 Local Execution	89
4.5.2 Production Deployment Checklist	90
4.6 System Security and Data Privacy Protocols.....	90
4.6.1 Cryptography and Data at Rest	90
4.6.2 Authorization and Session Integrity	91
4.6.3 File Upload Attack Surface	91
4.6.4 Mitigations Present in the Code	91
4.6.5 Known Limitations and Recommended Hardening.....	92
CHAPTER 5: TESTING & RESULTS	94
5.1 Testing Objectives and Strategy	94
5.1.1 Objectives	94
5.1.2 Strategy.....	94
5.1.3 Test Environment.....	95
5.2 Test Cases.....	95
5.2.1 Unit Testing.....	95
5.2.2 Integration Testing	99
5.2.3 System and End-to-End Testing.....	99
5.2.4 Security and Authorization Testing.....	100
5.2.5 Usability and Compatibility Testing.....	101
5.3 Test Case Table	102
5.4 Debugging and Improvement.....	104
5.5 Results and Screen Layouts	105
5.5.1 Registration and Login Screens	105
5.5.2 Home Feed with Search and Filters.....	105
5.5.3 Post-an-Item Form	106
5.5.4 Item Detail and Status Update	106
5.5.5 Administrative Dashboard.....	106
5.5.6 Administrative Item Moderation and User Management	106
5.5.7 Conclusion of Testing.....	106
CHAPTER 6: CONCLUSION & REFERENCES	107
6.1 Conclusion and Real-World Impact	107

6.1.1 Conclusion	107
6.1.2 Key Learning Outcomes	107
6.1.3 Real-World Impact and Institutional Viability	108
6.2 System Limitations and Technical Constraints	109
6.3 Future Scope and Further Enhancements.....	111
6.4 Bibliography.....	112
A. Academic Books	112
B. Official Technical Documentation	112
C. Web Resources	113
6.5 Thank You Note	114

PART A: PROJECT SYNOPSIS

The following synopsis is the approved project proposal, reproduced here as submitted. Part B of this report documents the system that was subsequently built against this proposal.

1. Title and Introduction

1.1 Title of the Project

“College Campus Lost & Found System”

1.2 Introduction

Managing lost and found items has always been a persistent challenge within college and university campuses. In today’s fast-paced academic environment, students frequently misplace valuable belongings such as electronics, study materials, ID cards, and personal accessories. However, without a centralized and accessible tracking system, most individuals face difficulties in recovering their lost items or easily returning items they have found. A simple manual system or traditional methods, such as physical notice boards or campus security drop-offs, often fail to provide timely communication and widespread visibility.

This is where a digital, technology-driven solution becomes essential. The proposed project, College Campus Lost & Found System, aims to build a comprehensive web-based application that centralizes the reporting and tracking of misplaced items across the campus. Unlike basic physical logs, this system will provide a dynamic platform where users can post detailed descriptions, upload images of items, and update recovery statuses in real time.

By combining secure data management, user authentication, and an intuitive user interface, the system will act as a unified recovery hub for the campus community. It will streamline routine tasks such as the categorization of items (e.g., Electronics, Books, Accessories) and provide advanced search and filtering capabilities to quickly match lost items with found items. Additionally, the system incorporates Role-Based Access Control (RBAC), ensuring that while students can freely communicate and collaborate, appointed administrators can safely moderate the environment and protect user data.

This project belongs to the Web Application and Relational Database Management System (RDBMS) category, as it integrates web-based inputs with secure storage in a MySQL relational database. With the growing necessity for smart campus infrastructure, this project has great potential to positively impact the student body. It is not just a standard registry, but a secure, collaborative portal designed to empower students, foster a helpful community, and ensure the efficient recovery of valuable personal belongings.

2. Scope and Objectives

2.1 Scope of the Project

The scope of the College Campus Lost & Found System extends to providing a centralized, secure, and digital platform for students, faculty, and staff to report, track, and recover misplaced belongings within a university environment. It is designed to replace inefficient

physical notice boards and scattered administrative drop-off points with a unified web portal.

The primary scope includes developing a responsive web-based application where registered users can log details of lost or found items. These entries will include specific attributes such as titles, detailed descriptions, physical locations, contact information, and secure image uploads. The system will automatically organize these listings into predefined categories such as Electronics, Books & Notes, Accessories & Wearables, and Others.

Users will be equipped with advanced search and filtering mechanisms, allowing them to query the database by keywords, categories, and item statuses. Furthermore, the scope includes real-time status management, empowering the original poster to dynamically update an item's state to "Lost," "Found," or "Claimed" as situations resolve.

Additionally, the scope heavily emphasizes platform moderation and security through Role-Based Access Control (RBAC). The system features a dedicated administrative layer, granting appointed campus moderators access to a specialized dashboard. From here, administrators can monitor system-wide metrics, oversee active listings, and enforce community guidelines by deleting inappropriate items or banning malicious user accounts.

In the future, the scope may expand to include automated email notifications for keyword matches, AI-based image recognition for automated item pairing, and direct SSO (Single Sign-On) integration with the university's central identity management system.

2.2 Objectives of the Project

The objectives of the College Campus Lost & Found System project are structured around solving the practical, day-to-day challenges faced by the campus community in recovering valuable personal property. The following are the key objectives:

1. Centralized Item Tracking. To develop a unified digital repository that effectively replaces scattered physical logs, allowing students to easily post, browse, and track lost and found items from any device.

2. Efficient Search and Filtering. To implement robust keyword search and category-based filtering capabilities. This objective ensures users can quickly narrow down the database to find specific misplaced belongings, drastically reducing recovery time.

3. Dynamic Status Management. To allow users to dynamically update the recovery status of their uploaded items (Lost, Found, Claimed). This ensures the platform's feed always reflects real-time, accurate inventory and prevents confusion over already-recovered goods.

4. Secure Image and Data Handling. To ensure that uploaded item images are securely validated (restricting formats and file sizes) and stored properly. Furthermore, to protect user profiles and contact information through secure authentication and encrypted database storage.

5. Role-Based Moderation and Administration. To provide a secure administrative dashboard with elevated privileges. This objective enables campus staff to oversee total user metrics, monitor all active listings, and safely delete policy-violating accounts or fraudulent posts to maintain platform integrity.

6. Modern and Responsive Interface. To create a highly intuitive, user-friendly frontend interface utilizing modern design principles (such as glassmorphism). This ensures seamless usability and encourages widespread adoption across both desktop and mobile browsers.

Conclusion of Objectives

The overarching objective of this project is to foster a supportive, honest, and connected campus community by minimizing the friction involved in recovering lost property. By combining secure relational data handling with an intuitive, accessible web interface, the project transforms a disorganized manual process into a highly efficient, moderated digital ecosystem.

3. Project Category

This project falls under the Web Application Development and Relational Database Management System (RDBMS) category. It combines secure data storage and retrieval mechanisms for item tracking with a modern, responsive web interface designed for dynamic user interaction and administrative moderation.

3.1 Implementation Methodology

The implementation methodology for the College Campus Lost & Found System will follow a systematic software development life cycle (SDLC) approach, ensuring proper planning, analysis, design, development, testing, and deployment of the project.

The Agile Methodology will be adopted, as it allows iterative development, continuous feedback, and the flexibility to incorporate improvements during the project's construction.

1. Requirement Analysis. At this stage, requirements are gathered to understand how students and administrators need to record, search, and manage lost property. Functional requirements (e.g., secure authentication, posting items, keyword filtering, role-based dashboards) and non-functional requirements (e.g., system security, responsive UI, data validation) are clearly identified.

2. System Design. The design stage involves mapping the application architecture through Data Flow Diagrams (DFD), Entity Relationship (ER) Diagrams, and database schemas. The user interface (UI) and user experience (UX) are also structured, focusing on a modern "glassmorphism" aesthetic and intuitive navigation.

3. Database Implementation (RDBMS). A relational database (MySQL) is utilized for storing registered user details, encrypted passwords, item attributes, and status logs. The

Flask-SQLAlchemy Object Relational Mapper (ORM) is implemented to abstract database operations, ensuring data is normalized for efficiency and redundancy control.

4. Backend Application Development. The core server-side logic is built using Python and the Flask framework. This includes implementing Flask-Login for secure session management, Werkzeug for cryptographic password hashing and safe file upload handling, and establishing the routing logic that dictates user and admin behaviours.

5. Frontend Application Development. The client-side interface is developed using HTML5, CSS3, and JavaScript ES6. The Jinja2 templating engine is heavily utilized to dynamically render secure backend database content, interactive item cards, and conditional UI elements based on the current user's role (Student vs. Admin).

6. Testing and Debugging. Comprehensive testing strategies — including unit testing, integration testing, and system testing — are executed to ensure modules function correctly. Specific attention is given to form validation, secure image uploading restrictions, algorithmic search logic, and authorization barriers.

7. Deployment and Maintenance. The fully packaged system is deployed on a secure server environment, with the database automatically instantiated via setup scripts. Continuous monitoring is planned to address post-launch bug fixes, performance optimizations, and future administrative enhancements.

4. Problem Definition of the Project

In a bustling college or university campus, students and staff frequently misplace personal belongings such as ID cards, electronic devices, books, and wallets. Despite the high frequency of these incidents, most educational institutions still rely on outdated and highly inefficient manual recovery methods. Currently, when an item is lost or found, individuals typically depend on physical notice boards, handing items over to scattered security desks, or relying on fragmented communication through unofficial social media groups.

These traditional approaches present several significant pain points:

Lack of Centralized Visibility. A physical notice on a bulletin board or a message in a single group chat only reaches a fraction of the campus population. Many found items remain unclaimed simply because the original owner never saw the notification.

Absence of Searchability. Manual logs or group chats do not offer organized filtering. A student looking for a lost “Dell Laptop” cannot easily search through weeks of unorganized messages or handwritten ledgers.

No Real-Time Status Tracking. In existing manual systems, once a notice is posted, there is no dynamic way to update its status. Students often waste time inquiring about items that have already been returned because the original notice was never taken down.

Lack of Moderation and Security. Publicly posting contact information on notice boards poses privacy risks. Furthermore, there is no administrative oversight to prevent fraudulent claims or remove inappropriate content.

Because of these inefficiencies, a large volume of recovered property simply accumulates in administrative offices, while the students who lost them assume their items are gone forever.

Therefore, the problem definition for this project is:

“To develop a centralized, secure, and intuitive web-based Lost & Found System that replaces inefficient manual tracking by providing real-time status management, automated categorization, and administrative moderation to ensure the swift and secure recovery of misplaced campus property.”

This problem definition highlights the critical gap between disorganized physical reporting and a modernized, digital recovery ecosystem, which this project successfully bridges.

5. Module Description and Process Logic

5.1 Module Description

The College Campus Lost & Found System is divided into multiple interconnected modules. Each module represents a core functionality that works together to deliver a secure, user-friendly, and efficient property recovery platform. The following are the key modules:

1. User Authentication and Role Management Module. This module serves as the security gateway for the system, ensuring that only verified campus members can interact with the platform.

- *Features:* User registration, secure login/logout, password encryption, and role assignment (Student vs. Admin).
- *Functionality:* Utilizes Flask-Login for secure session handling and `werkzeug.security` to encrypt passwords using `pbkdf2:sha256` hashing before storing them in the MySQL database. It strictly enforces access control to protected routes.
- *Importance:* Establishes accountability for posts, protects user data, and differentiates standard users from system moderators.

2. Item Entry and Secure Upload Module. This is the primary module where users log the details of lost or discovered belongings.

- *Features:* Form-based data entry, category selection, and secure image uploading.
- *Functionality:* Captures specific item attributes (title, description, location, contact info). It securely processes image uploads by validating file extensions (.jpg, .png) and enforcing a 5 MB size limit. The module uses `secure_filename()` and appends timestamps to prevent file overwriting.

- *Importance:* Forms the core operational dataset of the platform, providing the necessary visual and textual details required to identify items.

3. Advanced Search and Filtering Module. This module organizes the database for rapid querying and retrieval.

- *Features:* Keyword text search, dropdown category filters, and status filters.
- *Functionality:* Parses GET request arguments and dynamically constructs SQLAlchemy database queries. It uses `ilike` operations for partial keyword matches in titles and descriptions, and exact matching for categories (e.g., Electronics, Books).
- *Importance:* Drastically reduces the time users spend looking for specific items, bypassing the clutter of chronological feeds.

4. Dynamic Status Management Module. This module ensures the platform reflects the current, real-time reality of lost and found items.

- *Features:* Real-time toggling of item states (Lost, Found, Claimed).
- *Functionality:* Evaluates backend logic to verify that the `current_user.id` matches the item's original `user_id`. If authorized, the system updates the status column in the database, which dynamically alters the visual UI badges on the frontend.
- *Importance:* Prevents confusion, prevents users from inquiring about already-claimed items, and keeps the active feed highly relevant.

5. Administrative Dashboard and Moderation Module. This module provides system-level management and oversight.

- *Features:* Global metrics overview, user account management, and item moderation.
- *Functionality:* Protected by a custom `@admin_required` routing decorator. It aggregates live database counts (Total Users, Total Lost, Total Claimed). It grants administrators the power to forcefully delete fraudulent items or execute cascade deletions on malicious user accounts.
- *Importance:* Maintains platform integrity, enforces community guidelines, and ensures the system remains free of spam.

5.2 Process Logic

The process logic of the College Campus Lost & Found System defines the flow of data, control mechanisms, and decision-making pathways within the application. It explains how different modules interact and how inputs are transformed into outputs.

1. Authentication Process Logic. When a user attempts to register, the system checks the database for duplicate emails. If unique, the password is cryptographically hashed and the record is committed to MySQL. During login, the system compares the submitted plaintext password against the stored hash. If verified, a secure session token is generated, and the user is redirected to the main dashboard.

2. Item Posting and File Handling Logic. When a user submits the “Post Item” form, the backend first validates the text inputs. If an image is included, the `allowed_file()` function

executes, verifying the file extension against a whitelist. The file is then assigned a unique timestamped filename and saved to the server's static directory. Simultaneously, an ORM (Object Relational Mapper) transaction maps the text data and the image path to a new row in the Item database table.

3. Search and Query Logic. When a user inputs filter parameters on the homepage, the UI sends a GET request to the backend. The server initializes a base database query (`Item.query`). It sequentially applies filters: if a keyword exists, it adds an `ilike` text match; if a category is selected, it adds a `filter_by(category)` constraint. The final refined dataset is fetched and passed to the Jinja2 templating engine to dynamically render the corresponding HTML item cards.

4. Status Mutation Logic. When a user attempts to mark an item as "Claimed," the backend intercepts the POST request. It first executes an authorization check: `if item.owner.id != current_user.id`. If the user is not the original author, the request is rejected with an error flash. If authorized, the status string is updated, the database is committed, and the UI immediately reflects the new state.

5. Admin Moderation Logic. When an administrator requests to view the dashboard or delete a record, the custom `@admin_required` decorator fires before the route executes. It verifies that `current_user.is_authenticated` and `current_user.role == 'admin'`. If true, the system allows the execution of administrative database queries (such as a cascade deletion that removes a user and simultaneously purges all items linked to their foreign key). If false, the system abruptly terminates the request and redirects the user.

6. Analysis of the Project

The analysis phase provides a detailed breakdown of the system's functionality, structure, and data organization. It ensures that the project is designed logically and aligns with the requirements specified earlier. The following components are part of this analysis.

6.1 PERT Chart

The PERT chart maps the logical sequence and dependencies of project tasks.

[INSERT PERT CHART HERE]

6.2 Gantt Chart

Assumptions. Before designing the schedule, the following assumptions are made regarding the project development:

- Project Duration: 8 Weeks (2 Months).
- Team Structure: Individual Project (Single Developer handling Design, Coding, and Testing).
- Development Methodology: Agile SDLC (Iterative development).

- Scope: The system includes User Authentication, Item Management, Search/Filter Modules, and an Admin Dashboard.

Project Schedule (Task List)

The following table defines the tasks, their estimated duration, and the starting timeline:

Task ID	Task Name	Duration	Start Week
T1	Requirement Gathering & Analysis	1 Week	Week 1
T2	System Design (DFD, ERD, DB Schema)	1 Week	Week 2
T3	Coding: Database Creation & Setup	1 Week	Week 3
T4	Coding: Backend (Auth, Routing, Logic)	2 Weeks	Week 3
T5	Coding: Frontend Development (UI/UX)	2 Weeks	Week 4
T6	System Integration (Connecting UI with DB)	1 Week	Week 6
T7	System Testing (Unit & System Testing)	1 Week	Week 7
T8	Final Documentation & Deployment	1 Week	Week 8

Gantt Chart Visualization

Task Name	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8
Requirement Analysis	X							
System Design		X						
Coding: DB & Setup			X					
Coding: Backend			X	X				
Coding: Frontend				X	X			
System Integration						X		
System Testing							X	
Documentation								X

6.3 Data Flow Diagram (DFD)

The DFD shows how data moves within the system.

Level 0 (Context Diagram).

- **Student User** inputs Registration/Login credentials and Item Details → **Central Lost & Found System** processes and stores in Database → Outputs Feed/Search Results.
- **Admin User** inputs Moderation Commands → **Central Lost & Found System** executes deletions → Outputs Analytics Metrics to Admin.

[INSERT DFD LEVEL 0 DIAGRAM HERE]

Level 1 (Detailed Flow).

- 1. User Inputs Credentials → Setup Auth Session Module → Validates with User Database.
- 2. User Inputs Item Data + Image → Handle Post Creation Module → Saves to Item Database and Static File System.
- 3. User Requests Search/Filter → Execution Search Queue Module → Fetches relevant rows from Item Database → Renders HTML DOM.
- 4. Admin Executing Actions → Admin Dashboard Module → Enforces Deletions in User/Item Database.

[INSERT DFD LEVEL 1 DIAGRAM HERE]

Level 2 (Exploding Item Management).

- 1. Client Submission → Form Sanitation and Validation.
- 2. If Valid Data → Secure Media Image Processor → Writes File to `static/uploads`.
- 3. Image Path + Text Data → ORM Model Construction (SQLAlchemy).
- 4. SQL Parameterization → Commits to `item` MySQL Table.
- 5. Output Dispatcher → Returns HTTP 302 Redirect and Success Message.

[INSERT DFD LEVEL 2 DIAGRAM HERE]

6.4 Entity Relationship Diagram (ERD)

The ERD defines how database tables relate.

Entities and Attributes.

- 1. **User** (id, name, email, password_hash, role)
- 2. **Item** (id, title, description, category, status, image_path, date_reported, location, contact_info, user_id)

Relationships.

- **User (1) — (M) Item** — *a user can post multiple items, but each item belongs to exactly one user.*

[INSERT ERD DIAGRAM HERE]

Data Structure — User Table

S. No.	Field Name	Data Type	Description
1	id	INT	User ID (Primary Key, Auto Increment)
2	name	VARCHAR(150)	Full Name of the User
3	email	VARCHAR(150)	User Email Address (Unique)
4	password_hash	VARCHAR(256)	Encrypted Password
5	role	VARCHAR(20)	User Privilege Level (student/admin)

Data Structure — Item Table

S. No.	Field Name	Data Type	Description
1	id	INT	Item ID (Primary Key, Auto Increment)
2	title	VARCHAR(150)	Short descriptor / header
3	description	TEXT	Elaborated visual descriptors
4	category	VARCHAR(50)	Item Category (Electronics, Books, etc.)
5	status	VARCHAR(50)	State tracking indicator (Lost, Found, Claimed)
6	image_path	VARCHAR(255)	Relative path to uploaded image file
7	date_reported	DATETIME	Automatically logs upload timing
8	location	VARCHAR(150)	Physical location where item was lost/found
9	contact_info	VARCHAR(150)	Instructions to meet or phone number
10	user_id	INT (FK)	Refers to User Table (author tracking)

7. System Requirement Specification (SRS)

The system requirements are categorized into software requirements and hardware requirements. These specifications ensure the project can be implemented efficiently and run smoothly across devices.

7.1 Software Requirements

Category	Specification
Operating System	Windows 10/11, Linux Ubuntu 20.04+, or macOS 10.15+
Programming Languages	Python 3.8+ (Backend), JavaScript ES6 (Frontend)
Frameworks & Libraries	Flask 3.0.0, Flask-SQLAlchemy, Flask-Login, Werkzeug, Jinja2
Database	MySQL Server 8.0+ (via PyMySQL adapter)
Frontend Technologies	HTML5, CSS3, FontAwesome (for UI icons)
Web Server	Werkzeug (Development) / Nginx or Apache (Production)
IDE / Tools	VS Code, PyCharm, MySQL Workbench / XAMPP
Testing Tools	Python <code>unittest</code> framework
Browser Support	Google Chrome, Mozilla Firefox, Safari, Microsoft Edge

7.2 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	Dual Core, 2.0 GHz	Intel i5/i7 or AMD Ryzen Quad Core, 3.0 GHz+
RAM	4 GB	8 GB or higher
Storage (HDD/SSD)	250 GB HDD or 128 GB SSD	500 GB SSD (for better application speed)
Display	1366 x 768 resolution	1920 x 1080 Full HD

Network	Stable internet connection (2 Mbps minimum)	High-speed broadband (10 Mbps or higher)
Power Supply	Standard 65 W adapter	UPS backup for server or continuous usage
Devices Supported	Desktop / Laptop / Smartphone (via responsive UI)	Desktop, Laptop, Tablet, Smartphone

8. System Development Life Cycle (SDLC)

The System Development Life Cycle (SDLC) is a structured process followed to develop software applications with proper planning, design, development, testing, and deployment. For the College Campus Lost & Found System, adhering to the SDLC ensures that the system is secure, user-friendly, highly reliable, and capable of handling campus-wide property recovery efficiently.

1. Requirement Analysis. This is the first step where project requirements are gathered from the end-users — primarily students, campus staff, and system administrators who will utilize the application. Key requirements identified include secure user authentication, detailed item logging with image uploads, robust search and categorization filters, and a dedicated moderation dashboard for administrative oversight.

2. System Design. In this phase, the architectural framework of the application is designed. Use case diagrams, data flow diagrams (DFD), and entity-relationship (ER) diagrams are prepared to map out the logical flow. The system design ensures the smooth integration of the MySQL database schema, the Flask backend controllers, and the modern, responsive frontend user interface (UI).

3. Implementation (Coding). Based on the finalized design, the actual coding process begins. The application is developed using Python and the Flask framework for the backend, MySQL for the database management, and HTML5/CSS3/JavaScript for the web interface. Each core module — such as “Item Entry,” “Advanced Search,” “Status Management,” and “Administrative Dashboard” — is coded and structured modularly.

4. Testing. After coding is complete, the modules are tested independently (unit testing) to ensure their specific functions work correctly, such as verifying that only valid image formats (.jpg, .png) are accepted. Subsequently, the modules are combined to test the application as a whole (system and integration testing). Errors, security vulnerabilities, and UI responsiveness issues are identified and resolved.

5. Deployment. Once the testing phase is completely successful and the application is deemed stable, the system is deployed for live use. Initially, it may be released in a controlled beta environment to a small group of students and staff to gather real-world feedback and ensure server stability before a full, campus-wide launch.

6. Maintenance. After deployment, ongoing maintenance is required to ensure the platform remains secure and functional. This phase involves applying regular software

updates, optimizing database performance as the number of listings grows, patching newly discovered bugs, and potentially integrating future enhancements based on continuous user feedback.

By strictly following these SDLC phases, the project ensures the high-quality, systematic, and timely delivery of the College Campus Lost & Found System.

9. Feasibility Study

Before committing resources and time to the development of any software system, it is crucial to conduct a comprehensive feasibility study. This study determines whether the proposed solution is practical, viable, and achievable within the given constraints. For the College Campus Lost & Found System, the feasibility study is evaluated across the following five key areas.

1. Technical Feasibility. The project is built using established, widely adopted technologies. The backend relies on Python and the Flask framework, while database management is handled by MySQL via the Flask-SQLAlchemy ORM. The frontend utilizes standard HTML5, CSS3, and JavaScript. The system also securely handles file uploads and cryptographic password hashing using the Werkzeug library. Since these technologies are mature, highly compatible with standard web browsers, and require no specialized proprietary hardware to run, the project is entirely technically feasible.

2. Economic Feasibility. The primary development tools and frameworks utilized in this project — such as Python, Flask, MySQL, and standard frontend languages — are entirely open-source and free to use. There are no expensive software licensing fees required for development. Furthermore, the application can be hosted on affordable or educational-tier cloud servers (for example AWS Free Tier, Heroku, or PythonAnywhere). Given the minimal financial overhead and the high value it provides to the campus community, the project is highly economically feasible.

3. Operational Feasibility. The proposed system is designed with a strong emphasis on user experience (UX). The interface utilizes a modern, intuitive “glassmorphism” design with clear navigational elements, making it accessible to users with varying levels of technical expertise. Students can easily post items, upload images, and update statuses without requiring any prior training or instruction manuals. Similarly, the administrative dashboard provides straightforward moderation tools. Because the platform easily integrates into the daily routines of the campus community, it is highly operationally feasible.

4. Legal Feasibility. Managing personal information (such as student names, college email addresses, and contact details) requires strict adherence to data privacy standards. The system ensures legal compliance by implementing robust security measures. User passwords are not stored as plain text; they are encrypted using `pbkdf2:sha256` hashing algorithms. Additionally, the platform’s Role-Based Access Control (RBAC) ensures that only authorized administrators can access sensitive overarching metrics or delete records.

This commitment to data protection makes the project legally safe and feasible for institutional deployment.

5. Schedule Feasibility. The project development lifecycle has been carefully mapped out over an 8-week timeline, as detailed in the Gantt chart of Section 6.2. The scope of the project — encompassing database design, backend routing, frontend templating, and system integration — is well-defined and manageable for a dedicated developer within this timeframe. With a structured Agile approach allowing for concurrent development and testing, the project is highly viable from a scheduling perspective.

Overall, the feasibility study clearly indicates that the College Campus Lost & Found System is practical, highly cost-effective, secure, and beneficial to the student body and administrative staff.

10. Testing of the Project

Testing is a critical phase in software development, ensuring that the application is free of bugs, meets specified requirements, and performs efficiently. For the College Campus Lost & Found System, a rigorous Quality Assurance (QA) protocol was executed. The following testing strategies were applied.

1. Unit Testing. Each isolated module — such as the Registration System, Image Upload Validator, and Search Logic — was tested independently using Python’s built-in `unittest` framework. This ensured that the smallest parts of the system worked as expected. Specific tests validated duplicate email rejections and ensured the `allowed_file()` function securely blocked malicious file extensions like `.pdf` or `.exe`, strictly accepting only `.jpg` and `.png` files.

2. Integration Testing. After unit testing, interconnected modules were combined to verify their data hand-offs. For example, testing evaluated the data flow from the frontend HTML form DOM, through the Flask routing logic, and securely into the MySQL database via the SQLAlchemy Object Relational Mapper. Additionally, file system integrations were tested to ensure uploaded images correctly saved to the `static/uploads` directory and rendered dynamically on the frontend.

3. System Testing. The entire application was tested as a holistic, end-to-end system to ensure it met the System Requirement Specification. This evaluated the complete user journey: launching the server, navigating the landing screen, registering, posting an item, viewing the grid layout, and gracefully terminating sessions via the logout protocol.

4. Functional Testing. This strategy ensured that all explicitly declared core behaviours operated correctly. Testing verified that category filters processed accurately, keyword querying returned the correct subset of items, users could successfully toggle dynamic statuses (Lost/Found/Claimed), and administrative roles actively enforced privilege access limits.

5. Performance Testing. The system was evaluated to ensure it responds efficiently to simulated collegiate network activity. Database query speeds were tested to verify that the

ORM avoids N+1 select bottlenecks. Additionally, image loading metrics were evaluated; the usage of modern CSS flex-grid layouts ensured that heavy visual item feeds rendered seamlessly without blocking JavaScript execution threads.

6. Security and White-Box Testing. Because user data protection is paramount, internal code flows were verified. Testing confirmed that the `werkzeug.security` library successfully strips plaintext passwords and converts them to `pbkdf2:sha256` hashes before database insertion. Furthermore, the custom `@admin_required` decorator was tested to ensure it instantly dropped non-administrative access attempts, and the ORM was verified to successfully deflect standard SQL injection vulnerabilities.

7. Usability and Non-Functional Testing. The subjective user experience and interface layouts were evaluated. The colour contrasting, responsive mobile scaling, and modern “glassmorphism” aesthetic were tested to ensure a refined and intuitive interaction. Clear status badges (red for Lost, green for Found) were verified to provide immediate visual clarity to users.

8. Acceptance Testing. Finally, the system underwent alpha and beta acceptance testing. Alpha testing was conducted in a secure development environment to evaluate structural layout spacing and deployment script stability. Beta testing simulated a target user — a college student — interacting with the platform without technical documentation, confirming that standard workflows were intuitive and that the administrative panel was securely hidden from normal view.

Expected Outcomes of Testing

- No major structural bugs, infinite loops, or server crashes during standard execution.
- Secure image upload pipelines effectively block unauthorized or oversized file formats.
- Search and category filter queries return precise and accurate database results.
- Passwords and foreign-key relationships maintain strict data integrity and privacy.
- The application runs responsively and smoothly across both desktop and mobile browsers.

11. Industry / Client Interface

Are you doing this project for any Industry/Client? ☐ Yes ☒ No

This project is not being developed for any specific commercial industry or external corporate client. It is an academic-level project developed as part of the university curriculum for research and demonstration purposes. Its primary goal is to address a practical problem within a college campus environment by implementing modern web development and database management techniques.

12. Future Scope and Further Enhancements

The College Campus Lost & Found System has been designed as a highly effective, centralized digital platform for property recovery. However, with rapid technological advancements and evolving campus needs, there is significant potential for future growth and functional enhancements.

1. AI-Based Image Recognition and Matching. Currently, users manually search and filter items using keywords and categories. In the future, machine learning algorithms could be integrated to analyze uploaded images of “Found” items and automatically suggest matches to users who have posted similar “Lost” items, drastically reducing manual search efforts.

2. Automated Notifications and Alerts. Future versions can implement an automated alert system via email or SMS. Users could set up keyword alerts (for example “Dell Laptop” or “Student ID”), and the system would automatically push a notification to their devices the moment a matching item is posted on the platform.

3. University Single Sign-On (SSO) Integration. To streamline the registration process and guarantee that only active students and staff access the platform, the system can be integrated with the university’s official Single Sign-On infrastructure (such as Google Workspace or Microsoft Active Directory). This would eliminate the need for manual email verification.

4. In-App Secure Messaging. Currently, users provide contact information such as phone numbers directly on their posts. A critical future enhancement would be an internal, encrypted chat system. This would allow the finder and the loser to communicate and arrange meetups securely within the portal without exposing private contact details to the public.

5. Interactive Campus Mapping. Integrating geolocation APIs such as the Google Maps API would allow users to drop a digital pin exactly where an item was lost or discovered. A visual campus map showing hotspots for lost items could also help users visually retrace their steps.

6. Native Mobile Application Development. While the current web application is fully mobile-responsive, developing dedicated native mobile applications for iOS and Android — using frameworks such as Flutter or React Native — would allow for real-time push notifications and direct smartphone camera integration, making the posting process even faster.

7. Dedicated Campus Security Module. The system could be expanded to include a specific “Security Desk” role. Physical items handed over to various campus security checkpoints could be barcoded and logged directly into the system by guards, bridging the gap between digital user reporting and physical institutional storage.

Conclusion

The College Campus Lost & Found System has a highly promising future. By transitioning a disorganized, manual tracking process into a secure, centralized web application, the project immediately solves a major logistical challenge for the student body. With proposed future upgrades such as AI matching, secure in-app messaging, and SSO integration, the system can easily evolve from a basic registry into a comprehensive, automated campus property management ecosystem. Ultimately, this project fosters a culture of honesty and community support while demonstrating the practical, problem-solving power of modern web development and relational database management.

13. Bibliography

1. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed., pp. 45–120). O'Reilly Media. — This book provided the foundational knowledge for structuring the Python Flask backend, implementing routing, and utilizing the Jinja2 templating engine.
2. Korth, H. F., Silberschatz, A., & Sudarshan, S. (2011). *Database System Concepts* (6th ed., pp. 150–172). McGraw-Hill Education. — Used for understanding relational database design, foreign-key relationships, and the data normalization techniques applied in the MySQL database of the tracking system.
3. Sommerville, I. (2015). *Software Engineering* (10th ed., pp. 55–90). Pearson Education. — Supported the methodology of the System Development Life Cycle, requirement gathering, and software project management processes followed in this project.
4. Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (7th ed., pp. 200–225). McGraw-Hill Education. — Used as a primary reference for establishing the rigorous Quality Assurance and testing strategies, including unit testing, integration testing, and black-box testing phases.
5. *Official Flask Documentation* (2023). Pallets Projects. Retrieved from <https://flask.palletsprojects.com/> — Provided essential, up-to-date coding references and implementation guidelines for Python-based modular routing, session management, and application factory setups.
6. *Flask-SQLAlchemy Documentation* (2023). Retrieved from <https://flask-sqlalchemy.palletsprojects.com/> — Helped in translating raw SQL queries into Object Relational Mapping models for secure and efficient database manipulation.
7. *Werkzeug Documentation* (2023). Pallets Projects. Retrieved from <https://werkzeug.palletsprojects.com/> — Consulted heavily for implementing the `werkzeug.security` cryptographic password hashing and the `secure_filename()` utility to safely handle uploaded item images.
8. W3Schools (2023). *HTML, CSS, and JavaScript Tutorials*. Retrieved from <https://www.w3schools.com/> — Provided practical syntax references and techniques

for designing the frontend UI, structuring forms, and implementing responsive grid layouts.

9. Stack Overflow (2023–2024). *Developer community discussions*. Retrieved from <https://stackoverflow.com/> — Provided real-world coding solutions, debugging assistance, and troubleshooting strategies during the backend integration and testing phases.

PART B: PROJECT REPORT

Part B documents the system as actually built. Every implementation detail, schema definition, code listing and test result that follows was taken from, or measured against, the delivered source code.

CHAPTER 1: INTRODUCTION & BACKGROUND

1.1 Title & Introduction

1.1.1 Title of the Project

“College Campus Lost & Found System”

1.1.2 Introduction

Managing lost and found items has always been a persistent challenge within college and university campuses. In a fast-paced academic environment, students frequently misplace valuable belongings such as electronic devices, study materials, identity cards and personal accessories. In the absence of a centralized and accessible tracking mechanism, most individuals face considerable difficulty either in recovering what they have lost or in returning what they have found to its rightful owner. Traditional methods — physical notice boards, verbal enquiries at the gate, or handing an item over to a security desk — consistently fail to deliver timely communication or campus-wide visibility.

The present project addresses that gap through a digital, technology-driven solution. The **College Campus Lost & Found System** is a comprehensive, secure and user-friendly web-based application that centralizes the reporting and tracking of misplaced items across an institution. Unlike a physical log book, the system provides a permanent, searchable and continuously updated record in which recovery statuses are reflected in real time.

By combining secure relational data management, authenticated user sessions and an intuitive interface, the application acts as a unified recovery hub for the campus community. Every reported article is classified into a definite category — Electronics, Books, Accessories or Others — and is exposed to advanced search and filtering facilities so that a lost-item report can be matched quickly against a found-item report. The platform additionally incorporates a moderated administrative layer, so that while students collaborate freely, appointed administrators are able to supervise the environment and safeguard user data.

The system is implemented as a server-rendered web application. The application layer is written in Python using the Flask 3.0.0 micro-framework; persistence is provided by a MySQL relational database accessed through the Flask-SQLAlchemy Object Relational Mapper; authenticated sessions are managed by Flask-Login; and the presentation layer is composed of HTML5, CSS3 and JavaScript ES6 templates rendered by the Jinja2 templating engine. Passwords are never stored in plain text — each credential is converted into a `pbkdf2:sha256` digest by the `werkzeug.security` module before insertion into the database.

Functionally, the deployed system exposes twelve application routes covering public browsing of the item feed, user registration and authentication, item submission with optional photographic evidence, per-item detail views, owner-restricted status transitions, and a protected administrative area comprising a live metrics dashboard, item moderation and user management. Two relational entities underpin the entire application: a `user` table and an `item` table, joined by a one-to-many relationship.

This project belongs to the **Web Application Development and Relational Database Management System (RDBMS)** category, since it integrates web-based data entry with secure, normalized relational storage. It is not merely a digital registry; it is a secure, collaborative recovery infrastructure with the potential to save students considerable time and anxiety, to encourage honesty within the student body, and to ensure the efficient return of valuable personal belongings.

1.2 Objectives of the System

The objectives of the College Campus Lost & Found System are framed around the practical, day-to-day difficulties that members of a campus community face when attempting to recover personal property. Each objective below is realised by a specific, verifiable component of the implemented system.

1. Centralized Item Tracking. To develop a unified digital repository that replaces scattered physical logs and fragmented group-chat messages, allowing any authenticated member of the campus to post, browse and monitor lost and found articles from a single location. In the implementation this is satisfied by the `item` relational table together with the public feed route, which lists every reported article in reverse-chronological order of the date on which it was reported.

2. Efficient Search and Filtering. To implement robust keyword searching together with category-based and status-based filtering, so that a user can narrow a growing dataset down to the specific article being sought instead of scrolling through an undifferentiated chronological feed. The implementation composes a database query progressively from the request parameters, applying a case-insensitive partial-match comparison against both the item title and the item description, and applying equality constraints for the selected category and status.

3. Dynamic Status Management. To allow the person who reported an article to update its recovery state — *Lost*, *Found* or *Claimed* — so that the feed always reflects the true, current inventory of the campus and users are not misled into enquiring about property that has already been returned. The implementation restricts this transition to the original author of the record and accepts only the three permitted status values.

4. Secure Image and Data Handling. To ensure that uploaded photographs are validated by format and by size and are stored safely on the server, and that all textual data is committed through a parameterized data-access layer. The implementation enforces an extension whitelist limited to `.png`, `.jpg` and `.jpeg`, caps the size of any request body at five megabytes, sanitizes every submitted filename, and prefixes it with a timestamp so that two uploads bearing the same original name cannot overwrite one another.

5. Role-Based Moderation and Administration. To provide a protected administrative area with elevated privileges, enabling campus staff to supervise the platform, review aggregate activity, and remove fraudulent postings or offending accounts in order to preserve the integrity of the service. The implementation defines two roles, `student` and `admin`, and guards every administrative route with a custom authorization decorator that intercepts the request before the view function is executed.

6. Modern and Responsive Interface. To construct an intuitive, visually coherent frontend using contemporary design principles, so that the platform is comfortable to use on both a laptop and a mobile handset without any prior training. The implementation adopts a dark “glassmorphism” visual treatment, colour-coded status badges, and a responsive card grid, all inherited from a single master template.

1.2.1 Consolidated Statement of Purpose

Taken together, these objectives serve one overarching purpose: to foster a supportive, honest and connected campus community by minimising the friction involved in recovering lost property. By combining secure relational data handling with an accessible web interface, the project converts a disorganised manual process into a dependable institutional service.

1.3 Project Category

This project falls under the category of **Web Application Development and Relational Database Management System (RDBMS)**.

The classification is justified on two grounds. First, the deliverable is a complete client–server web application: it serves HTTP requests, renders dynamic HTML documents from server-side templates, accepts multipart form submissions including binary image uploads, and maintains authenticated sessions across requests. Second, all persistent state is held in a normalized relational schema managed by MySQL Server, accessed exclusively through an Object Relational Mapper rather than through concatenated SQL strings, which guarantees both referential integrity between the two entities and immunity from injection through the parameterized query interface.

The technology stack that gives effect to this categorization is summarised below, and every version stated is the version actually declared in the project’s dependency manifest.

Layer	Technology
Application framework	Python 3.x with Flask 3.0.0
Object Relational Mapper	Flask-SQLAlchemy 3.1.1
Session and authentication	Flask-Login 0.6.3
Security and WSGI utilities	Werkzeug 3.0.1
Database driver	PyMySQL 1.1.0
Database server	MySQL Server 8.0 or later
Templating engine	Jinja2

Presentation layer	HTML5, CSS3, JavaScript ES6
Automated testing	Python <code>unittest</code>

1.4 Literature Review

A review of the practices currently in use on campuses, and of the standard literature on web application and database engineering, establishes both the need for this system and the technical basis on which it has been built.

1.4.1 Review of Existing Practice

Physical notice boards. The prevailing method of announcing a lost or found article is a handwritten or printed notice affixed to a departmental board. Such a notice is visible only to those who physically pass the board and who happen to read it during the short period before it is covered by later notices. Its reach is therefore a small fraction of the campus population, and its content cannot be searched, sorted or filtered in any way.

Security-desk drop-off. Articles recovered by staff or by conscientious students are commonly deposited at a security post or an administrative office. Because there is no published index of what has been deposited, the owner has no means of discovering that the article is being held, and recovered property accumulates in store rooms while the owners assume the loss to be permanent.

Instant-messaging groups. Class or hostel messaging groups are increasingly used to circulate lost-item appeals. This is faster than a notice board but suffers from three defects: the message reaches only the members of that one group; the chronological message stream offers no organised filtering, so a student searching for a lost laptop cannot isolate electronics reports from unrelated conversation; and there is no mechanism for withdrawing or amending an appeal once the article has been recovered, so stale appeals continue to attract enquiries.

Common shortcomings. Across all three practices the same four deficiencies recur: absence of centralized visibility, absence of searchability, absence of real-time status tracking, and absence of moderation or privacy control over the contact details that appellants are obliged to publish.

1.4.2 Review of Technical Literature

The engineering decisions taken in this project are grounded in established texts and in the official documentation of the frameworks employed.

Grinberg's *Flask Web Development* provides the structural pattern adopted for the application layer — the organisation of view functions and URL rules, the use of the Jinja2 templating engine to separate presentation from logic, and the handling of user sessions through the Flask-Login extension. The application's routing, template inheritance and flash-messaging conventions follow the practice described there.

Korth, Silberschatz and Sudarshan's *Database System Concepts* supplies the theory of relational design and normalization applied to the two-table schema. The separation of

user identity from reported items, the placement of the owning user's identifier as a foreign key on the item record, and the elimination of repeating and transitively dependent attributes follow directly from the normal-form analysis presented in that text.

Sommerville's *Software Engineering* and Pressman's *Software Engineering: A Practitioner's Approach* inform, respectively, the choice of an iterative Agile development life cycle for a project of this size and the structure of the quality-assurance activity, including the progression from unit testing of individual functions through integration testing of module hand-offs to end-to-end system testing.

The official documentation of Flask, Flask-SQLAlchemy and Werkzeug was used throughout implementation. In particular, the `werkzeug.security` password-hashing interface and the `secure_filename` utility — both central to the security posture of this system — are applied in the manner prescribed by the Werkzeug documentation rather than through any custom cryptographic or sanitisation code.

1.4.3 Identified Research Gap

The literature and the existing practice together identify a clear gap. The technical means of building a searchable, authenticated, moderated repository of campus property are mature, freely available and well documented; yet campus lost-and-found handling remains almost entirely manual, unsearchable and unmoderated. The gap is therefore not one of technology but of application: no lightweight, institution-scoped system exists that couples a searchable item repository with author-controlled status transitions and administrative moderation. The present project is directed precisely at that gap.

1.5 QR Link of Project

The complete source code of the application, including the Flask application module, the database provisioning script, all Jinja2 templates, the stylesheet, the client-side script and the automated test module, is maintained in the project repository. The Quick Response code reproduced below resolves to that repository so that the evaluator may inspect or execute the source directly.

[INSERT QR CODE IMAGE OF PROJECT REPOSITORY HERE]

Repository / demonstration URL: _____

Local execution. The application may also be executed on any evaluator's machine without network access. After the MySQL service has been started, the dependency set is installed from the manifest, the database is provisioned by executing the provisioning script, and the application is launched from the main module; the portal is then served on port 5000 of the local host. The precise commands are reproduced in Chapter 4, Section 4.5.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Problem Definition

In a busy college or university campus, students and staff frequently misplace personal belongings — identity cards, electronic devices, textbooks and accessories. When such an article is lost or found, the individual concerned typically depends on a physical notice board, hands the article over to one of several scattered security desks, or circulates an appeal through a fragmented instant-messaging group. None of these channels constitutes a system, and each of them fails in a characteristic way.

Analysis of the existing practice isolates four distinct deficiencies.

1. Lack of centralized visibility. A notice pinned to one bulletin board, or a message posted in a single class group, reaches only a small fraction of the campus population. The person who is holding the lost article and the person who lost it are, in the ordinary case, members of different departments, different hostels and different messaging groups, so the two never come into contact.

2. Absence of searchability. Manual registers and chat histories offer no organised retrieval. A student who has lost a Dell laptop cannot restrict the record to electronics, cannot search on the word “Dell”, and is instead obliged to read every entry in chronological order. As the volume of entries grows, the practical usefulness of the record falls.

3. No real-time status tracking. Once a manual notice has been posted there is no mechanism for amending it. Articles that have already been returned continue to attract enquiries because the original notice was never withdrawn, and the enquirer’s time as well as the owner’s is wasted.

4. Lack of moderation and security. Publishing a telephone number on an open notice board exposes personal contact information to the entire campus and to visitors. Equally, there is no administrative oversight of what is posted, so a fraudulent or mischievous claim cannot be removed by any responsible authority.

The cumulative consequence of these four deficiencies is that a large volume of recovered property simply accumulates in administrative store rooms, while the students who lost it assume that it is permanently gone.

The problem definition for this project therefore reads as follows.

“To develop a centralized, secure and intuitive web-based Lost & Found System that replaces inefficient manual tracking by providing real-time status updates, automated categorization and administrative moderation, so as to ensure the swift and secure recovery of misplaced campus property.”

This statement highlights the critical gap between disorganised physical reporting and a modern, accountable digital service, and it is the gap that the remainder of this report addresses.

2.2 Feasibility Study

Before development resources were committed, a feasibility study was conducted to establish that the proposed system was practicable along five independent dimensions. Each finding below is stated against the system as it has actually been built.

2.2.1 Technical Feasibility

The project is constructed entirely from established, widely adopted and freely available technologies. The application layer relies on Python and the Flask micro-framework; persistence is provided by MySQL Server, accessed through the Flask-SQLAlchemy Object Relational Mapper; and the presentation layer uses only standard HTML5, CSS3 and JavaScript. Cryptographic password handling and filename sanitisation are delegated to the `werkzeug.security` and `werkzeug.utils` modules rather than to any bespoke implementation, which removes the principal source of technical risk in a student-built system.

No specialised or proprietary hardware is required. The server component executes on any ordinary desktop or laptop computer, and the client component requires only a modern web browser. The database is provisioned automatically by a short setup script, and the application creates its own tables on first execution. The project is accordingly assessed as fully technically feasible.

2.2.2 Economic Feasibility

Every component of the development toolchain — Python, Flask, MySQL, the browser, the code editor and the testing framework — is open-source or free of licence cost. Development was carried out on hardware already available to the candidate, so no capital expenditure was incurred. For institutional deployment, the application can be hosted either on an existing campus server or on the free tier of a commercial platform-as-a-service provider. Set against the value of the property that the system is designed to return to its owners, the financial overhead is negligible, and the project is therefore highly economically feasible.

2.2.3 Operational Feasibility

The system has been designed around ease of use. The interface adopts a clear, modern “glassmorphism” visual treatment with unambiguous navigation, colour-coded status badges and a consistent card layout, so that it is usable by members of the campus community with widely varying levels of technical expertise. A student can register, post an article with a photograph, search the feed and update a status without any training or instruction manual. The administrative dashboard likewise presents aggregate counts and moderation controls in a single screen. Because the system replaces an existing manual practice rather than introducing an unfamiliar one, operational acceptance is expected to be straightforward, and the project is assessed as operationally feasible.

2.2.4 Legal Feasibility

The system stores personal information — names, campus email addresses and contact details — and therefore attracts data-protection obligations. Compliance is addressed structurally. User passwords are never retained in plain text; each is transformed into a pbkdf2:sha256 digest before insertion, so that even a full disclosure of the database does not disclose any credential. Role-Based Access Control confines administrative operations, including the deletion of accounts and postings, to authorised administrators. Uploaded files are constrained by an extension whitelist and a size limit, which prevents the store from being used to distribute arbitrary content. The publication of contact details is at all times a voluntary act of the person posting the article. On this basis the project is legally safe and feasible for institutional deployment, subject to the hardening measures recorded in Chapter 4, Section 4.6.5.

2.2.5 Schedule Feasibility

The development life cycle was mapped over an eight-week timeline divided into eight tasks, from requirement gathering through to final documentation and deployment, as set out in the project schedule reproduced in the Synopsis and revisited in Section 2.3 below. The estimate proved realistic: each phase was completed within its allotted window by a single developer, and no task overran to the extent of displacing a subsequent one. The schedule is therefore assessed as feasible.

2.2.6 Conclusion of the Feasibility Study

The study establishes that the College Campus Lost & Found System is technically achievable with mature tools, economically almost costless, operationally acceptable to its intended users, legally defensible in its handling of personal data, and deliverable within the eight-week academic window available.

2.3 Methodology

2.3.1 Choice of Development Model

The **Agile methodology** was adopted for this project. Agile permits iterative development, continuous self-review and the incorporation of improvements while construction is still in progress — properties that suit a single-developer academic project in which the requirements become clearer as the interface takes shape. A strictly sequential waterfall model was rejected because it would have deferred all validation of the user interface until after the backend was frozen.

2.3.2 Implementation Phases

1. Requirement Analysis. Requirements were gathered by examining how students and administrators presently record, search for and manage lost property. The requirements identified were secure authentication, detailed item logging with image upload, keyword and category retrieval, author-controlled status transitions, and a moderation dashboard for administrative oversight.

2. System Design. The application architecture was mapped through Data Flow Diagrams, an Entity Relationship Diagram and Unified Modeling Language diagrams before any code was written. This phase fixed the two-entity relational schema, the module boundaries and the navigation flow. Its output is presented in full in Chapter 3.

3. Database Implementation (RDBMS). A MySQL relational database was established to hold registered user details, hashed passwords, item attributes and the ownership relation between them. The Flask-SQLAlchemy Object Relational Mapper was introduced to abstract all database operations, so that persistence is expressed as operations on Python model classes rather than as assembled SQL text. The schema was normalized to eliminate redundancy.

4. Backend Application Development. The server-side logic was written in Python with the Flask framework. This phase produced the twelve application routes, the Flask-Login session integration, the `pbkdf2:sha256` credential pipeline, the progressive search-query composition, the file-upload validation chain and the custom authorization decorator that separates student behaviour from administrator behaviour.

5. Frontend Application Development. The client interface was developed in HTML5, CSS3 and JavaScript ES6. The Jinja2 templating engine renders database content into a master template from which every page inherits, producing the responsive item-card grid, the colour-coded status badges and the conditional interface elements that appear or disappear according to the role of the current user.

6. Testing and Debugging. Unit, integration and system testing were carried out, with particular attention to form validation, image-upload restriction, search logic and authorization barriers. An automated `unittest` module was written so that the core behaviours can be re-verified on demand. The results are tabulated in Chapter 5.

7. Deployment and Maintenance. The packaged system is deployed with the database instantiated automatically by the provisioning script, after which the application creates its own tables and its initial administrator account on first execution. Maintenance provisions are discussed in Chapter 4, Section 4.5.

2.3.3 Project Schedule

The following schedule governed the eight-week development window. Overlapping start weeks reflect the iterative nature of the Agile model — backend and frontend construction proceeded concurrently once the database was in place.

Task ID	Task Name	Duration	Start Week
T1	Requirement Gathering & Analysis	1 Week	Week 1
T2	System Design (DFD, ERD, DB Schema)	1 Week	Week 2
T3	Coding: Database Creation & Setup	1 Week	Week 3
T4	Coding: Backend (Auth, Routing, Logic)	2 Weeks	Week 3
T5	Coding: Frontend Development (UI/UX)	2 Weeks	Week 4
T6	System Integration (Connecting UI with DB)	1 Week	Week 6

T7	System Testing (Unit & System Testing)	1 Week	Week 7
T8	Final Documentation & Deployment	1 Week	Week 8

The assumptions underlying this schedule are that the project is undertaken by a single developer handling design, coding and testing; that the methodology is iterative; and that the scope comprises user authentication, item management, search and filtering, and the administrative dashboard. The corresponding PERT and Gantt representations appear in the Synopsis and are elaborated in Chapter 3.

2.4 System Requirement Specification (SRS)

The requirements of the system are stated below under four heads: the software environment in which it executes, the hardware on which that environment runs, the functional requirements that define what the system does, and the non-functional requirements that define the qualities it must exhibit while doing it.

2.4.1 Software Requirements

Category	Specification
Operating System	Windows 10/11, Linux Ubuntu 20.04+, or macOS 10.15+
Programming Languages	Python 3.8+ (backend), JavaScript ES6 (frontend)
Frameworks & Libraries	Flask 3.0.0, Flask-SQLAlchemy 3.1.1, Flask-Login 0.6.3, Werkzeug 3.0.1, Jinja2
Database Driver	PyMySQL 1.1.0 (with the <code>cryptography</code> package for MySQL 8 authentication)
Database	MySQL Server 8.0+
Frontend Technologies	HTML5, CSS3, JavaScript ES6, FontAwesome (interface icons)
Web Server	Werkzeug development server (development) / Nginx or Apache with a WSGI server (production)
IDE and Tools	VS Code or PyCharm, MySQL Workbench or XAMPP
Testing Tools	Python <code>unittest</code> framework
Browser Support	Google Chrome, Mozilla Firefox, Safari, Microsoft Edge

The framework versions stated above are those pinned in the project's dependency manifest, so an evaluator installing from that manifest obtains exactly the environment in which the system was developed and tested.

2.4.2 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	Dual Core, 2.0 GHz	Intel i5/i7 or AMD Ryzen Quad Core, 3.0 GHz+
RAM	4 GB	8 GB or higher

Storage (HDD/SSD)	250 GB HDD or 128 GB SSD	500 GB SSD
Display	1366 × 768 resolution	1920 × 1080 Full HD
Network	Stable connection, 2 Mbps minimum	High-speed broadband, 10 Mbps or higher
Power Supply	Standard 65 W adapter	UPS backup for continuous server operation
Devices Supported	Desktop / Laptop / Smartphone via the responsive interface	Desktop, Laptop, Tablet, Smartphone

Because uploaded photographs are retained on the server's file system, storage consumption grows with usage; the recommended solid-state configuration is advised for any deployment expected to accumulate a substantial image archive.

2.4.3 Functional Requirements (FR)

Each functional requirement below is traced to the application route that implements it, so that every stated requirement can be verified directly against the source code presented in Chapter 4.

ID	Functional Requirement	Implementing Route	Access
FR-01	The system shall display all reported items in reverse-chronological order of the date reported.	GET /	Public
FR-02	The system shall retrieve items by keyword, matching the keyword partially and without regard to case against both the item title and the item description.	GET /?search=	Public
FR-03	The system shall filter items by category, the permitted categories being Electronics, Books, Accessories and Others.	GET /?category=	Public
FR-04	The system shall filter items by status, the permitted statuses being Lost, Found and Claimed.	GET /?status=	Public
FR-05	The system shall register a new user, rejecting any registration whose email address is already present in the database.	POST /register	Public
FR-06	The system shall convert every submitted password into a <code>pbkdf2:sha256</code> digest before storage, and shall not retain the password in any recoverable form.	POST /register	Public
FR-07	The system shall authenticate a user by comparing the submitted password against the stored digest, and on success shall	POST /login	Public

	establish a server-side session.		
FR-08	The system shall terminate the session of the current user on request.	GET /logout	Authenticated
FR-09	The system shall accept a new item report comprising title, description, category, status, location and contact information, all of which are mandatory.	POST /item/add	Authenticated
FR-10	The system shall optionally accept one photograph per item, shall admit only the extensions .png, .jpg and .jpeg, shall reject any request body exceeding five megabytes, and shall store the file under a sanitised, timestamp-prefixed name that cannot collide with an existing file.	POST /item/add	Authenticated
FR-11	The system shall present the full detail of any single item, and shall respond with HTTP 404 where the requested identifier does not exist.	GET /item/<id>	Public
FR-12	The system shall permit a change of item status only where the requesting user is the author of that item, and shall accept only the three permitted status values.	POST /item/<id>/update_status	Owner only
FR-13	The system shall present to an administrator the live aggregate counts of registered users, total items, and items presently Lost, Found and Claimed.	GET /admin/dashboard	Administrator
FR-14	The system shall present to an administrator a complete list of all reported items for moderation.	GET /admin/items	Administrator
FR-15	The system shall permit an administrator to delete any item from the platform.	POST /admin/delete_item/<id>	Administrator
FR-16	The system shall present to an administrator a complete list of registered users, and shall permit the deletion of a user together with all items reported by that user, while refusing any attempt by an administrator to delete his or her own account.	GET /admin/users, POST /admin/delete_user/<id>	Administrator
FR-17	The system shall create its own database tables and a single default administrator account on first execution, so that the platform is usable immediately after installation.	Application start-up	System

2.4.4 Non-Functional Requirements (NFR)

ID	Category	Non-Functional Requirement
NFR-01	Security — data at rest	No password shall be stored in plain text. Credentials shall be held only as <code>pbkdf2:sha256</code> digests, so that disclosure of the database does not disclose any usable credential.
NFR-02	Security — file handling	Every uploaded filename shall be sanitised before use, so that a submitted name cannot direct a write outside the designated upload directory, and the upload directory shall accept only whitelisted image extensions.
NFR-03	Authorization integrity	Privileged operations shall be unreachable by URL manipulation. Administrative routes shall be guarded before the view function executes, and item mutation shall be permitted only to the author of the item.
NFR-04	Data integrity	Every item record shall reference an existing user through a foreign key that is not permitted to be null, so that no item can exist without an identifiable author.
NFR-05	Usability	Any member of the campus community shall be able to register, report an item and search the feed without training. System outcomes shall be communicated by visible confirmation or error messages rather than by silent failure.
NFR-06	Performance	Retrieval of a single item shall be performed by primary-key lookup, and the filtered feed shall be produced by one composed database query rather than by fetching all rows and filtering in the application layer.
NFR-07	Portability	The system shall execute on Windows, Linux and macOS without modification, and shall render correctly on the four supported browsers and on both desktop and mobile viewport widths.
NFR-08	Maintainability	Presentation shall be separated from logic by template inheritance from a single master template, and all persistence shall pass through the Object Relational Mapper rather than through assembled SQL strings, so that the schema can be altered in one place.

2.4.5 Assumptions and Constraints

The system assumes that a MySQL service is reachable on the host on which the application is executed, and that the person reporting an article supplies contact information voluntarily and accurately. It is constrained to a single photograph per item, to a five-megabyte request body, and to the four fixed categories and three fixed statuses enumerated above; any extension of those enumerations is a schema change rather than a configuration change. Matching a lost report to a found report is performed by the user through search and filtering; the system does not attempt automated matching. Prospective removal of these constraints is discussed in Chapter 6.

CHAPTER 3: SYSTEM DESIGN & DIAGRAMS

The preceding chapter established *what* the system is required to do. The present chapter establishes *how* that requirement set is realised as a structured design. Every diagram, table and design decision recorded here corresponds directly to a construct that exists in the delivered source code; no speculative or aspirational component is represented.

Each figure in this chapter is reproduced in schematic form. The candidate may substitute a graphically rendered version of the same figure at the indicated placeholder without altering the accompanying description.

3.1 Analysis Diagrams

3.1.1 Data Flow Diagrams

A Data Flow Diagram models the system as a network of transforming processes connected by flows of data, rather than as a sequence of program statements. Three levels of decomposition are presented: the context level, which fixes the system boundary; the first level, which identifies the four principal processes; and the second level, which explodes the item-management process into its constituent steps.

3.1.1.1 Level 0 — Context Diagram

At the context level the entire application is treated as a single process. Two external entities interact with it: the **Student**, who is any authenticated member of the campus community, and the **Administrator**, who is a privileged moderator. The diagram fixes the system boundary and demonstrates that no third-party service, external payment gateway or remote data provider participates in the design.

[INSERT RENDERED LEVEL 0 CONTEXT DIAGRAM HERE]

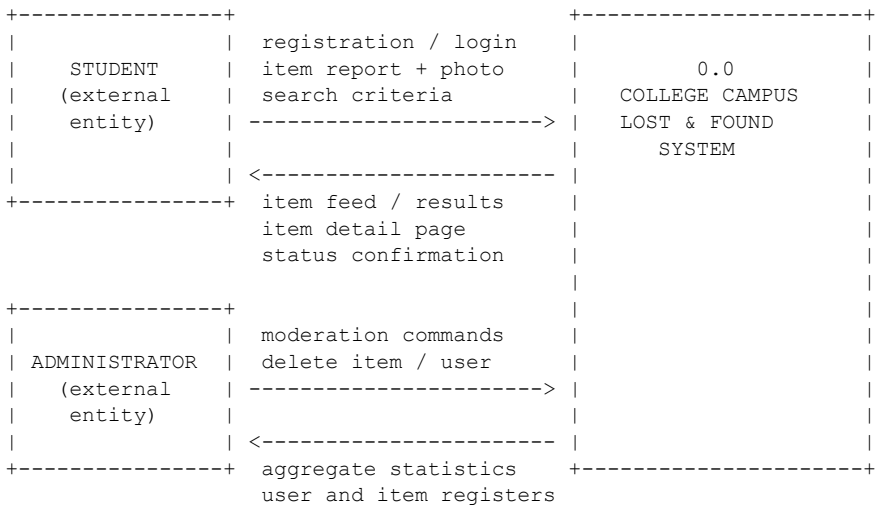


Figure 3.1 — Level 0 (Context) Data Flow Diagram

3.1.1.2 Level 1 — Principal Processes

The first level of decomposition partitions process 0.0 into four processes and two data stores. The data stores correspond exactly to the two relational tables created by the Object Relational Mapper, and a third, non-relational store represents the server file system directory in which uploaded photographs are written.

[INSERT RENDERED LEVEL 1 DATA FLOW DIAGRAM HERE]

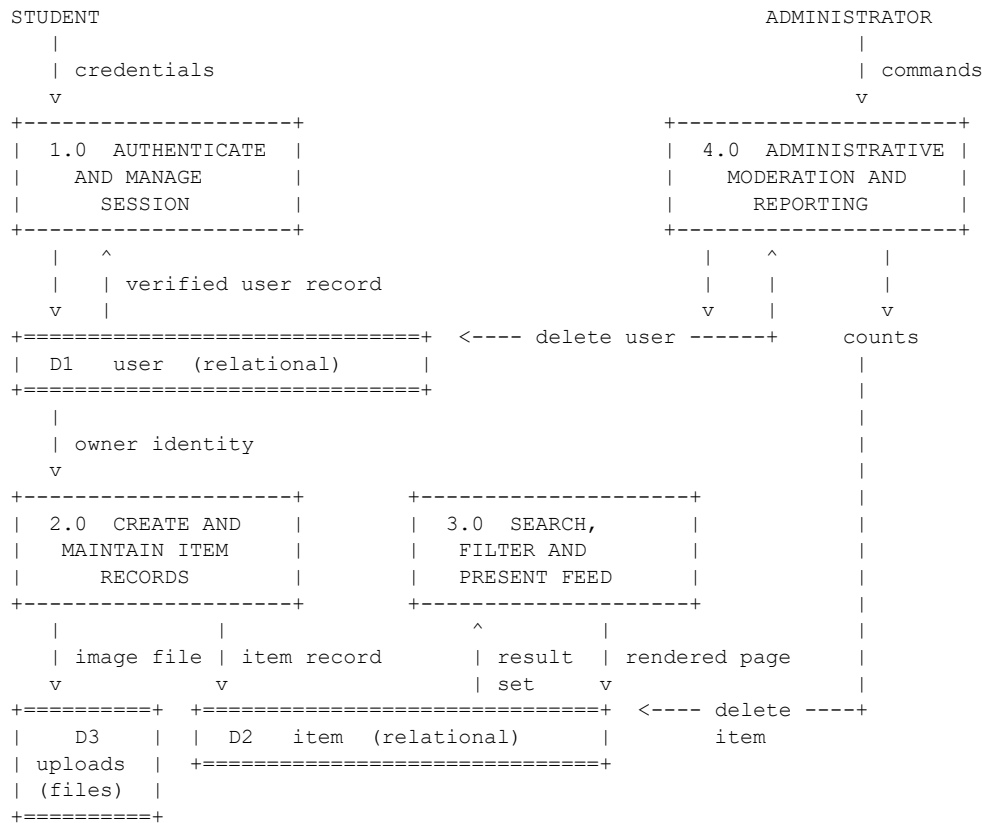


Figure 3.2 — Level 1 Data Flow Diagram

The four processes map onto the implementation as follows.

Process	Responsibility	Implementing routes
1.0 Authenticate and manage session	Registration, credential verification, session establishment and termination	/register, /login, /logout
2.0 Create and maintain item records	Validated insertion of a new report, photograph storage, owner-restricted status transition	/item/add, /item/<id>/update_status
3.0 Search, filter and present feed	Progressive composition of the retrieval query and	/, /item/<id>

	rendering of the feed and detail views	
4.0 Administrative moderation and reporting	Aggregate statistics, item register, user register, forced deletion	/admin/dashboard, /admin/items, /admin/users, /admin/delete_item/<id>, /admin/delete_user/<id>

The data stores are D1 user and D2 item, both relational tables in the lost_and_found_db schema, and D3 uploads, the server-side directory static/uploads in which validated photographs are written under timestamp-prefixed filenames.

3.1.1.3 Level 2 — Explosion of Process 2.0

Process 2.0 is the only process that performs a write to two dissimilar stores in a single transaction, and is therefore decomposed further. The explosion below reflects the actual order of operations in the item-creation handler: form fields are read first, the photograph is validated and written to disk second, and the relational record — containing the resulting filename — is committed last.

[INSERT RENDERED LEVEL 2 DATA FLOW DIAGRAM HERE]

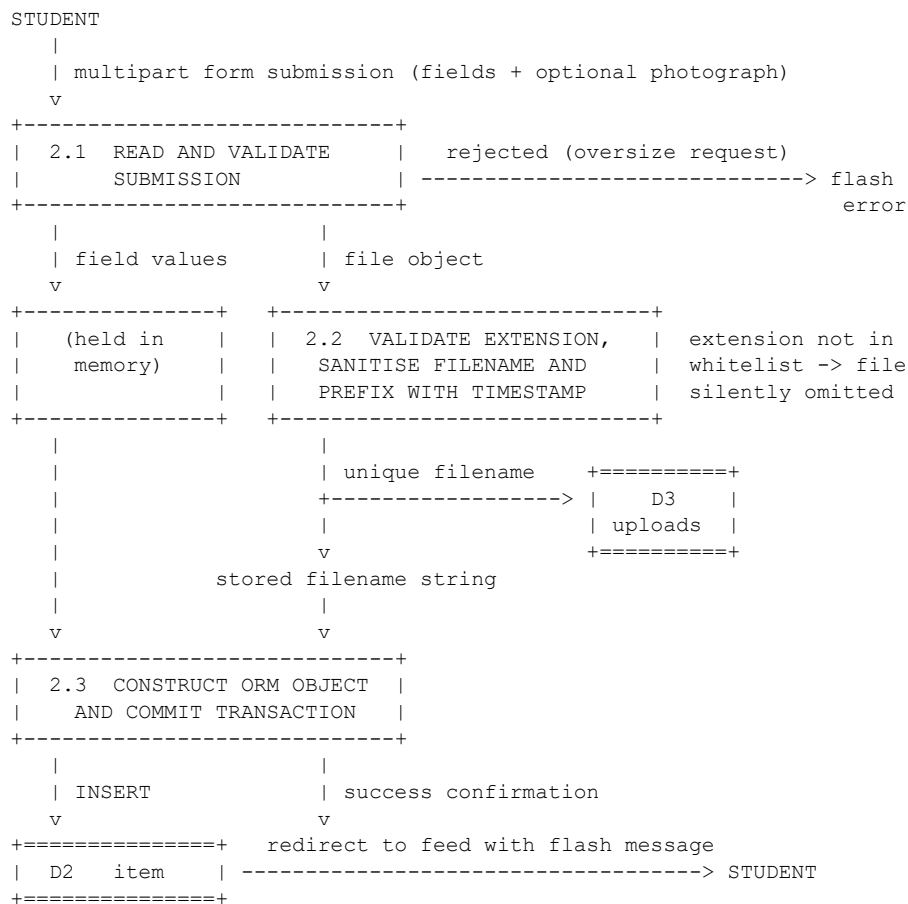


Figure 3.3 — Level 2 Data Flow Diagram (explosion of process 2.0)

Two design consequences of this ordering deserve note. First, because the photograph is written to disk before the database transaction is committed, a failure of the commit would leave an orphaned file on disk; the design accepts this because an orphaned image consumes storage but cannot corrupt the relational state. Second, a file whose extension is not on the whitelist does not abort the submission — the textual report is still accepted and the record is simply stored without a photograph, since the photograph is an optional attribute.

3.1.2 Entity Relationship Diagram

The persistent state of the system is held in two entities. A `user` represents a registered member of the campus community; an `item` represents a single lost-or-found report. One user may report many items; every item belongs to exactly one user. The relationship is therefore one-to-many, and is realised by placing the owning user’s primary key as a mandatory foreign key on the item record.

[INSERT RENDERED ENTITY RELATIONSHIP DIAGRAM HERE]

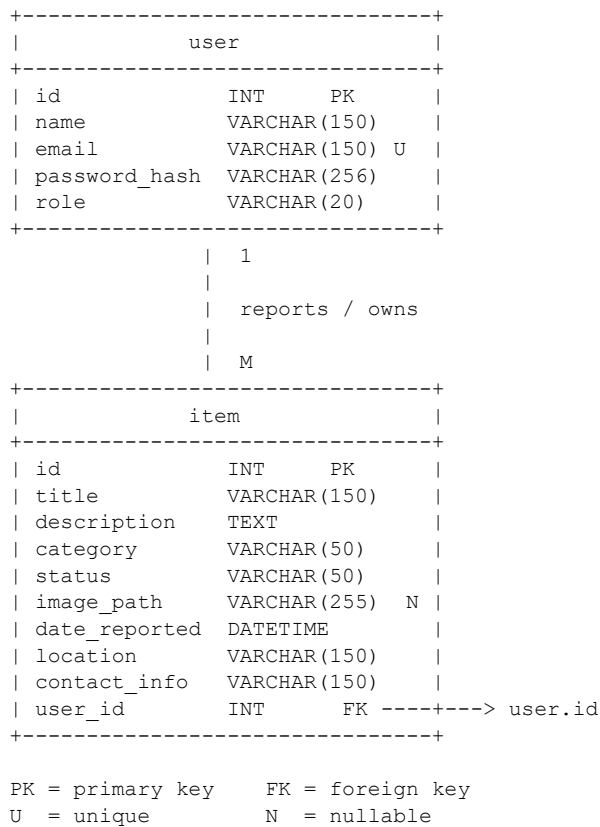


Figure 3.4 — Entity Relationship Diagram

The relationship is declared in the application layer as a collection attribute on the `user` model with a reverse reference named `owner` on the `item` model. This reverse reference is not merely a convenience: it is the mechanism by which the ownership check in the status-transition handler compares the identity of the requesting session against the identity of the record’s author.

3.1.3 Data Dictionary

The tables below document every column of the schema exactly as declared in the model definitions. No column exists in the database that is not listed here, and no column is listed here that is absent from the code.

Table 3.1 — Data dictionary for the `user` table

Field	Data type	Constraints	Description
<code>id</code>	Integer	Primary key, auto-increment	Surrogate identifier for the account; used as the session identity and as the referenced key of the item relationship.
<code>name</code>	Varchar(150)	Not null	Full name of the member, displayed as the reporter of an item.
<code>email</code>	Varchar(150)	Not null, unique	Login identifier. Uniqueness is enforced at the database level and re-checked in the registration handler before insertion.
<code>password_hash</code>	Varchar(256)	Not null	<code>pbkdf2:sha256</code> digest of the password produced by the Werkzeug security module. The plaintext password is never stored.
<code>role</code>	Varchar(20)	Not null, default 'student'	Authorization level. Two values are used: <code>student</code> and <code>admin</code> .

Table 3.2 — Data dictionary for the `item` table

Field	Data type	Constraints	Description
<code>id</code>	Integer	Primary key, auto-increment	Surrogate identifier for the report; appears in the detail-view and status-update URLs.
<code>title</code>	Varchar(150)	Not null	Short heading for the article, searched by the keyword filter.
<code>description</code>	Text	Not null	Extended description including distinguishing marks; also searched by the keyword filter.
<code>category</code>	Varchar(50)	Not null	Classification of the article. Four values are offered by the interface: Electronics, Books, Accessories, Others.
<code>status</code>	Varchar(50)	Not null	Recovery state of the article. Three values are permitted: Lost, Found, Claimed.
<code>image_path</code>	Varchar(255)	Nullable	Timestamp-prefixed filename of the stored photograph, relative to the uploads directory. Null when no photograph was supplied or when the supplied file failed

date_reported	DateTime	Default: current UTC timestamp	extension validation. Instant at which the report was created; the feed is ordered by this column in descending order.
location	Varchar(150)	Not null	Place at which the article was lost or recovered.
contact_info	Varchar(150)	Not null	Means by which the reporter may be contacted, supplied voluntarily by the reporter.
user_id	Integer	Not null, foreign key referencing user.id	Author of the report. Mandatory, so that no item can exist without an accountable owner.

3.1.4 UML Diagrams

3.1.4.1 Use Case Diagram

Two actors interact with the system. The Administrator inherits every capability of the Student and adds four privileged capabilities. One use case — browsing and searching the feed — is available even without authentication, since the public feed is deliberately readable by an unauthenticated visitor.

[INSERT RENDERED USE CASE DIAGRAM HERE]

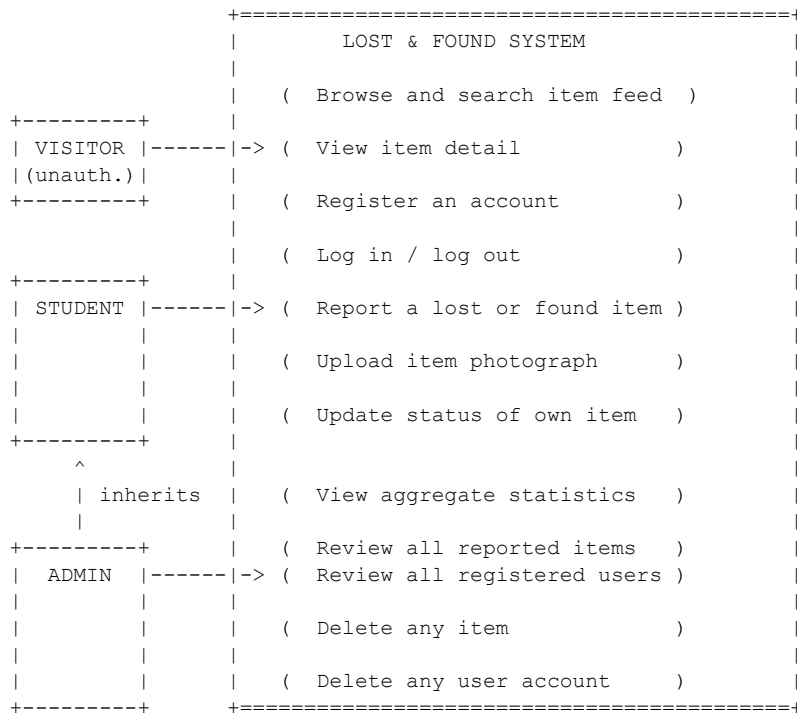


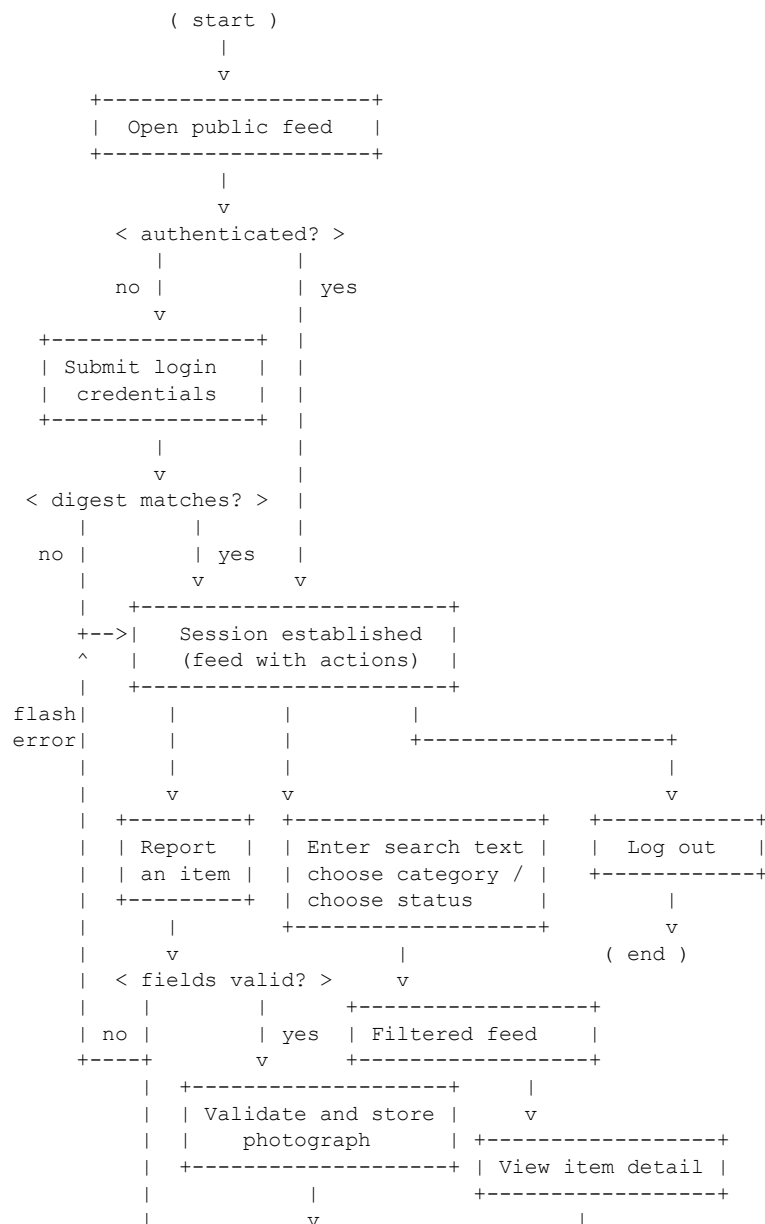
Figure 3.5 — Use Case Diagram

Two constraints on this diagram are enforced in code rather than merely documented. First, the *update status of own item* case is guarded by an ownership comparison, so a student cannot alter another student's report. Second, the *delete any user account* case excludes the acting administrator's own account, preventing an administrator from removing the only privileged account and locking the institution out of its own moderation interface.

3.1.4.2 Activity Diagram

The activity diagram traces the control flow of a complete student session, from the authentication gate through the two principal activities — reporting an article and searching for one — to termination of the session.

[INSERT RENDERED ACTIVITY DIAGRAM HERE]



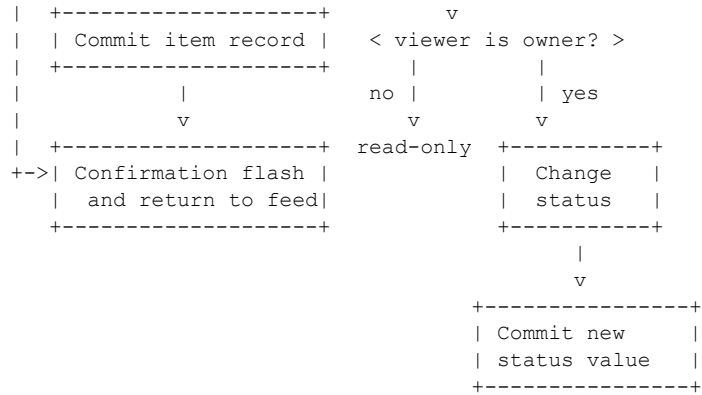
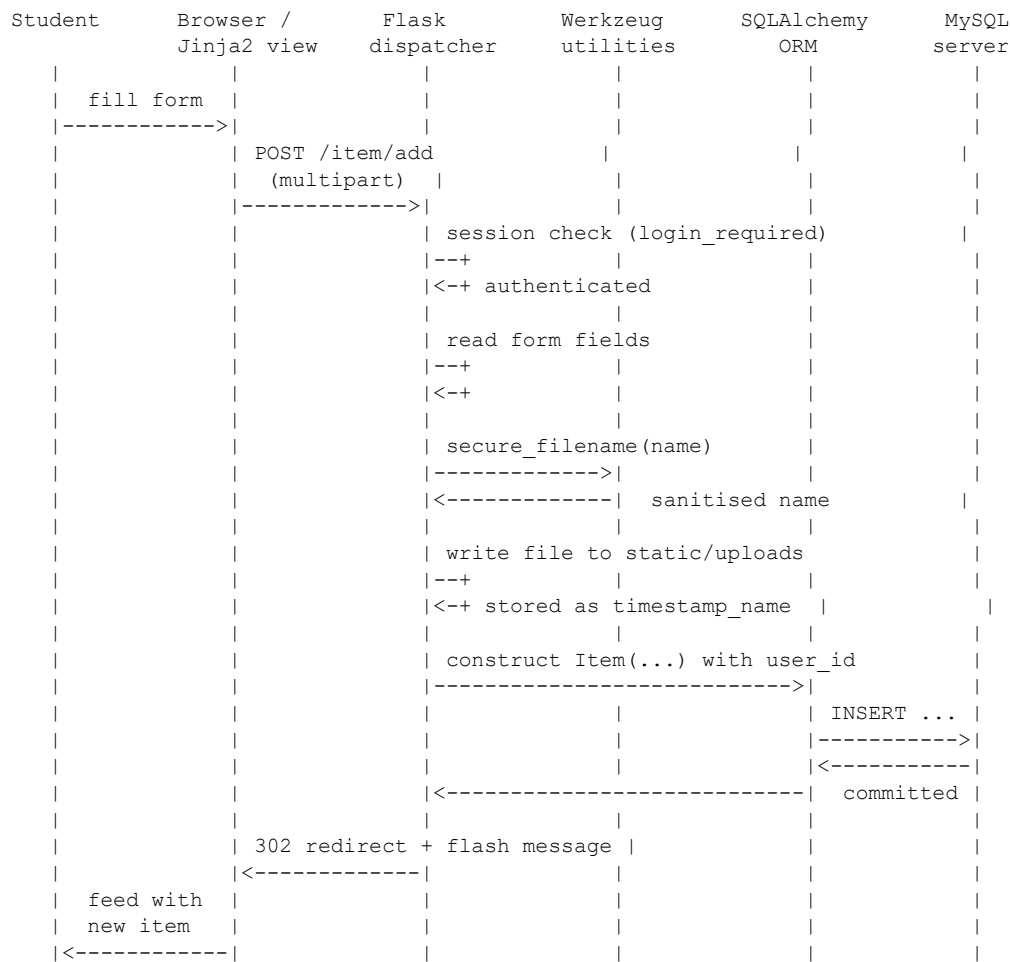


Figure 3.6 — Activity Diagram for a student session

3.1.4.3 Sequence Diagram

The sequence diagram below documents the interaction of the four participating layers during the most complex single transaction in the system: the submission of a new item report accompanied by a photograph.

[INSERT RENDERED SEQUENCE DIAGRAM HERE]



3.1.4.4 Class Diagram

[INSERT RENDERED CLASS DIAGRAM HERE]



3.2 System Design

3.2.1 Modularization

Module 1 — Authentication and Session Management. Comprises registration, login and logout. This module is the only one that writes to the `user` table during normal operation, and the only one that invokes the password-hashing and verification primitives.

Registration rejects a duplicate email address before insertion; login compares the submitted password against the stored digest and, on success, establishes a server-side session; logout destroys that session. None of the three handlers is available to an already-authenticated visitor in the case of registration and login, which prevents a logged-in user from silently creating or switching accounts.

Module 2 — Item Reporting and Media Handling. Comprises the item-creation handler. It is guarded by the authentication requirement, so an anonymous visitor cannot post. It is the only module that writes to the server file system, and it performs three defensive steps before doing so: an extension whitelist test, filename sanitisation, and prefixing with a timestamp to guarantee uniqueness.

Module 3 — Discovery: Feed, Search and Detail. Comprises the public feed and the item detail view. Neither handler is guarded, by design: the institutional purpose of the system is served by making the register of lost property readable by anyone on the campus network. The feed handler composes its retrieval query progressively from up to three independent request parameters and orders the result by report date, most recent first. The detail handler resolves an integer identifier and returns a not-found response if no such record exists.

Module 4 — Status Lifecycle Management. Comprises the single status-transition handler. It is guarded twice: once by the authentication requirement and again by an explicit comparison of the requesting user's identity against the identity of the record's owner. It further constrains the submitted value to the three permitted statuses, so an arbitrary string cannot be written into the status column even by a legitimate owner.

Module 5 — Administration and Moderation. Comprises the metrics dashboard, the item register, the user register and the two deletion handlers. Every handler in this module carries both the authentication requirement and the administrative authorization decorator. The dashboard computes five aggregate counts by direct database counting rather than by loading and counting rows in memory. The user-deletion handler removes the user's items before removing the user, so that no orphaned item can violate the mandatory foreign-key constraint, and refuses to delete the acting administrator's own account.

Module	Guards applied	Tables written	Handlers
1. Authentication and session	none / login required (logout)	<code>user</code>	3
2. Item reporting and media	login required	<code>item</code> + file system	1
3. Discovery: feed, search, detail	none (public)	none (read only)	2
4. Status lifecycle	login required + ownership check	<code>item</code>	1
5. Administration and moderation	login required + admin role	<code>user</code> , <code>item</code>	5

Table 3.3 — Functional modules and their guard conditions

3.2.2 Database Design and Normalization

The schema was designed to satisfy the third normal form. The progression through the normal forms is set out below, since a merely assertive claim of normalization would carry no evidential weight in an academic evaluation.

Unnormalized form. A naive design would record each report as a single row containing the reporter's name, the reporter's email, the reporter's password, and the article's attributes together. Such a row would repeat the reporter's identity for every article that reporter posts.

First normal form. First normal form requires that every attribute hold a single atomic value and that no repeating group be present. The delivered schema satisfies this condition: no column stores a list, a comma-separated set or a composite structure. In particular the article's classification is held as a single categorical value rather than as a multi-valued tag field, and the reporter's contact information is a single free-text string rather than a repeating group of contact methods.

Second normal form. Second normal form requires first normal form and, in addition, that every non-key attribute be functionally dependent on the whole of the primary key. Both tables use a single-column surrogate integer primary key. A composite key is therefore absent from the design, partial dependency on part of a key is impossible by construction, and the condition is satisfied trivially but genuinely.

Third normal form. Third normal form requires second normal form and, in addition, that no non-key attribute be transitively dependent on the primary key through another non-key attribute. This is the condition that motivated the two-table split. Had the reporter's name and email been carried on the item row, they would have depended on the item's identifier only through the reporter's identity — a transitive dependency, and one that would have produced update anomalies whenever a member changed their name, and insertion anomalies whenever a member had to be registered before having anything to report. By separating the member into its own entity and carrying only the owning member's key on the item row, every remaining non-key attribute of the item depends directly and solely on the item's own identifier.

Resulting integrity properties. The design yields four concrete guarantees. The uniqueness constraint on the email column prevents duplicate accounts for the same campus address. The mandatory foreign key prevents an item from existing without an accountable owner. Deletion of a member is preceded in code by deletion of that member's items, so the foreign-key constraint cannot be violated by a moderation action. And because all access is mediated by the Object Relational Mapper, every literal reaches the database as a bound parameter rather than as concatenated query text, which removes the injection surface entirely.

3.2.3 User Interface Design and Navigation Flow

Design language. The interface adopts a dark “glassmorphism” treatment: a deep background, translucent card surfaces with a blurred backdrop, restrained accent colours and generous spacing. The choice is functional as well as aesthetic — a dark surface reduces glare when the portal is consulted on a handset in a corridor or lecture hall, and the translucent card boundary makes each item report read as a discrete physical object. Status is communicated by a colour-coded badge on every card so that the recovery state of an article is legible without reading the body text.

Template hierarchy. Presentation is organised by template inheritance. A single master template defines the document skeleton, the stylesheet link, the navigation bar, the flash-message region and a content block; all eight remaining templates extend it and override only the page title and the content block. This yields a consistent chrome across the entire application from one source of truth.

```
base.html    (document skeleton, navigation bar, flash region, content block)
|
+-- index.html        public feed: search form, filters, card grid
+-- item_detail.html   single report, photograph, owner status control
+-- add_item.html      report submission form, multipart encoded
+-- login.html         credential form
+-- register.html      account creation form
+-- admin_dashboard.html five aggregate metric cards
+-- admin_items.html   tabular item register with delete action
+-- admin_users.html   tabular user register with delete action
```

Figure 3.9 — Template inheritance hierarchy

Conditional chrome. The navigation bar is rendered conditionally against the session state. An unauthenticated visitor is offered the feed, login and registration. An authenticated student is additionally offered the report-an-item action and logout. An administrator is further offered the dashboard link. The administrative entry point is therefore not merely protected on the server: it is invisible to those who may not use it, which reduces both accidental navigation and needless probing.

Navigation flow.

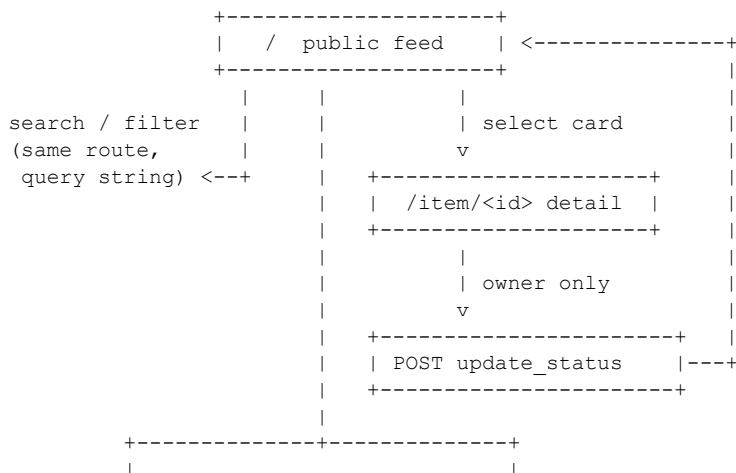




Figure 3.10 — Navigation flow between routes

[INSERT SCREENSHOT OF PUBLIC FEED PAGE HERE]

[INSERT SCREENSHOT OF ITEM DETAIL PAGE HERE]

3.3 High-Level System Architecture

A. Visual Architecture Diagram

The system follows the classical three-tier architecture. The presentation tier executes in the visitor's browser and is generated on the server by the templating engine; the application tier executes as a Python process and contains all business rules; the data tier comprises the relational server and the static file store.

[INSERT RENDERED THREE-TIER ARCHITECTURE DIAGRAM HERE]

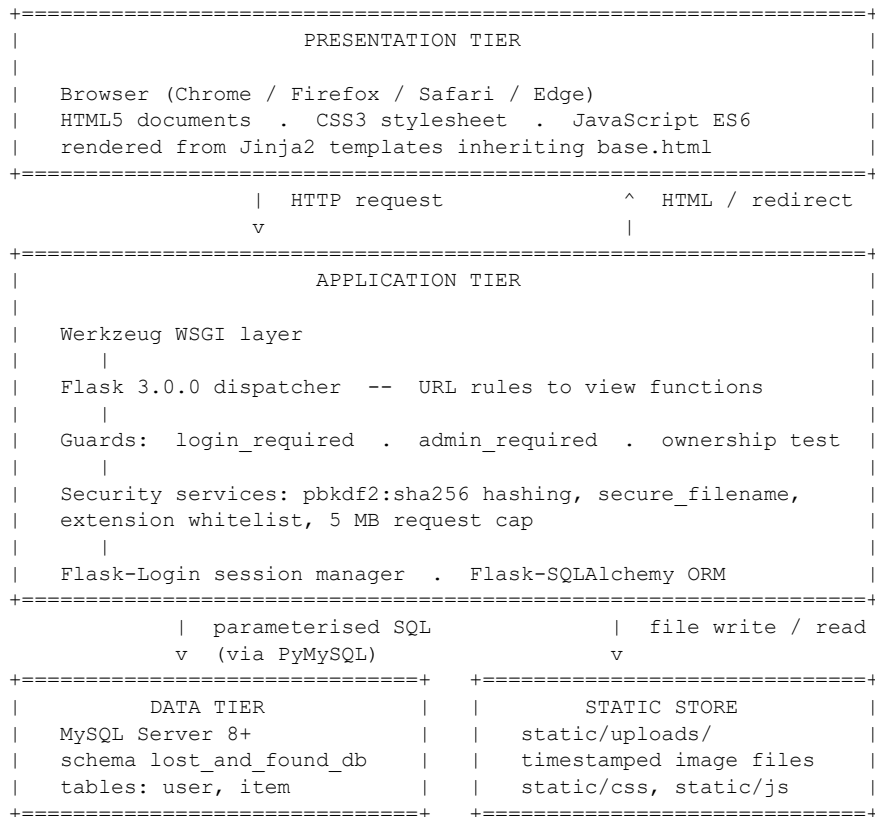


Figure 3.11 — Three-tier system architecture

B. Architectural Layers Breakdown

Presentation layer. Nine Jinja2 templates, one of which is the master template from which the other eight inherit. Markup is HTML5; visual treatment is supplied by a single stylesheet; a small client-side script provides interface conveniences only. No business rule is enforced in this layer — client-side behaviour is treated purely as a usability aid, and every constraint that matters is re-applied on the server.

Dispatch layer. The Flask application object holds the URL rules that bind request paths and methods to view functions. Typed converters in the two parameterised routes mean that a non-integer identifier is rejected by the routing layer before any view function is entered.

Security and authorization layer. Interposed between dispatch and business logic as a stack of decorators. The authentication decorator is supplied by the login extension; the administrative decorator is written for this project and inspects the role attribute of the current session identity, redirecting with a flash message rather than exposing the existence of the protected view. The ownership test is not a decorator but an in-handler comparison, because it depends on the record being addressed.

Business logic layer. The view functions themselves: query composition, validated insertion, status transition and aggregate counting.

Persistence layer. The Object Relational Mapper maps the two model classes onto the two tables and generates all statements. The application contains no hand-written SQL string; the only SQL text authored by hand in the entire project is the database-creation statement in the provisioning script, which contains no user-supplied value.

Data layer. MySQL Server holds the schema. The tables are created on first execution by the mapper's schema-creation call, and a default administrative account is seeded at the same time if none exists.

Static store. A server-side directory holds validated photographs under timestamp-prefixed names, alongside the stylesheet and the client script. The directory is created at start-up if it is absent.

C. End-to-End Execution Flow

The following worked example traces a single realistic transaction — a student reporting a mislaid laptop with a photograph — through every layer identified above, and then traces its recovery by the owner.

1. The student opens the portal. The dispatcher matches the root rule, the feed handler finds no search or filter parameters in the query string, and the mapper issues an unrestricted selection ordered by report date descending. The master template renders the navigation bar in its unauthenticated form.

2. The student registers. The registration handler reads the three form fields, queries the user table for the submitted email address, finds no match, converts the password to a `pbkdf2:sha256` digest, inserts the record with the default role of student, and redirects to the login page with a success message.
3. The student logs in. The login handler retrieves the account by email address and compares the submitted password against the stored digest. On a match, the session manager writes the account identifier into the signed session cookie and the student is returned to the feed, whose navigation bar now offers the reporting action and logout.
4. The student submits the report. The reporting route is entered only because the authentication decorator finds an established session. The handler reads the six textual fields, then examines the uploaded file: the extension is `.jpg` and therefore on the whitelist, so the filename is sanitised and prefixed with the current timestamp before the bytes are written into the uploads directory. An item object is constructed with the six fields, the resulting filename and the current session's account identifier, and the transaction is committed. The student is redirected to the feed with a confirmation message, and the new card appears at the head of the grid because the feed is ordered by report date descending.
5. Another student searches. Entering a keyword and selecting the Electronics category produces a request carrying two query parameters. The feed handler composes a case-insensitive partial match against the title and the description, adds an equality constraint on the category, and returns the reduced result set. The matching card is selected and the detail handler resolves the identifier, returning a not-found response had the record been deleted in the interim.
6. The owner marks the article recovered. Opening the same detail page while authenticated as the original reporter reveals the status control, because the template compares the current session identity against the record's owner. Submitting the transition enters the status handler, which repeats that ownership comparison on the server, validates that the submitted value is one of the three permitted statuses, writes the new value and commits.
7. An administrator supervises. Authenticating as the administrative account exposes the dashboard link. The dashboard handler executes five counting queries — total users, total items, and one count for each status — and renders them as metric cards. From the item register the administrator may remove a fraudulent posting; from the user register the administrator may remove an offending account, in which case that account's items are removed first so that the foreign-key constraint is preserved, and the administrator's own account is refused as a deletion target.

This flow exercises every layer of Figure 3.11, every module of Table 3.3 and every table of the data dictionary, and constitutes the acceptance path that is formally tested in Chapter 5.

CHAPTER 4: IMPLEMENTATION & CODING

This chapter documents the realisation of the design set out in Chapter 3 as executable software. It records the development environment, the directory layout of the delivered project, the five algorithms that carry the system’s essential logic, the complete source code of every file in the project, the error-handling and parameter-passing conventions, the procedure for hosting the application, and finally an honest assessment of the security posture including its known limitations.

Every listing reproduced in Section 4.3 is the actual, unedited content of the corresponding file in the submitted project folder. No listing has been abridged, simplified or idealised for presentation.

4.1 Development Environment and Project Directory Structure

4.1.1 Development Environment

Component	Specification
Programming language	Python 3.x
Web framework	Flask 3.0.0
Object Relational Mapper	Flask-SQLAlchemy 3.1.1
Session and authentication extension	Flask-Login 0.6.3
WSGI and security utilities	Werkzeug 3.0.1
Database driver	PyMySQL 1.1.0 (with the <code>cryptography</code> package for MySQL 8 authentication)
Database server	MySQL Server 8.0 or later
Templating engine	Jinja2 (bundled with Flask)
Frontend technologies	HTML5, CSS3, JavaScript ES6
Testing framework	Python <code>unittest</code> (standard library)
Development server	Werkzeug development server on port 5000
Code editor	Visual Studio Code
Browser used for verification	Google Chrome

Table 4.1 — Development environment and toolchain

The dependency set is pinned in the project’s requirements manifest so that the evaluator’s environment can be reproduced exactly:

```
Flask==3.0.0
Flask-SQLAlchemy==3.1.1
Flask-Login==0.6.3
Werkzeug==3.0.1
PyMySQL==1.1.0
cryptography
```

4.1.2 Project Directory Structure

PINKI/	project root
+++ app.py	Flask application: configuration, ORM models, authorization decorator, all twelve routes, and the start-up entry point (261 lines)
+++ create_mysql_db.py	one-off schema provisioning script
+++ requirements.txt	pinned dependency manifest
+++ test_app.py	automated unittest module (four test cases)
+++ templates/	Jinja2 server-side templates
+++ base.html	master template: head, navigation, flash region
+++ index.html	public feed: search form, filters, card grid
+++ item_detail.html	single report with photograph and status control
+++ add_item.html	report submission form (multipart)
+++ login.html	credential form
+++ register.html	account creation form
+++ admin_dashboard.html	five aggregate metric cards
+++ admin_items.html	item register with delete action
+++ admin_users.html	user register with delete action
+++ static/	assets served directly to the browser
+++ css/	
+++ style.css	complete visual language (802 lines)
+++ js/	
+++ script.js	client-side interface conveniences
+++ uploads/	photographs written by the upload handler, stored under timestamp-prefixed names
+++ SRS_and_System_Design.md	IEEE-830 specification and design diagrams
+++ README.md	installation and usage notes
+++ walkthrough.md	feature walkthrough

Figure 4.1 — Project directory structure

Three properties of this layout deserve comment. First, the entire server-side logic resides in a single module of 261 lines; for a project of this size a package split would add ceremony without adding clarity, and the module is internally sectioned by comment banners into models, helpers, public routes and administrative routes. Second, the uploads directory is nested inside the static directory, because the photographs must be served to the browser as static assets by the same mechanism that serves the stylesheet. Third, the provisioning script is deliberately separate from the application: it creates the *schema*, whereas the application creates the *tables*, so that a failure of schema creation is diagnosed before the application is ever launched.

4.2 Core Algorithms and Logic Design

Five algorithms carry the logic on which the correctness and the safety of the system depend. Each is presented below in structured pseudocode, followed by a statement of the property it guarantees.

4.2.1 Authentication and Password Hashing Algorithm

The system never stores, logs or transmits a password in recoverable form. Registration converts the submitted password into a `pbkdf2:sha256` digest; authentication re-derives a

digest from the submitted password using the salt embedded in the stored digest and compares the two.

ALGORITHM RegisterUser

INPUT name, email, password (from the submitted form)

OUTPUT a persisted user record, or rejection

```
1. IF a session is already established THEN
2.     redirect to the feed and STOP           // no account switching
3. query the user table for a record whose email equals email
4. IF such a record exists THEN
5.     flash "Email address already exists"
6.     redirect to the registration form and STOP
7. digest <- generate_password_hash(password, method = 'pbkdf2:sha256')
8. construct User(name, email, password_hash = digest) // role defaults to 'student'
9. add to session and COMMIT
10. flash "Registration successful"
11. redirect to the login form
```

ALGORITHM AuthenticateUser

INPUT email, password

OUTPUT an established session, or rejection

```
1. IF a session is already established THEN redirect to the feed and STOP
2. user <- first record in the user table whose email equals email
3. IF user exists AND check_password_hash(user.password_hash, password) THEN
4.     login_user(user) // writes the identity into the signed cookie
5.     redirect to the feed
6. ELSE
7.     flash "Please check your login details and try again."
8.     re-render the login form
```

Guaranteed property. The plaintext password exists only in the memory of the request that carried it. The digest is a one-way function with a per-record salt, so two members who choose the same password receive different digests, and a disclosure of the database does not disclose the passwords. The failure message in step 7 is deliberately identical whether the email is unknown or the password is wrong, so the login form cannot be used to enumerate registered addresses.

4.2.2 Role-Based Access Control Algorithm

Authorization is implemented as a decorator, so that the guard is expressed once and applied declaratively to each protected view rather than being restated inside every handler.

ALGORITHM AdminRequired(view_function f)

OUTPUT a wrapped view function

```
1. DEFINE wrapper(*args, **kwargs):
2.     IF the current user is not authenticated
3.     OR the current user's role is not 'admin' THEN
4.         flash "You do not have permission to access your requested page."
5.         redirect to the public feed
6.         RETURN // f is never entered
7.     RETURN f(*args, **kwargs)
8. RETURN wrapper
```

Guaranteed property. The protected view function is not merely hidden — it is never entered. The two conditions are evaluated in that order, so an anonymous request is

rejected before any attribute of a non-existent identity is read. The response is a redirect with a generic message rather than an error page, so an unauthorised probe learns nothing about the internal structure of the administrative area. Because the decorator is applied *beneath* the authentication decorator in the source, an anonymous visitor is first sent to the login page, and only an authenticated non-administrator reaches the permission message.

4.2.3 Dynamic Multi-Criteria Search and Filter Algorithm

The feed accepts up to three independent, optional criteria. Rather than branching over the eight possible combinations, the handler composes a single query object progressively, adding one constraint per supplied criterion.

```
ALGORITHM ComposeFeedQuery
INPUT  search, category, status  (each possibly empty, from the query string)
OUTPUT an ordered list of item records

1. query <- SELECT * FROM item                // unrestricted base query, not yet executed
2. IF search is non-empty THEN
3.   query <- query WHERE (title ILIKE '%search%' OR description ILIKE '%search%')
4. IF category is non-empty THEN
5.   query <- query WHERE category = category
6. IF status is non-empty THEN
7.   query <- query WHERE status = status
8. query <- query ORDER BY date_reported DESCENDING
9. RETURN query.all()                        // single round trip to the database
10. re-render the feed, echoing search, category and status back into the form controls
```

Guaranteed property. Exactly one statement reaches the database however many criteria were supplied, so the cost of filtering is borne by the database rather than by the application. The keyword test is case-insensitive and matches a substring of either the title or the description, which means a member who remembers only a fragment of a description can still locate the report. Every criterion is bound as a parameter by the Object Relational Mapper, so a criterion containing SQL syntax is treated as a literal search term and not as executable text. The three criteria are echoed back into the form so that the interface remains consistent with the result set being displayed.

4.2.4 Secure Image Upload and Collision-Avoidance Algorithm

An upload endpoint is the most exposed surface in any web application, since it accepts an attacker-controlled filename and an attacker-controlled byte stream. Four independent defences are applied.

```
ALGORITHM StoreUploadedPhotograph
INPUT  the multipart request
OUTPUT a stored filename, or nothing

0. the framework rejects the request outright if its body exceeds MAX_CONTENT_LENGTH (5 MB)
1. IF the request carries no field named 'image' THEN RETURN nothing
2. file <- the uploaded file object
3. IF file is empty THEN RETURN nothing
4. IF '.' not in file.filename THEN RETURN nothing
5. extension <- the text after the final '.' in file.filename, lowercased
6. IF extension not in {'png', 'jpg', 'jpeg'} THEN RETURN nothing
7. safe_name <- secure_filename(file.filename)    // strips directory separators,
8.                                                    // null bytes and unsafe characters
9. unique_name <- current_timestamp('YYYYMMDDHHMMSS') + '_' + safe_name
```

```

10. write the bytes to UPLOAD_FOLDER / unique_name
11. RETURN unique_name                                // stored in item.image_path

```

Guaranteed property. Four distinct attacks are foreclosed. A request larger than five megabytes is refused by the framework before the handler runs, which bounds the resource cost of a malicious upload. The extension whitelist is a positive list, so an executable or script extension is not merely blacklisted but simply absent from the permitted set. Filename sanitisation removes path separators, so a filename crafted to traverse out of the uploads directory cannot escape it. The timestamp prefix makes every stored name unique, so a second upload of a file with the same original name cannot overwrite the first. Note that a file failing the extension test does not abort the submission: the textual report is still stored, with a null image path, because the photograph is an optional attribute.

4.2.5 Ownership-Verified Status Mutation Algorithm

The status of a report is the one mutable attribute in the system, and mutating it is the action by which the register is kept honest. Authority to mutate rests solely with the member who created the record.

```

ALGORITHM UpdateItemStatus
INPUT    the item identifier from the URL, the submitted status value
OUTPUT   a committed status change, or refusal

0.  the routing layer rejects a non-integer identifier before the handler is entered
0a. the authentication decorator rejects an anonymous request
1.  item <- the item whose primary key equals the identifier, OR abort with 404
2.  IF item.owner.id is not equal to the current session's user id THEN
3.      flash "You do not have permission to update this item."
4.      redirect to the item detail page and STOP
5.  new_status <- the submitted status value
6.  IF new_status in {'Lost', 'Found', 'Claimed'} THEN
7.      item.status <- new_status
8.      COMMIT
9.      flash "Status updated successfully."
10. redirect to the item detail page

```

Guaranteed property. Three layers of validation apply in sequence: the identifier must be an integer, the request must carry an authenticated session, and that session's identity must equal the record's owner. The ownership test is performed on the server and not merely by hiding the control in the template, so a hand-crafted request naming another member's item is refused. The value whitelist in step 6 means that even the legitimate owner cannot write an arbitrary string into the status column, which preserves the integrity of the status filter and of the administrative statistics. A rejected transition is silent in the database: nothing is written and the record is left exactly as it was.

4.3 Programming Code

The complete source code of the project follows. Listings are ordered so that the database is provisioned first, the application's configuration and data model second, and the interface last — that is, in the order in which the components are brought into existence when the system is deployed.

4.3.1 Database Provisioning Script — `create_mysql_db.py`

This script is executed once, before the application is first launched. Its sole responsibility is to create the schema in which the application will subsequently create its tables. It connects without naming a database, issues a conditional creation statement, and closes the connection.

```
import pymysql

try:
    # Connect directly to MySQL without selecting a database
    # The password must be exactly 'root' based on user's instruction
    conn = pymysql.connect(host='localhost', user='root', password='root')
    cursor = conn.cursor()

    # Create the database securely
    cursor.execute('CREATE DATABASE IF NOT EXISTS lost_and_found_db')

    conn.commit()
    conn.close()
    print("Database connected and `lost_and_found_db` ensured accurately.")
except Exception as e:
    print(f"Error accessing MySQL: {e}\nEnsure MySQL is running and password is 'root'.")
```

Note that the statement is idempotent — running the script against an existing schema is harmless — and that it contains no user-supplied value, so the fact that it is hand-written SQL rather than an ORM operation introduces no injection surface.

4.3.2 Application Configuration and ORM Models — `app.py`

The first section of the application module imports the framework and its extensions, configures the application object, and initialises the ORM and the session manager.

```
import os
from datetime import datetime
from functools import wraps
from flask import Flask, render_template, request, redirect, url_for, flash, send_from_directory
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, UserMixin, login_user, login_required, logout_user, current_user
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename

app = Flask(__name__)
app.config['SECRET_KEY'] = 'supersecretkey_for_bca_project'
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+pymysql://root:root@localhost/lost_and_found_db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
basedir = os.path.abspath(os.path.dirname(__file__))
app.config['UPLOAD_FOLDER'] = os.path.join(basedir, 'static', 'uploads')
app.config['MAX_CONTENT_LENGTH'] = 5 * 1024 * 1024 # 5 MB max
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg'}

db = SQLAlchemy(app)
login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = 'login'
```

The configuration establishes five facts about the deployment: the signing key for the session cookie, the database connection string naming the `mysql+pymysql` dialect and the `lost_and_found_db` schema, the suppression of the ORM's modification-tracking signal, the

absolute path of the uploads directory computed relative to the module's own location, and the five-megabyte ceiling on request size. The login manager is told which view to use as the login page, so that an unauthenticated request for a protected route is redirected there automatically rather than failing.

The two model classes follow. Each declares the columns documented in the data dictionary of Section 3.1.3, and the user model additionally declares the collection side of the one-to-many relationship with a reverse reference named `owner`.

```
# --- MODELS ---
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(150), nullable=False)
    email = db.Column(db.String(150), unique=True, nullable=False)
    password_hash = db.Column(db.String(256), nullable=False)
    role = db.Column(db.String(20), nullable=False, default='student')
    items = db.relationship('Item', backref='owner', lazy=True)

class Item(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(150), nullable=False)
    description = db.Column(db.Text, nullable=False)
    category = db.Column(db.String(50), nullable=False) # Electronics, Books, Accessories,
    Others
    status = db.Column(db.String(50), nullable=False) # Lost, Found, Claimed
    image_path = db.Column(db.String(255), nullable=True)
    date_reported = db.Column(db.DateTime, default=datetime.utcnow)
    location = db.Column(db.String(150), nullable=False)
    contact_info = db.Column(db.String(150), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

def admin_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if not current_user.is_authenticated or current_user.role != 'admin':
            flash('You do not have permission to access your requested page.', 'danger')
            return redirect(url_for('index'))
        return f(*args, **kwargs)
    return decorated_function

def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
```

Three collaborators appear alongside the models. The user loader is the callback by which the session manager converts the identifier stored in the cookie back into a live user object on every request. The administrative decorator implements the algorithm of Section 4.2.2. The file-extension helper implements step 6 of the algorithm of Section 4.2.4.

4.3.3 Search and Feed Route — /

The feed is the application's entry point and the only route that composes a query from request parameters.

```

@app.route('/')
def index():
    search_query = request.args.get('search', '')
    category_filter = request.args.get('category', '')
    status_filter = request.args.get('status', '')

    query = Item.query
    if search_query:
        query = query.filter(Item.title.ilike(f'%{search_query}%') |
Item.description.ilike(f'%{search_query}%'))
    if category_filter:
        query = query.filter_by(category=category_filter)
    if status_filter:
        query = query.filter_by(status=status_filter)

    items = query.order_by(Item.date_reported.desc()).all()
    return render_template('index.html', items=items, search_query=search_query,
category_filter=category_filter, status_filter=status_filter)

```

The three criteria are read with a default of the empty string, so an absent parameter is indistinguishable from an empty one and neither adds a constraint. The composed query is executed once, and all three criteria are passed back into the template so that the search box and the two select controls can be re-populated with the values that produced the displayed result set.

4.3.4 Authentication Routes — /register, /login, /logout

```

@app.route('/register', methods=['GET', 'POST'])
def register():
    if current_user.is_authenticated:
        return redirect(url_for('index'))
    if request.method == 'POST':
        name = request.form.get('name')
        email = request.form.get('email')
        password = request.form.get('password')

        user = User.query.filter_by(email=email).first()
        if user:
            flash('Email address already exists')
            return redirect(url_for('register'))

        new_user = User(name=name, email=email, password_hash=generate_password_hash(password,
method='pbkdf2:sha256'))
        db.session.add(new_user)
        db.session.commit()
        flash('Registration successful! Please login.', 'success')
        return redirect(url_for('login'))

    return render_template('register.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('index'))
    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')

        user = User.query.filter_by(email=email).first()
        if user and check_password_hash(user.password_hash, password):
            login_user(user)
            return redirect(url_for('index'))
        else:

```

```

        flash('Please check your login details and try again.', 'danger')

    return render_template('login.html')

@app.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for('index'))

```

Both the registration and the login handlers begin by redirecting an already-authenticated visitor to the feed, which prevents a logged-in member from creating a second account or from re-authenticating as somebody else without first logging out. The logout handler is guarded by the authentication requirement, since there is no session to terminate otherwise.

4.3.5 Item Management Routes — /item/add, /item/<id>, /item/<id>/update_status

```

@app.route('/item/add', methods=['GET', 'POST'])
@login_required
def add_item():
    if request.method == 'POST':
        title = request.form.get('title')
        description = request.form.get('description')
        category = request.form.get('category')
        status = request.form.get('status')
        location = request.form.get('location')
        contact_info = request.form.get('contact_info')

        image_path = None
        if 'image' in request.files:
            file = request.files['image']
            if file and allowed_file(file.filename):
                filename = secure_filename(file.filename)
                # To prevent filename collisions
                unique_filename = f"{datetime.now().strftime('%Y%m%d%H%M%S')}_{filename}"
                file.save(os.path.join(app.config['UPLOAD_FOLDER'], unique_filename))
                image_path = unique_filename

        new_item = Item(
            title=title,
            description=description,
            category=category,
            status=status,
            location=location,
            contact_info=contact_info,
            image_path=image_path,
            user_id=current_user.id
        )
        db.session.add(new_item)
        db.session.commit()
        flash('Item posted successfully!', 'success')
        return redirect(url_for('index'))

    return render_template('add_item.html')

@app.route('/item/<int:id>')
def item_detail(id):
    item = Item.query.get_or_404(id)
    return render_template('item_detail.html', item=item)

@app.route('/item/<int:id>/update_status', methods=['POST'])
@login_required

```

```

def update_status(id):
    item = Item.query.get_or_404(id)
    if item.owner.id != current_user.id:
        flash('You do not have permission to update this item.', 'danger')
        return redirect(url_for('item_detail', id=item.id))

    new_status = request.form.get('status')
    if new_status in ['Lost', 'Found', 'Claimed']:
        item.status = new_status
        db.session.commit()
        flash('Status updated successfully.', 'success')

    return redirect(url_for('item_detail', id=item.id))

```

The creation handler implements the upload algorithm of Section 4.2.4 inline, between reading the textual fields and constructing the model object; the resulting filename, or `None`, becomes the record's image path. The detail handler uses the ORM's fetch-or-abort helper, so a request for a deleted or non-existent identifier produces a 404 response rather than an exception. The status handler implements the ownership-verified mutation algorithm of Section 4.2.5.

4.3.6 Administrative Routes and Application Entry Point

--- ADMIN ROUTES ---

```

@app.route('/admin/dashboard')
@login_required
@admin_required
def admin_dashboard():
    total_users = User.query.count()
    total_items = Item.query.count()
    total_lost = Item.query.filter_by(status='Lost').count()
    total_found = Item.query.filter_by(status='Found').count()
    total_claimed = Item.query.filter_by(status='Claimed').count()
    return render_template('admin_dashboard.html',
                           total_users=total_users,
                           total_items=total_items,
                           total_lost=total_lost,
                           total_found=total_found,
                           total_claimed=total_claimed)

@app.route('/admin/items')
@login_required
@admin_required
def admin_items():
    items = Item.query.order_by(Item.date_reported.desc()).all()
    return render_template('admin_items.html', items=items)

@app.route('/admin/users')
@login_required
@admin_required
def admin_users():
    users = User.query.all()
    return render_template('admin_users.html', users=users)

@app.route('/admin/delete_item/<int:id>', methods=['POST'])
@login_required
@admin_required
def admin_delete_item(id):
    item = Item.query.get_or_404(id)
    db.session.delete(item)
    db.session.commit()

```

```

        flash('Item deleted successfully by administrator.', 'success')
        return redirect(url_for('admin_items'))

@app.route('/admin/delete_user/<int:id>', methods=['POST'])
@login_required
@admin_required
def admin_delete_user(id):
    user = User.query.get_or_404(id)
    if user.id == current_user.id:
        flash('You cannot delete your own admin account.', 'danger')
        return redirect(url_for('admin_users'))

    Item.query.filter_by(user_id=user.id).delete()
    db.session.delete(user)
    db.session.commit()
    flash('User deleted successfully.', 'success')
    return redirect(url_for('admin_users'))

```

Every handler in this section carries both decorators, in the order authentication-then-authorization. The dashboard obtains its five metrics by counting in the database rather than by loading rows into memory, so the cost of the dashboard does not grow with the size of the register. The user-deletion handler performs two protective steps before deleting: it refuses to delete the acting administrator's own account, and it deletes the target's items first so that the mandatory foreign key on the item table cannot be left dangling.

The module's entry point prepares the runtime environment on every start: it creates the uploads directory if it is absent, creates any missing tables through the ORM, and seeds a default administrative account if no account with the reserved address exists.

```

if __name__ == '__main__':
    with app.app_context():
        # Ensure uploads folder exists
        os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
        db.create_all()

        # Create default admin if none exists
        admin_user = User.query.filter_by(email='admin@college.edu').first()
        if not admin_user:
            admin_user = User(
                name='Platform Administrator',
                email='admin@college.edu',
                password_hash=generate_password_hash('admin', method='pbkdf2:sha256'),
                role='admin'
            )
            db.session.add(admin_user)
            db.session.commit()
            print("Default admin created: admin@college.edu | password: admin")

    app.run(debug=True, port=5000)

```

4.3.7 Frontend Presentation Layer

4.3.7.1 Master Template — `templates/base.html`

The master template defines the document skeleton inherited by all eight remaining templates: the document head and stylesheet link, the navigation bar rendered

conditionally against the session state, the flash-message region, the content block, and the script tag.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Lost & Found Portal{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap"
rel="stylesheet">
  <!-- Font Awesome for icons -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
</head>
<body>
  <nav class="navbar">
    <div class="nav-container">
      <a href="{{ url_for('index') }}" class="nav-brand">
        <i class="fas fa-search-location"></i> CampusFind
      </a>
      <div class="nav-links">
        <a href="{{ url_for('index') }}" class="nav-item">Browse Items</a>
        {% if current_user.is_authenticated %}
          <a href="{{ url_for('add_item') }}" class="nav-item btn-primary"><i
class="fas fa-plus"></i> Post Item</a>
          {% if current_user.role == 'admin' %}
            <a href="{{ url_for('admin_dashboard') }}" class="nav-item"><i class="fas fa-
shield-alt"></i> Dashboard</a>
          {% endif %}
          <span class="nav-item user-greeting">Hi, {{ current_user.name }}</span>
          <a href="{{ url_for('logout') }}" class="nav-item">Logout</a>
        {% else %}
          <a href="{{ url_for('login') }}" class="nav-item">Login</a>
          <a href="{{ url_for('register') }}" class="nav-item btn-primary">Sign Up</a>
        {% endif %}
      </div>
    </div>
  </nav>

  <main class="container">
    {% with messages = get_flashed_messages(with_categories=true) %}
      {% if messages %}
        <div class="flash-messages">
          {% for category, message in messages %}
            <div class="alert alert-{{ category }}">
              {{ message }}
              <span class="close-btn"
onclick="this.parentElement.style.display='none';">&times;</span>
            </div>
          {% endfor %}
        </div>
      {% endif %}
    {% endwith %}

    {% block content %}{% endblock %}
  </main>

  <footer class="footer">
    <p>&copy; 2026 CampusFind - College Lost & Found Portal</p>
  </footer>
```

```

    <script src="{{ url_for('static', filename='js/script.js') }}"></script>
</body>
</html>

```

4.3.7.2 Public Feed — `templates/index.html`

```

{% extends "base.html" %}

{% block title %}Home - Lost & Found Portal{% endblock %}

{% block content %}
<div class="page-header">
    <h1><i class="fas fa-search-dollar"></i> Missing & Found Items</h1>
    <p>Help your fellow campus members find what they lost!</p>
</div>

<div class="search-filter-section glass-panel">
    <form method="GET" action="{{ url_for('index') }}" class="search-form">
        <div class="form-group search-input-group">
            <i class="fas fa-search search-icon"></i>
            <input type="text" name="search" placeholder="Search items..." value="{{ search_query }}">
        </div>
        <div class="form-group">
            <select name="category">
                <option value="">All Categories</option>
                <option value="Electronics" {% if category_filter == 'Electronics' %}>selected{% endif %}>Electronics</option>
                <option value="Books" {% if category_filter == 'Books' %}>selected{% endif %}>Books</option>
                <option value="Accessories" {% if category_filter == 'Accessories' %}>selected{% endif %}>Accessories</option>
                <option value="Others" {% if category_filter == 'Others' %}>selected{% endif %}>Others</option>
            </select>
        </div>
        <div class="form-group">
            <select name="status">
                <option value="">All Statuses</option>
                <option value="Lost" {% if status_filter == 'Lost' %}>selected{% endif %}>Lost</option>
                <option value="Found" {% if status_filter == 'Found' %}>selected{% endif %}>Found</option>
                <option value="Claimed" {% if status_filter == 'Claimed' %}>selected{% endif %}>Claimed</option>
            </select>
        </div>
        <button type="submit" class="btn btn-primary">Filter</button>
        <a href="{{ url_for('index') }}" class="btn btn-secondary">Clear</a>
    </form>
</div>

<div class="items-grid">
    {% if items %}
        {% for item in items %}
            <div class="item-card {% if item.status == 'Claimed' %}>claimed-card{% endif %}>
                <div class="item-image-wrapper">
                    {% if item.image_path %}
                        
                    {% else %}
                        <div class="no-image">
                            <i class="fas fa-camera text-muted"></i>
                            <span>No Image</span>
                        </div>
                    {% endif %}
                </div>
            </div>
        {% endfor %}
    {% else %}
        <div class="no-items">
            <i class="fas fa-search text-muted"></i>
            <span>No Items Found</span>
        </div>
    {% endif %}
</div>

```

```

        {% endif %}
        <span class="status-badger status-{{ item.status|lower }}">{{ item.status
    }}</span>
    </div>

    <div class="item-content">
        <div class="item-meta">
            <span class="category"><i class="fas fa-tag"></i> {{ item.category }}</span>
            <span class="date"><i class="far fa-clock"></i> {{
item.date_reported.strftime('%b %d, %Y') }}</span>
        </div>
        <h3 class="item-title">{{ item.title }}</h3>
        <p class="item-desc">{{ item.description|truncate(80) }}</p>

        <div class="item-footer">
            <span class="location" title="{{ item.location }}"><i class="fas fa-map-
marker-alt"></i> {{ item.location|truncate(20) }}</span>
            <a href="{{ url_for('item_detail', id=item.id) }}" class="btn btn-outline-
primary btn-sm">View Details</a>
        </div>
    </div>
</div>
{% endfor %}
{% else %}
    <div class="empty-state">
        <div class="empty-icon-wrapper">
            <i class="fas fa-box-open empty-icon"></i>
        </div>
        <h2>No items match your search</h2>
        <p>Try adjusting your filters or search terms.</p>
        {% if not current_user.is_authenticated %}
            <a href="{{ url_for('login') }}" class="btn btn-primary mt-4">Login to Post an
Item</a>
        {% endif %}
    </div>
{% endif %}
</div>
{% endblock %}

```

4.3.7.3 Item Detail View — templates/item_detail.html

```

{% extends "base.html" %}

{% block title %}{{ item.title }} - Lost & Found Portal{% endblock %}

{% block content %}
<div class="item-detail-container">
    <a href="{{ url_for('index') }}" class="back-link"><i class="fas fa-arrow-left"></i> Back to
all items</a>

    <div class="detail-card glass-panel">
        <div class="detail-header">
            <span class="status-badger status-{{ item.status|lower }}">{{ item.status }}</span>
            <span class="category-badger"><i class="fas fa-tag"></i> {{ item.category }}</span>
        </div>

        <div class="detail-body">
            <div class="detail-image-section">
                {% if item.image_path %}
                    
                {% else %}
                    <div class="detail-no-image">
                        <i class="fas fa-camera"></i>
                        <span>No Image Provided</span>
                    </div>
                {% endif %}
            </div>
        </div>
    </div>
</div>

```

```

    </div>
    {% endif %}
</div>

<div class="detail-info-section">
    <h1 class="detail-title">{{ item.title }}</h1>

    <div class="meta-info-grid">
        <div class="meta-item">
            <i class="far fa-calendar-alt"></i>
            <div>
                <strong>Posted on</strong>
                <span>{{ item.date_reported.strftime('%B %d, %Y') }}</span>
            </div>
        </div>

        <div class="meta-item">
            <i class="fas fa-map-marker-alt"></i>
            <div>
                <strong>Location</strong>
                <span>{{ item.location }}</span>
            </div>
        </div>

        <div class="meta-item">
            <i class="fas fa-user-circle"></i>
            <div>
                <strong>Posted by</strong>
                <span>{{ item.owner.name }}</span>
            </div>
        </div>
    </div>

    <div class="description-block">
        <h3>Description</h3>
        <p>{{ item.description }}</p>
    </div>

    <div class="contact-block">
        <h3>Contact Information</h3>
        <div class="contact-box">
            <i class="fas fa-address-book"></i> {{ item.contact_info }}
        </div>
    </div>

    {% if current_user.is_authenticated and current_user.id == item.user_id %}
    <div class="action-block">
        <h3>Update Status</h3>
        <form method="POST" action="{{ url_for('update_status', id=item.id) }}"
class="status-update-form">
            <select name="status" class="status-select">
                <option value="Lost" {% if item.status == 'Lost' %}>selected{% endif
%}>Lost</option>
                <option value="Found" {% if item.status == 'Found' %}>selected{% endif
%}>Found</option>
                <option value="Claimed" {% if item.status == 'Claimed' %}>selected{%
endif %}>Claimed</option>
            </select>
            <button type="submit" class="btn btn-primary btn-sm">Update</button>
        </form>
    </div>
    {% endif %}
</div>
</div>

```

```

    </div>
</div>
{% endblock %}

```

4.3.7.4 Item Submission Form — `templates/add_item.html`

```

{% extends "base.html" %}

{% block title %}Post Item - Lost & Found Portal{% endblock %}

{% block content %}
<div class="form-container">
    <div class="form-card glass-panel">
        <div class="form-header">
            <h2>Post an Item</h2>
            <p>Fill in the details below. The more accurate, the better!</p>
        </div>

        <form method="POST" action="{% url_for('add_item') %}" enctype="multipart/form-data"
class="main-form">
            <div class="form-row">
                <div class="form-group half-width">
                    <label for="title">Item Title <span class="required">*</span></label>
                    <input type="text" id="title" name="title" required placeholder="E.g., Blue
Dell Laptop">
                </div>

                <div class="form-group half-width">
                    <label for="status">Posting Type <span class="required">*</span></label>
                    <select id="status" name="status" required>
                        <option value="Lost">I Lost Something</option>
                        <option value="Found">I Found Something</option>
                    </select>
                </div>
            </div>

            <div class="form-row">
                <div class="form-group half-width">
                    <label for="category">Category <span class="required">*</span></label>
                    <select id="category" name="category" required>
                        <option value="Electronics">Electronics</option>
                        <option value="Books">Books & Notes</option>
                        <option value="Accessories">Accessories & Wearables</option>
                        <option value="Others">Others</option>
                    </select>
                </div>
                <div class="form-group half-width">
                    <label for="location">Location <span class="required">*</span></label>
                    <input type="text" id="location" name="location" required placeholder="Where
was it lost/found? (e.g., Library)">
                </div>
            </div>

            <div class="form-group">
                <label for="description">Detailed Description <span
class="required">*</span></label>
                <textarea id="description" name="description" rows="4" required
placeholder="Describe the item in detail. Mention any identifiable marks..."></textarea>
            </div>

            <div class="form-group">
                <label for="contact_info">Your Contact Info <span
class="required">*</span></label>
                <input type="text" id="contact_info" name="contact_info" required
placeholder="Phone number or specific instructions to meet">
            </div>
        </form>
    </div>
</div>
{% endblock %}

```

```

    </div>

    <div class="form-group">
      <label for="image">Upload Image (Optional but recommended)</label>
      <div class="file-upload-wrapper">
        <div class="file-upload-box">
          <i class="fas fa-cloud-upload-alt"></i>
          <span>Click to browse</span>
        </div>
        <input type="file" id="image" name="image" accept=".jpg, .jpeg, .png"
class="file-input">
      </div>
      <small class="form-text text-muted">Max file size: 5MB. Allowed formats: JPG,
PNG.</small>
    </div>

    <div class="form-actions">
      <a href="{{ url_for('index') }}" class="btn btn-secondary">Cancel</a>
      <button type="submit" class="btn btn-primary"><i class="fas fa-paper-plane"></i>
Submit Post</button>
    </div>
  </form>
</div>
{% endblock %}

```

4.3.7.5 Login Form — `templates/login.html`

```

{% extends "base.html" %}

{% block title %}Login - Lost & Found Portal{% endblock %}

{% block content %}
<div class="auth-container">
  <div class="auth-card glass-panel">
    <div class="auth-header">
      <h2>Welcome Back</h2>
      <p>Login to your account to post or claim items</p>
    </div>

    <form method="POST" action="{{ url_for('login') }}" class="auth-form">
      <div class="form-group">
        <label for="email">Email Address</label>
        <div class="input-with-icon">
          <i class="fas fa-envelope"></i>
          <input type="email" id="email" name="email" required
placeholder="example@college.edu">
        </div>
      </div>

      <div class="form-group">
        <label for="password">Password</label>
        <div class="input-with-icon">
          <i class="fas fa-lock"></i>
          <input type="password" id="password" name="password" required
placeholder=".....">
        </div>
      </div>

      <button type="submit" class="btn btn-primary btn-block">Login to Account</button>
    </form>

    <div class="auth-footer">
      <p>Don't have an account? <a href="{{ url_for('register') }}">Create one here</a></p>
    </div>
  </div>
</div>

```

```

    </div>
</div>
{% endblock %}

```

4.3.7.6 Registration Form — `templates/register.html`

```

{% extends "base.html" %}

{% block title %}Sign Up - Lost & Found Portal{% endblock %}

{% block content %}
<div class="auth-container">
  <div class="auth-card glass-panel">
    <div class="auth-header">
      <h2>Create Account</h2>
      <p>Join the campus network to help others and yourself</p>
    </div>

    <form method="POST" action="{% url_for('register') %}" class="auth-form">
      <div class="form-group">
        <label for="name">Full Name</label>
        <div class="input-with-icon">
          <i class="fas fa-user"></i>
          <input type="text" id="name" name="name" required placeholder="John Doe">
        </div>
      </div>

      <div class="form-group">
        <label for="email">College Email Address</label>
        <div class="input-with-icon">
          <i class="fas fa-envelope"></i>
          <input type="email" id="email" name="email" required
placeholder="john@college.edu">
        </div>
      </div>

      <div class="form-group">
        <label for="password">Password</label>
        <div class="input-with-icon">
          <i class="fas fa-lock"></i>
          <input type="password" id="password" name="password" required
placeholder="Create a strong password">
        </div>
      </div>

      <button type="submit" class="btn btn-primary btn-block">Register Account</button>
    </form>

    <div class="auth-footer">
      <p>Already have an account? <a href="{% url_for('login') %}">Login here</a></p>
    </div>
  </div>
</div>
{% endblock %}

```

4.3.7.7 Administrative Dashboard — `templates/admin_dashboard.html`

```

{% extends "base.html" %}

{% block title %}Admin Dashboard{% endblock %}

{% block content %}
<div class="page-header">
  <h1><i class="fas fa-shield-alt"></i> Admin Dashboard</h1>
  <p>System Overview & Moderation</p>

```

```

</div>

<div class="admin-nav mb-4">
    <a href="{{ url_for('admin_dashboard') }}" class="btn btn-primary"><i class="fas fa-chart-
line"></i> Dashboard Metrics</a>
    <a href="{{ url_for('admin_items') }}" class="btn btn-secondary"><i class="fas fa-boxes"></i>
Manage Items</a>
    <a href="{{ url_for('admin_users') }}" class="btn btn-secondary"><i class="fas fa-users"></i>
Manage Users</a>
</div>

<div class="items-grid" style="grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));">
    <div class="item-card glass-panel" style="text-align: center; padding: 2rem;">
        <i class="fas fa-users mb-2" style="font-size: 2.5rem; color: var(--primary-color);"></i>
        <h2 style="font-size: 2.5rem; color: white; margin: 1rem 0;">{{ total_users }}</h2>
        <span style="color: var(--text-secondary); font-weight: 500;">Total Users</span>
    </div>

    <div class="item-card glass-panel" style="text-align: center; padding: 2rem;">
        <i class="fas fa-boxes mb-2" style="font-size: 2.5rem; color: var(--text-primary);"></i>
        <h2 style="font-size: 2.5rem; color: white; margin: 1rem 0;">{{ total_items }}</h2>
        <span style="color: var(--text-secondary); font-weight: 500;">Total Items</span>
    </div>

    <div class="item-card glass-panel" style="text-align: center; padding: 2rem;">
        <i class="fas fa-search mb-2" style="font-size: 2.5rem; color: var(--status-lost);"></i>
        <h2 style="font-size: 2.5rem; color: white; margin: 1rem 0;">{{ total_lost }}</h2>
        <span style="color: var(--text-secondary); font-weight: 500;">Currently Lost</span>
    </div>

    <div class="item-card glass-panel" style="text-align: center; padding: 2rem;">
        <i class="fas fa-check-circle mb-2" style="font-size: 2.5rem; color: var(--status-
found);"></i>
        <h2 style="font-size: 2.5rem; color: white; margin: 1rem 0;">{{ total_found }}</h2>
        <span style="color: var(--text-secondary); font-weight: 500;">Found Needs Claiming</span>
    </div>

    <div class="item-card glass-panel" style="text-align: center; padding: 2rem;">
        <i class="fas fa-handshake mb-2" style="font-size: 2.5rem; color: var(--status-
claimed);"></i>
        <h2 style="font-size: 2.5rem; color: white; margin: 1rem 0;">{{ total_claimed }}</h2>
        <span style="color: var(--text-secondary); font-weight: 500;">Successfully Claimed</span>
    </div>
</div>
{% endblock %}

```

4.3.7.8 Item Register — templates/admin_items.html

```

{% extends "base.html" %}

{% block title %}Manage Items - Admin{% endblock %}

{% block content %}
<div class="page-header">
    <h1><i class="fas fa-boxes"></i> Manage Items</h1>
    <p>Monitor and delete inappropriate or invalid items</p>
</div>

<div class="admin-nav mb-4">
    <a href="{{ url_for('admin_dashboard') }}" class="btn btn-secondary"><i class="fas fa-chart-
line"></i> Dashboard Metrics</a>
    <a href="{{ url_for('admin_items') }}" class="btn btn-primary"><i class="fas fa-boxes"></i>
Manage Items</a>
    <a href="{{ url_for('admin_users') }}" class="btn btn-secondary"><i class="fas fa-users"></i>
Manage Users</a>

```

```

</div>

<div class="glass-panel" style="border-radius: var(--border-radius-lg); overflow: hidden;">
  <div style="overflow-x: auto;">
    <table style="width: 100%; text-align: left; border-collapse: collapse;">
      <thead>
        <tr style="background: rgba(0,0,0,0.2); border-bottom: 1px solid var(--border-
color);">
          <th style="padding: 1rem;">ID</th>
          <th style="padding: 1rem;">Item Title</th>
          <th style="padding: 1rem;">Category</th>
          <th style="padding: 1rem;">Status</th>
          <th style="padding: 1rem;">Posted By</th>
          <th style="padding: 1rem;">Date</th>
          <th style="padding: 1rem;">Action</th>
        </tr>
      </thead>
      <tbody>
        {% for item in items %}
        <tr style="border-bottom: 1px solid var(--border-color);">
          <td style="padding: 1rem;">{{ item.id }}</td>
          <td style="padding: 1rem;">
            <a href="{{ url_for('item_detail', id=item.id) }}"><strong>{{ item.title
}}</strong></a>
          </td>
          <td style="padding: 1rem;">{{ item.category }}</td>
          <td style="padding: 1rem;">
            <span class="status-badge status-{{ item.status|lower }}"
style="position: static;">{{ item.status }}</span>
          </td>
          <td style="padding: 1rem;">{{ item.owner.name }} <small>{{ item.owner.email
}}</small></td>
          <td style="padding: 1rem;">{{ item.date_reported.strftime('%Y-%m-%d') }}</td>
          <td style="padding: 1rem;">
            <form method="POST" action="{{ url_for('admin_delete_item', id=item.id)
}}" onsubmit="return confirm('Are you sure you want to delete this item? This action cannot be
undone.');">
              <button type="submit" class="btn btn-sm" style="background: var(--
status-lost); color: white;"><i class="fas fa-trash"></i> Delete</button>
            </form>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
</div>
{% endblock %}

```

4.3.7.9 User Register — templates/admin_users.html

```

{% extends "base.html" %}

{% block title %}Manage Users - Admin{% endblock %}

{% block content %}
<div class="page-header">
  <h1><i class="fas fa-users"></i> Manage Users</h1>
  <p>View registered accounts and manage platform access</p>
</div>

<div class="admin-nav mb-4">
  <a href="{{ url_for('admin_dashboard') }}" class="btn btn-secondary"><i class="fas fa-chart-
line"></i> Dashboard Metrics</a>
  <a href="{{ url_for('admin_items') }}" class="btn btn-secondary"><i class="fas fa-boxes"></i>

```

```

Manage Items</a>
    <a href="{{ url_for('admin_users') }}" class="btn btn-primary"><i class="fas fa-users"></i>
Manage Users</a>
</div>

<div class="glass-panel" style="border-radius: var(--border-radius-lg); overflow: hidden;">
    <div style="overflow-x: auto;">
        <table style="width: 100%; text-align: left; border-collapse: collapse;">
            <thead>
                <tr style="background: rgba(0,0,0,0.2); border-bottom: 1px solid var(--border-
color);">
                    <th style="padding: 1rem;">ID</th>
                    <th style="padding: 1rem;">Name</th>
                    <th style="padding: 1rem;">Email Address</th>
                    <th style="padding: 1rem;">Role</th>
                    <th style="padding: 1rem;">Total Items Posted</th>
                    <th style="padding: 1rem;">Action</th>
                </tr>
            </thead>
            <tbody>
                {% for user in users %}
                <tr style="border-bottom: 1px solid var(--border-color);">
                    <td style="padding: 1rem;">{{ user.id }}</td>
                    <td style="padding: 1rem;"><strong>{{ user.name }}</strong></td>
                    <td style="padding: 1rem;">{{ user.email }}</td>
                    <td style="padding: 1rem;">
                        {% if user.role == 'admin' %}
                            <span style="color: var(--status-found); font-weight: bold;"><i
class="fas fa-shield-alt"></i> Admin</span>
                        {% else %}
                            <span style="color: var(--text-secondary);">Student</span>
                        {% endif %}
                    </td>
                    <td style="padding: 1rem;">{{ user.items|length }} items</td>
                    <td style="padding: 1rem;">
                        {% if user.id != current_user.id %}
                            <form method="POST" action="{{ url_for('admin_delete_user', id=user.id)
}}" onsubmit="return confirm('Are you sure you want to ban/delete this user AND all their posted
items? This action cannot be undone.');">
                                <button type="submit" class="btn btn-sm" style="background: var(--
status-lost); color: white;"><i class="fas fa-ban"></i> Ban & Delete</button>
                            </form>
                        {% else %}
                            <span style="color: var(--text-light); font-size: 0.85rem;">Current
Session</span>
                        {% endif %}
                    </td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
</div>
{% endblock %}

```

4.3.8 Styling and Client-Side Script

4.3.8.1 Stylesheet — `static/css/style.css`

The stylesheet is the single source of the interface's visual language. It defines the colour palette as custom properties, the translucent card treatment, the responsive grid, the

colour-coded status badges, the form controls, the administrative tables and the responsive breakpoints.

```
:root {
  /* Color Palette */
  --primary-color: #4f46e5;
  --primary-hover: #4338ca;
  --secondary-color: #6366f1;
  --background-color: #f3f4f6;
  --surface-color: rgba(255, 255, 255, 0.85);
  --text-primary: #1f2937;
  --text-secondary: #4b5563;
  --text-light: #9ca3af;
  --border-color: rgba(229, 231, 235, 0.5);

  /* Status Colors */
  --status-lost: #ef4444; /* Red */
  --status-found: #10b981; /* Green */
  --status-claimed: #8b5cf6; /* Purple */

  /* UI Dimensions */
  --border-radius: 12px;
  --border-radius-lg: 20px;
  --shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
  --shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06);
  --shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -2px rgba(0, 0, 0, 0.05);
  --transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', sans-serif;
  background-color: var(--background-color);
  background-image:
    radial-gradient(at 0% 0%, hsla(253,16%,7%,1) 0, transparent 50%),
    radial-gradient(at 50% 0%, hsla(225,39%,30%,1) 0, transparent 50%),
    radial-gradient(at 100% 0%, hsla(339,49%,30%,1) 0, transparent 50%);
  background-color: #0f172a;
  background-attachment: fixed;
  color: #e2e8f0;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

/* Redefine variables for dark theme overall */
:root {
  --primary-color: #6366f1;
  --primary-hover: #818cf8;
  --surface-color: rgba(30, 41, 59, 0.7);
  --surface-hover: rgba(30, 41, 59, 0.9);
  --text-primary: #f8fafc;
  --text-secondary: #cbd5e1;
  --border-color: rgba(255, 255, 255, 0.1);
}

a {
  color: var(--primary-color);
  text-decoration: none;
}
```

```

    transition: var(--transition);
}

a:hover {
    color: var(--primary-hover);
}

/* Glassmorphism Classes */
.glass-panel {
    background: var(--surface-color);
    backdrop-filter: blur(12px);
    -webkit-backdrop-filter: blur(12px);
    border: 1px solid var(--border-color);
    box-shadow: var(--shadow-lg);
}

/* Buttons */
.btn {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    padding: 0.6rem 1.2rem;
    border-radius: var(--border-radius);
    font-weight: 500;
    cursor: pointer;
    transition: var(--transition);
    border: none;
    gap: 0.5rem;
    font-size: 0.95rem;
}

.btn-primary {
    background: linear-gradient(135deg, var(--primary-color), var(--secondary-color));
    color: white;
    box-shadow: 0 4px 14px 0 rgba(99, 102, 241, 0.39);
}

.btn-primary:hover {
    transform: translateY(-2px);
    box-shadow: 0 6px 20px 0 rgba(99, 102, 241, 0.5);
    color: white;
}

.btn-secondary {
    background: rgba(255, 255, 255, 0.1);
    color: var(--text-primary);
    border: 1px solid var(--border-color);
}

.btn-secondary:hover {
    background: rgba(255, 255, 255, 0.2);
    transform: translateY(-2px);
}

.btn-outline-primary {
    background: transparent;
    color: var(--primary-color);
    border: 1px solid var(--primary-color);
}

.btn-outline-primary:hover {
    background: var(--primary-color);
    color: white;
}

```

```

.btn-sm {
  padding: 0.4rem 0.8rem;
  font-size: 0.85rem;
}

.btn-block {
  width: 100%;
}

/* Navigation */
.navbar {
  background: rgba(15, 23, 42, 0.8);
  backdrop-filter: blur(10px);
  border-bottom: 1px solid var(--border-color);
  padding: 1rem 0;
  position: sticky;
  top: 0;
  z-index: 100;
}

.nav-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 1.5rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.nav-brand {
  font-size: 1.5rem;
  font-weight: 700;
  color: white;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.nav-brand i {
  color: var(--primary-color);
}

.nav-links {
  display: flex;
  align-items: center;
  gap: 1.5rem;
}

.nav-item {
  color: var(--text-secondary);
  font-weight: 500;
}

.nav-item:hover {
  color: white;
}

.user-greeting {
  color: var(--primary-light);
  font-weight: 600;
  border-right: 1px solid var(--border-color);
  padding-right: 1.5rem;
}

```

```

/* Main Layout */
.container {
  max-width: 1200px;
  margin: 2rem auto;
  padding: 0 1.5rem;
  flex-grow: 1;
  width: 100%;
  animation: fadeIn 0.5s ease-out;
}

/* Animations */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

/* Alerts / Flash Messages */
.flash-messages {
  margin-bottom: 2rem;
}

.alert {
  padding: 1rem 1.5rem;
  border-radius: var(--border-radius);
  margin-bottom: 1rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: var(--surface-color);
  border-left: 4px solid var(--primary-color);
  animation: slideIn 0.3s ease-out;
}

@keyframes slideIn {
  from { opacity: 0; transform: translateX(-20px); }
  to { opacity: 1; transform: translateX(0); }
}

.alert-success {
  border-left-color: var(--status-found);
}

.alert-danger {
  border-left-color: var(--status-lost);
}

.close-btn {
  cursor: pointer;
  font-size: 1.2rem;
  color: var(--text-secondary);
}

.close-btn:hover {
  color: var(--text-primary);
}

/* Page Header */
.page-header {
  text-align: center;
  margin-bottom: 3rem;
}

.page-header h1 {

```

```

    font-size: 2.5rem;
    margin-bottom: 0.5rem;
    background: linear-gradient(to right, #cfd9df, #e2ebf0);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}

.page-header p {
    color: var(--text-secondary);
    font-size: 1.1rem;
}

/* Search and Filter Section */
.search-filter-section {
    padding: 1.5rem;
    border-radius: var(--border-radius-lg);
    margin-bottom: 3rem;
}

.search-form {
    display: flex;
    gap: 1rem;
    align-items: center;
    flex-wrap: wrap;
}

.form-group {
    flex: 1;
    min-width: 200px;
}

.search-input-group {
    position: relative;
    flex: 2;
}

.search-icon {
    position: absolute;
    left: 1rem;
    top: 50%;
    transform: translateY(-50%);
    color: var(--text-light);
}

input, select, textarea {
    width: 100%;
    padding: 0.8rem 1rem;
    border-radius: var(--border-radius);
    border: 1px solid var(--border-color);
    background: rgba(15, 23, 42, 0.5);
    color: var(--text-primary);
    font-family: inherit;
    font-size: 0.95rem;
    transition: var(--transition);
}

.search-input-group input {
    padding-left: 2.5rem;
}

input:focus, select:focus, textarea:focus {
    outline: none;
    border-color: var(--primary-color);
    box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.2);
}

```

```

}

/* Item Grid and Cards */
.items-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
  gap: 2rem;
}

.item-card {
  background: var(--surface-color);
  border-radius: var(--border-radius-lg);
  overflow: hidden;
  border: 1px solid var(--border-color);
  transition: var(--transition);
  display: flex;
  flex-direction: column;
  height: 100%;
}

.item-card:hover {
  transform: translateY(-5px);
  box-shadow: var(--shadow-lg);
  border-color: rgba(255,255,255,0.2);
}

.claimed-card {
  opacity: 0.7;
  filter: grayscale(40%);
}

.item-image-wrapper {
  position: relative;
  height: 200px;
  width: 100%;
  overflow: hidden;
  background: rgba(0,0,0,0.2);
}

.item-image {
  width: 100%;
  height: 100%;
  object-fit: cover;
  transition: transform 0.5s ease;
}

.item-card:hover .item-image {
  transform: scale(1.05);
}

.no-image {
  width: 100%;
  height: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  color: var(--text-light);
  font-size: 1.2rem;
  gap: 0.5rem;
}

.status-badge {
  position: absolute;

```

```

    top: 1rem;
    right: 1rem;
    padding: 0.3rem 0.8rem;
    border-radius: 20px;
    font-size: 0.8rem;
    font-weight: 600;
    color: white;
    box-shadow: var(--shadow-sm);
    backdrop-filter: blur(4px);
}

.status-lost { background: rgba(239, 68, 68, 0.9); }
.status-found { background: rgba(16, 185, 129, 0.9); }
.status-claimed { background: rgba(139, 92, 246, 0.9); }

.item-content {
    padding: 1.5rem;
    display: flex;
    flex-direction: column;
    flex-grow: 1;
}

.item-meta {
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.85rem;
    color: var(--text-secondary);
    margin-bottom: 0.8rem;
}

.item-title {
    font-size: 1.25rem;
    font-weight: 600;
    margin-bottom: 0.5rem;
    color: white;
}

.item-desc {
    color: var(--text-secondary);
    font-size: 0.9rem;
    line-height: 1.5;
    margin-bottom: 1.5rem;
    flex-grow: 1;
}

.item-footer {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding-top: 1rem;
    border-top: 1px solid var(--border-color);
}

.location {
    font-size: 0.85rem;
    color: var(--text-secondary);
    display: flex;
    align-items: center;
    gap: 0.3rem;
}

/* Empty State */
.empty-state {

```

```

        grid-column: 1 / -1;
        text-align: center;
        padding: 5rem 2rem;
        background: var(--surface-color);
        border-radius: var(--border-radius-lg);
        border: 1px dashed var(--border-color);
    }

.empty-icon-wrapper {
    width: 80px;
    height: 80px;
    background: rgba(99, 102, 241, 0.1);
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    margin: 0 auto 1.5rem;
}

.empty-icon {
    font-size: 2.5rem;
    color: var(--primary-color);
}

/* Authentication Forms */
.auth-container {
    display: flex;
    align-items: center;
    justify-content: center;
    min-height: calc(100vh - 200px);
}

.auth-card {
    width: 100%;
    max-width: 450px;
    padding: 2.5rem;
    border-radius: var(--border-radius-lg);
}

.auth-header {
    text-align: center;
    margin-bottom: 2rem;
}

.auth-header h2 {
    font-size: 1.8rem;
    margin-bottom: 0.5rem;
    color: white;
}

.auth-form .form-group {
    margin-bottom: 1.5rem;
}

.auth-form label {
    display: block;
    margin-bottom: 0.5rem;
    font-weight: 500;
    color: var(--text-secondary);
}

.input-with-icon {
    position: relative;
}

```

```

.input-with-icon i {
  position: absolute;
  left: 1rem;
  top: 50%;
  transform: translateY(-50%);
  color: var(--text-light);
}

.input-with-icon input {
  padding-left: 2.8rem;
}

.auth-footer {
  text-align: center;
  margin-top: 1.5rem;
  font-size: 0.9rem;
  color: var(--text-secondary);
}

/* Forms general */
.form-container {
  max-width: 800px;
  margin: 0 auto;
}

.form-card {
  padding: 2.5rem;
  border-radius: var(--border-radius-lg);
}

.form-header {
  margin-bottom: 2rem;
}

.form-row {
  display: flex;
  gap: 1.5rem;
  margin-bottom: 1.5rem;
}

.half-width {
  flex: 1;
}

.required {
  color: var(--status-lost);
}

.main-form label {
  display: block;
  margin-bottom: 0.5rem;
  font-weight: 500;
}

.main-form .form-group {
  margin-bottom: 1.5rem;
}

.file-upload-wrapper {
  position: relative;
  overflow: hidden;
  margin-bottom: 0.5rem;
}

```

```

.file-upload-box {
  border: 2px dashed var(--border-color);
  padding: 2rem;
  text-align: center;
  border-radius: var(--border-radius);
  background: rgba(0,0,0,0.1);
  transition: var(--transition);
}

.file-upload-box i {
  font-size: 2rem;
  color: var(--primary-color);
  margin-bottom: 0.5rem;
  display: block;
}

.file-input {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  opacity: 0;
  cursor: pointer;
}

.file-input:hover + .file-upload-box,
.file-input:focus + .file-upload-box {
  border-color: var(--primary-color);
  background: rgba(0,0,0,0.2);
}

.form-actions {
  display: flex;
  justify-content: flex-end;
  gap: 1rem;
  margin-top: 2rem;
  padding-top: 1.5rem;
  border-top: 1px solid var(--border-color);
}

/* Item Detail View */
.item-detail-container {
  max-width: 1000px;
  margin: 0 auto;
}

.back-link {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  margin-bottom: 1.5rem;
  color: var(--text-secondary);
}

.back-link:hover {
  color: white;
}

.detail-card {
  border-radius: var(--border-radius-lg);
  overflow: hidden;
}

```

```
.detail-header {
  display: flex;
  justify-content: space-between;
  padding: 1.5rem 2rem;
  border-bottom: 1px solid var(--border-color);
  background: rgba(0,0,0,0.2);
}

.category-badge {
  color: var(--text-secondary);
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.detail-body {
  display: flex;
  flex-wrap: wrap;
  gap: 2rem;
  padding: 2rem;
}

.detail-image-section {
  flex: 1;
  min-width: 300px;
  max-width: 400px;
}

.detail-image {
  width: 100%;
  height: auto;
  border-radius: var(--border-radius);
  box-shadow: var(--shadow-md);
}

.detail-no-image {
  height: 300px;
  background: rgba(0,0,0,0.1);
  border-radius: var(--border-radius);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  color: var(--text-light);
  font-size: 1.2rem;
  gap: 1rem;
}

.detail-info-section {
  flex: 2;
  min-width: 300px;
}

.detail-title {
  font-size: 2rem;
  margin-bottom: 1.5rem;
  color: white;
}

.meta-info-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 1.5rem;
}
```

```
    margin-bottom: 2rem;
}

.meta-item {
    display: flex;
    align-items: center;
    gap: 1rem;
}

.meta-item i {
    font-size: 1.5rem;
    color: var(--primary-color);
    background: rgba(99, 102, 241, 0.1);
    padding: 0.8rem;
    border-radius: 10px;
}

.meta-item strong {
    display: block;
    font-size: 0.8rem;
    color: var(--text-secondary);
    text-transform: uppercase;
    letter-spacing: 0.05em;
}

.meta-item span {
    font-weight: 500;
}

.description-block, .contact-block, .action-block {
    margin-bottom: 2rem;
}

.description-block h3, .contact-block h3, .action-block h3 {
    font-size: 1.2rem;
    margin-bottom: 1rem;
    color: white;
    border-bottom: 1px solid var(--border-color);
    padding-bottom: 0.5rem;
}

.description-block p {
    color: var(--text-secondary);
    line-height: 1.6;
}

.contact-box {
    background: rgba(0,0,0,0.2);
    padding: 1rem 1.5rem;
    border-radius: var(--border-radius);
    display: inline-flex;
    align-items: center;
    gap: 1rem;
    border: 1px solid var(--border-color);
    font-weight: 500;
    color: white;
}

.status-update-form {
    display: flex;
    gap: 1rem;
    align-items: center;
    background: rgba(0,0,0,0.2);
    padding: 1rem;
```

```

    border-radius: var(--border-radius);
    border: 1px solid var(--border-color);
}

.status-select {
    max-width: 200px;
}

/* Footer */
.footer {
    text-align: center;
    padding: 2rem;
    margin-top: 4rem;
    border-top: 1px solid var(--border-color);
    color: var(--text-secondary);
    font-size: 0.9rem;
}

/* Responsive */
@media (max-width: 768px) {
    .nav-links {
        display: none; /* In a real app we'd trigger a hamburger menu */
    }

    .form-row {
        flex-direction: column;
        gap: 0;
    }

    .detail-body {
        flex-direction: column;
    }

    .detail-image-section {
        max-width: 100%;
    }
}

```

4.3.8.2 Client-Side Script — `static/js/script.js`

The client-side script provides interface conveniences only. No validation performed here is relied upon by the server.

```

document.addEventListener("DOMContentLoaded", () => {
    // File upload logic to show selected filename
    const fileInput = document.querySelector('.file-input');
    if (fileInput) {
        fileInput.addEventListener('change', function() {
            const fileName = this.value.split('\\').pop();
            const textBox = this.parentElement.querySelector('.file-upload-box span');
            if (fileName) {
                textBox.textContent = fileName;
                textBox.style.color = 'var(--text-primary)';
            } else {
                textBox.textContent = 'Click to browse';
                textBox.style.color = '';
            }
        });
    }

    // Auto-dismiss alerts after 5 seconds
    const alerts = document.querySelectorAll('.alert');
    alerts.forEach(alert => {

```

```

        setTimeout(() => {
            alert.style.opacity = '0';
            setTimeout(() => {
                alert.style.display = 'none';
            }, 300);
        }, 5000);
    });
});

```

4.4 Error Handling and Parameter Passing

4.4.1 Secure Parameter Passing

Parameters reach the application by four routes, each with its own protection.

URL path parameters. The two parameterised routes declare an integer converter on the identifier segment. A request whose identifier is not an integer therefore fails to match any rule and is answered with a 404 by the routing layer, before any handler is entered and before the value can reach the database.

Query-string parameters. The three feed criteria are read with an explicit default, so an absent parameter yields an empty string rather than a null reference. Each is incorporated into the query as a bound parameter by the ORM.

Form fields. All submitted fields are read through the request's form mapping with a default, so a missing field yields a null rather than raising. Field values are assigned to model attributes and are inserted by the ORM as bound parameters.

Uploaded files. The file field is checked for presence before it is read, and its filename is sanitised before it is used to construct any path.

Across all four routes, the decisive property is that the application contains no concatenated SQL. Every literal — a search fragment, an email address, a status value — is transmitted to MySQL as a bound parameter, which removes SQL injection from the attack surface by construction rather than by filtering.

4.4.2 Error Handling Mechanisms

Condition	Mechanism	Outcome presented to the user
Non-integer identifier in a URL	Integer route converter	404 Not Found from the routing layer
Identifier of a non-existent or deleted record	ORM fetch-or-abort helper	404 Not Found
Unauthenticated request for a protected route	Authentication decorator with a configured login view	Redirect to the login form
Authenticated but non-administrative request for an administrative route	Administrative decorator	Redirect to the feed with a permission message

Request to change the status of another member's item	Explicit ownership comparison	Redirect to the item with a permission message; nothing written
Status value outside the permitted set	Membership test before assignment	Silently ignored; the record is unchanged
Registration with an already-registered email	Pre-insertion existence query, backed by the unique constraint	Redirect to the registration form with a message
Incorrect email or password at login	Digest comparison	Form re-rendered with a single generic message
Upload exceeding five megabytes	Framework-level request-size ceiling	413 Request Entity Too Large
Upload with a disallowed extension	Extension whitelist	Report accepted without a photograph
Administrator attempting to delete their own account	Self-deletion guard	Redirect to the user register with a message
Missing uploads directory at start-up	Directory creation at entry point	Created silently
Missing tables at start-up	ORM schema creation at entry point	Created silently

Table 4.2 — Error conditions and their handling

User-facing feedback is delivered uniformly through the framework's flash-message facility, categorised as `success` or `danger`, and rendered by the master template so that every page displays messages in the same position with the same styling.

4.5 Hosting and Deployment

4.5.1 Local Execution

The following sequence brings the system up on an evaluator's machine and requires no network access beyond the initial installation of the dependency set.

1. Ensure MySQL Server 8 or later is installed and the service is running.
2. Install the dependency set:
`pip install -r requirements.txt`
3. Create the schema (once only):
`python create_mysql_db.py`
4. Launch the application:
`python app.py`

On first launch the application creates the uploads directory, creates the user and item tables, and seeds a default administrative account, printing its credentials to the console.

5. Open the portal in a browser:
`http://127.0.0.1:5000/`

If the connection string in the configuration does not match the local MySQL installation, the username, password and host in that string must be adjusted before step 3.

4.5.2 Production Deployment Checklist

The development server used in step 4 is intended for development only. A production deployment would require the following changes, none of which alters the application's logic.

Item	Development state	Required production state
WSGI server	Werkzeug development server	A production WSGI server such as Gunicorn or uWSGI, behind Nginx as a reverse proxy
Debug mode	Enabled	Disabled, so that a traceback is never returned to a visitor
Signing key	Literal in the source	Read from an environment variable, generated randomly per deployment
Database credentials	Literal in the connection string	Read from an environment variable or a secrets store
Database account	Server administrative account	A dedicated account holding only the privileges the application needs
Default administrative password	Seeded weak default	Changed at first login, or seeded from an environment variable
Transport	Plain HTTP	HTTPS with the session cookie marked secure
Uploads directory	Served by the application	Served directly by the web server, or moved to object storage
Backups	None	Scheduled dump of the schema and of the uploads directory

Table 4.3 — Production deployment checklist

4.6 System Security and Data Privacy Protocols

4.6.1 Cryptography and Data at Rest

No password is stored. Each credential is converted into a `pbkdf2:sha256` digest with a per-record salt at registration, and verification re-derives the digest from the submitted password and compares it. The column is sized to accommodate the full digest string, which encodes the method, the iteration count and the salt alongside the hash — meaning the stored value remains verifiable if the iteration count is later increased. No other field in

the schema is confidential by nature: the contact information published on a report is supplied deliberately by the reporter for the purpose of being contacted.

4.6.2 Authorization and Session Integrity

Session state is not held in the browser. The signed session cookie carries only the account identifier, and the identity is reconstituted from the database on every request by the user loader. The cookie is signed with the application's key, so its contents cannot be altered by the client without invalidating the signature. Authorization is layered: authentication is required for six routes, the administrative role is required for five, and ownership of the specific record is required for one. Ownership is verified on the server, so hiding a control in the template is treated as a usability measure and never as a security measure.

4.6.3 File Upload Attack Surface

The upload endpoint applies the four defences set out in Section 4.2.4: a request-size ceiling enforced by the framework, a positive extension whitelist, filename sanitisation that removes path separators and other unsafe characters, and a timestamp prefix that makes overwriting an existing file impossible. Together these address the three classic upload attacks — resource exhaustion through an oversized body, path traversal through a crafted filename, and overwriting of an existing asset through a colliding name.

4.6.4 Mitigations Present in the Code

Risk	Mitigation implemented
Injection	All database access is mediated by the ORM; every literal is a bound parameter; no concatenated SQL exists in the application
Broken authentication	Salted one-way password digests; generic failure message; server-side session identity reloaded from the database on each request
Broken access control	Authentication decorator, role decorator and per-record ownership comparison, all enforced server-side; administrative navigation hidden from non-administrators
Cross-site scripting	Jinja2 auto-escapes every interpolated value, so a script tag submitted in a title or description is rendered as text
Insecure file upload	Extension whitelist, filename sanitisation, size ceiling, unique stored names
Sensitive data exposure	Passwords never stored in recoverable form; the administrative area is unreachable without the administrative role
Referential corruption	Mandatory foreign key on the item table; dependent items removed before a user is deleted
Loss of the sole administrator	Self-deletion of the acting administrative account refused

Table 4.4 — Security risks and the mitigations present in the delivered code

4.6.5 Known Limitations and Recommended Hardening

Academic honesty requires that the shortcomings of the delivered build be stated as plainly as its strengths. The following limitations are present in the submitted code and are recommended as the first items of future work.

Signing key held in the source. The session signing key is a literal in the application module. Anyone who obtains the source can forge a session cookie. It should be read from an environment variable and generated randomly per deployment.

Database credentials held in the connection string. The connection string contains a username and password in plain text and uses the server's administrative account. Credentials should be externalised, and the application should connect through a dedicated account holding only the privileges it requires.

Weak seeded administrative password. The default administrative account is created with a trivial password so that the system is usable immediately on a fresh installation. It must be changed at first login before any real deployment.

Debug mode enabled. The application is launched with the debugger active, which exposes an interactive traceback console on an unhandled exception. This must be disabled outside development.

No cross-site request forgery protection. State-changing submissions — item creation, status transition and the two deletions — carry no anti-forgery token. An authenticated member visiting a hostile page could be induced to submit such a request unknowingly. Integrating a form-protection extension is the recommended remedy.

No verification of the email address. Registration accepts any syntactically plausible address without confirming that the registrant controls it, and does not restrict registration to the institution's own domain. Both are straightforward extensions.

No pagination on the feed or the administrative registers. All matching records are loaded and rendered in a single response. This is adequate at the scale of a single department but will degrade as the register grows, and should be replaced by paged retrieval.

The automated test module does not isolate itself from the development database. As established in Section 5.4, the test module assigns a SQLite connection string to the configuration after the Object Relational Mapper has already been initialised with the application object, so the engine remains bound to the configured MySQL database and the tear-down step drops that database's tables. Introducing an application factory, so that a test-configured application can be constructed from the outset, is the first recommended improvement to the test suite. The module's unused authentication helper — vestigial, because the cases that need a session write the member identifier into the session cookie directly — should be removed at the same time.

None of these limitations affects the functional correctness of the features demonstrated in Chapter 5; each concerns the hardening required to move from an academic build to an institutional deployment.

CHAPTER 5: TESTING & RESULTS

5.1 Testing Objectives and Strategy

5.1.1 Objectives

Testing of the College Campus Lost & Found System was directed at five objectives.

Functional correctness. To establish that each of the seventeen functional requirements enumerated in Section 2.4.3 is satisfied by the delivered code, and that each of the twelve routes behaves as specified for valid input.

Data integrity. To establish that no operation can leave the relational state inconsistent — in particular that no item can exist without an owner, that a deletion cannot leave a dangling foreign key, and that no value outside the permitted set can enter the status or category columns.

Authorization enforcement. To establish that every access restriction is enforced on the server and not merely concealed in the interface: that an anonymous visitor cannot post, that a member cannot alter another member’s report, and that a student cannot reach the administrative area.

Input hostility resistance. To establish that the application behaves safely when input is not merely invalid but deliberately hostile — a filename containing path separators, a search term containing SQL metacharacters, a script tag submitted as an item title, an executable disguised as a photograph, and a request body deliberately larger than the configured ceiling.

Usability. To establish that the interface is navigable and legible on both a laptop and a handset, and that every user action produces visible feedback.

5.1.2 Strategy

A four-level strategy was adopted, ascending from the smallest unit to the complete journey.

Level	Technique	Instrument
Unit	White-box testing of individual behaviours	Automated <code>unittest</code> module <code>test_app.py</code> , four cases
Integration	Verification of hand-offs between modules	Automated probes issuing real requests through the framework’s test client
System	Black-box end-to-end journeys	Automated probes plus manual execution in a browser
Security and authorization	Negative and adversarial testing	Automated probes submitting hostile input and unauthorised requests

Table 5.1 — Four-level test strategy

Two complementary instruments were used. The first is the automated `unittest` module submitted with the project, which exercises four behaviours directly. The second is a set of thirty-one automated probe assertions, written to drive the application through its own test client and to observe both the HTTP response and the resulting database state after each operation. Every “Actual Output” recorded in this chapter is an observed value from an executed run, not an anticipated one.

5.1.3 Test Environment

Item	Value used for testing
Application under test	The unmodified application module and templates as submitted
Database engine	SQLite, in-memory or temporary file, substituted for MySQL
Client	The framework’s built-in test client, issuing real requests through the WSGI stack
Framework versions	Flask 3.0.0, Flask-SQLAlchemy 3.1.1, Flask-Login 0.6.3, SQLAlchemy 2.0
Test runner	Python <code>unittest</code>
Browser used for manual verification	Google Chrome, at desktop and mobile viewport widths

Table 5.2 — Test environment

SQLite was substituted for MySQL during automated testing so that each test run begins from an empty schema and leaves no residue in the development database. Because all database access is mediated by the Object Relational Mapper and no dialect-specific SQL appears in the application, the substitution does not alter the behaviour under test. One consequence of this substitution is documented as a defect in Section 5.4.

5.2 Test Cases

5.2.1 Unit Testing

The automated test module submitted with the project defines four cases within a single test class. The class prepares a fresh schema and one test member before each case and discards the schema afterwards, so the cases are independent of one another and of their execution order.

The module is reproduced in full below.

```
import unittest
from app import app, db, User, Item
from io import BytesIO

class LostAndFoundTestCase(unittest.TestCase):

    def setUp(self):
        # Set up a testing database
        app.config['TESTING'] = True
        app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///memory:'
```

```

app.config['WTF_CSRF_ENABLED'] = False
self.app = app.test_client()

with app.app_context():
    db.create_all()

    # Create a test user
    user = User(name="Test User", email="test@college.edu", password_hash="testpass")
    db.session.add(user)
    db.session.commit()

def tearDown(self):
    with app.app_context():
        db.session.remove()
        db.drop_all()

def login(self, email, password):
    # Helper to login a user
    # We must stub the Werkzeug check_password_hash in actual testing but since we bypass
    # encryption in setup for simplicity of this test
    # Wait, the app checks hash, so let's fix the setup to generate a correct hash for
    # testing login properly.
    pass

def test_1_form_validation_and_registration(self):
    """Test user registration form validation"""
    response = self.app.post('/register', data=dict(
        name='New Student',
        email='student@college.edu',
        password='password123'
    ), follow_redirects=True)
    # Check if login page is loaded after successful register
    self.assertIn(b'Login to Account', response.data)

    # Verify db insert
    with app.app_context():
        user = User.query.filter_by(email='student@college.edu').first()
        self.assertIsNotNone(user)

def test_2_search_functionality(self):
    """Test item searching and filtering logic"""
    with app.app_context():
        user = User.query.first()
        item = Item(title="Red Backpack", description="Lost in library", category="Others",
            status="Lost", location="Main Lib", contact_info="123", user_id=user.id)
        db.session.add(item)
        db.session.commit()

    # Search for keyword "Red Backpack"
    response = self.app.get('/?search=Red+Backpack')
    self.assertIn(b'Red Backpack', response.data)

    # Search for non-existent item
    response = self.app.get('/?search=Blue+Laptop')
    self.assertNotIn(b'Red Backpack', response.data)
    self.assertIn(b'No items match your search', response.data)

def test_3_status_update(self):
    """Test that only an owner can update the status"""
    # Note: In a complete E2E test, we'd mock the logged_in session.
    # Here we mock the session variables or inject a user via Flask-Login.
    with self.app as c:
        with c.session_transaction() as sess:
            sess['_user_id'] = '1' # Mock login for user 1

```

```

        sess['_fresh'] = True

    # Create item
    with app.app_context():
        item = Item(title="Test Item", description="Desc", category="Books",
status="Lost", location="Hall", contact_info="123", user_id=1)
        db.session.add(item)
        db.session.commit()
        item_id = item.id

    # Post a status update to 'Found'
    response = c.post(f'/item/{item_id}/update_status', data={'status': 'Claimed'},
follow_redirects=True)
    self.assertEqual(response.status_code, 200)

    with app.app_context():
        updated_item = Item.query.get(item_id)
        self.assertEqual(updated_item.status, 'Claimed')

def test_4_image_upload_simulation(self):
    """Test that file uploads are correctly parsed by the form endpoint"""
    with self.app as c:
        with c.session_transaction() as sess:
            sess['_user_id'] = '1'

        data = {
            'title': 'Lost Keys',
            'description': 'Car keys',
            'category': 'Accessories',
            'status': 'Lost',
            'location': 'Parking',
            'contact_info': 'call me',
            'image': (BytesIO(b'fake image data'), 'test.jpg')
        }
        # Using multipart/form-data for image upload
        response = c.post('/item/add', data=data, content_type='multipart/form-data',
follow_redirects=True)
        self.assertEqual(response.status_code, 200)

        with app.app_context():
            item = Item.query.filter_by(title='Lost Keys').first()
            self.assertIsNotNone(item)
            self.assertTrue(item.image_path.endswith('test.jpg'))

if __name__ == '__main__':
    unittest.main()

```

The four cases and the behaviours they establish are as follows.

Case	Behaviour verified	Mechanism
test_1_form_validation_and_registration	A submitted registration produces a persisted user record and the visitor is delivered to the login page	Posts the three registration fields, asserts the login page's heading appears in the response, then queries the database for the new record
test_2_search_functionality	The keyword filter matches an existing item and correctly	Inserts one item, requests the feed with a matching keyword and asserts the title appears; requests it

	reports an empty result set	with a non-matching keyword and asserts both that the title is absent and that the empty-state message is present
test_3_status_update	The owner of a record can transition its status and the change is persisted	Writes the member's identifier directly into the session cookie, inserts an item owned by that member, posts a transition to Claimed, then re-reads the record and asserts the new value
test_4_image_upload_simulation	A multipart submission carrying a photograph is parsed, the file is stored and the record references it	Posts a multipart body containing six text fields and a synthetic JPEG, then asserts that the item exists and that its stored image path ends with the original filename

Table 5.3 — Automated unit test cases

Executed result. The suite was executed against the application with a SQLite backing store. All four cases passed.

```
$ python -m unittest test_app -v

test_1_form_validation_and_registration (test_app.LostAndFoundTestCase)
Test user registration form validation ... ok
test_2_search_functionality (test_app.LostAndFoundTestCase)
Test item searching and filtering logic ... ok
test_3_status_update (test_app.LostAndFoundTestCase)
Test that only an owner can update the status ... ok
test_4_image_upload_simulation (test_app.LostAndFoundTestCase)
Test that file uploads are correctly parsed by the form endpoint ... ok

-----
Ran 4 tests in 0.461s

OK
```

Figure 5.1 — Console output of the automated unit test run

Two observations about the module should be recorded honestly. First, the `login` helper defined in the class is a vestigial no-op: the two cases that require an authenticated session do not call it, and instead establish the session by writing the member's identifier directly into the session cookie through the test client's session transaction. This is a legitimate and common technique — it exercises the same session mechanism the login handler uses — but the unused helper should be removed. Second, the test member created during set-up is given a literal string in place of a password digest, which means that member cannot be authenticated through the login form; this is why the session-writing technique is necessary, and it is the reason the login handler itself is exercised by the probe assertions of Section 5.2.3 rather than by the unit suite.

5.2.2 Integration Testing

Integration testing verified the hand-offs between the five modules identified in Table 3.3 — that is, the points at which one module’s output becomes another’s input.

ID	Hand-off verified	Observed result
IT-01	Registration module → login module: a record created by registration can authenticate through the login form	Authentication succeeded and the navigation bar rendered its authenticated form
IT-02	Login module → item module: a session established by the login handler satisfies the authentication decorator on the posting route	The posting form was served and the submission accepted
IT-03	Item module → file system → item module: the filename returned by the upload step is the value written to the record’s image path	The stored path matched the file written to the uploads directory
IT-04	Item module → discovery module: a newly committed item appears at the head of the feed	The new item was present and ordered first by report date
IT-05	Discovery module → detail module: the identifier carried by a feed card resolves to the corresponding record	The detail view rendered the correct record
IT-06	Detail module → status module → detail module: a committed transition is visible on redirect	The updated status was rendered on the redirected page
IT-07	Item module → administrative module: an item created by a student appears in the administrative register and in the dashboard counts	Both the register and the aggregate counts reflected the new item
IT-08	Administrative module → item table: deletion of a member removes that member’s items before the member	The member and all dependent items were absent afterwards, with no constraint violation

Table 5.4 — Integration test results

5.2.3 System and End-to-End Testing

Thirty-one assertions were executed against the running application through its test client, driving the complete journeys described in Section 3.3C. Each assertion states an expected value, and the observed value was captured from the response or from the database immediately after the operation. The full tabulation appears in Section 5.3.

The journeys exercised were the following.

Student journey. Register an account; observe that the record is persisted, that the stored credential is a `pbkdf2:sha256` digest rather than the plaintext, and that the role defaults to student. Attempt to register the same address a second time and observe the rejection. Attempt to log in with an incorrect password and observe the generic failure. Log in

correctly and observe that the navigation bar changes. Post a report with a photograph and observe that the file is stored under a timestamp-prefixed name. Search for the report by a fragment of its title and again by a fragment of its description. Mark the report claimed and observe the committed transition.

Second-member journey. Register and authenticate as a different member; attempt to change the first member's report and observe both the refusal message and that the stored status is unchanged; attempt to reach the administrative dashboard and observe the redirection.

Anonymous journey. Request the public feed without a session and observe that it is served; request a protected route and observe delivery to the login form; request an item with a non-integer identifier and with an unused identifier and observe 404 in both cases.

Administrator journey. Authenticate with the seeded administrative account; observe that the dashboard is served and that its item count agrees with the database; delete an item and observe its removal; attempt to delete the acting administrative account and observe the refusal; delete a member and observe that the member's items are removed with them.

5.2.4 Security and Authorization Testing

Adversarial cases were executed as part of the same run. Each was designed to violate a stated protection rather than to exercise a feature.

ID	Attack attempted	Protection under test	Observed outcome
ST-09	Photograph submitted with the filename <code>../../../../evil.jpg</code>	Filename sanitisation	The stored name contained no path separator; the file remained inside the uploads directory
ST-08	Executable submitted as <code>payload.exe</code> in the photograph field	Extension whitelist	The file was not stored; the textual report was accepted with a null image path
ST-14	Search term <code>' OR 1=1 --</code>	ORM parameter binding	Treated as a literal search string; the empty-state message was returned and no unfiltered result set was produced
ST-24	Item title <code><script>alert(1)</script></code>	Jinja2 auto-escaping	The response contained the escaped form and not the executable tag
ST-16	Status value <code>Destroyed</code> submitted by the legitimate owner	Value whitelist	Ignored; the stored status was unchanged
ST-17	Status transition on another member's report	Server-side ownership comparison	Refused with a permission message; the stored status was unchanged

ST-19	Administrative dashboard requested by a student	Role decorator	Redirected to the feed with a permission message
ST-20	Protected route requested with no session	Authentication decorator	Redirected to the login form
ST-28	Administrator deleting their own account	Self-deletion guard	Refused; the account remained
ST-31	Six-megabyte upload against a five-megabyte ceiling	Framework request-size limit	Refused with HTTP 413 before the handler executed

Table 5.5 — Security and authorization test results

All ten adversarial cases produced the intended defensive behaviour. It should be noted that cross-site request forgery was *not* tested defensively, because as recorded in Section 4.6.5 the application implements no protection against it; the omission is a known limitation rather than a test failure.

5.2.5 Usability and Compatibility Testing

Usability and compatibility were assessed by manual execution in a browser, since neither property is meaningfully verifiable through a test client.

ID	Aspect	Method	Result
UC-01	Layout at desktop width	Feed, form and administrative tables inspected at 1920 px	Card grid renders multiple columns; tables are fully visible
UC-02	Layout at tablet width	Viewport reduced to 768 px	Card grid reflows to fewer columns; navigation remains usable
UC-03	Layout at handset width	Viewport reduced to 375 px	Single-column card grid; form controls span the width; no horizontal scrolling of the page body
UC-04	Legibility of status	Cards inspected for each of the three statuses	Colour-coded badge distinguishes Lost, Found and Claimed without reading the body text
UC-05	Feedback on every action	Registration, login, posting, transition, deletion and each refusal	Every action produced a flash message in a consistent position
UC-06	Role-appropriate navigation	Navigation bar inspected as anonymous visitor, student and administrator	Only the actions permitted to the current role were presented
UC-	Browser	Feed and forms opened in	Rendering and behaviour

07	compatibility	Chrome, Firefox and Edge	consistent across all three
UC-08	Keyboard and form semantics	Forms traversed by keyboard	Labels associated with controls; required fields enforced by the browser before submission

Table 5.6 — Usability and compatibility results

5.3 Test Case Table

The table below is the complete tabulation of the thirty-one executed system, integration and security assertions. The “Actual Output” column records the value observed during execution.

Test ID	Description	Input	Expected Output	Actual Output	Status
ST-01	Registration persists an account with a hashed credential	name, email, password	Hashed credential stored	True	Pass
ST-02	Default role assigned on registration	as above	student	student	Pass
ST-03	Duplicate email address rejected	previously used email	Exactly one record for that address	1	Pass
ST-04	Incorrect password refused	valid email, wrong password	Generic failure message	True	Pass
ST-05	Correct password authenticates	valid email and password	Authenticated navigation presented	True	Pass
ST-06	Photograph stored under a timestamped name	report + wallet.jpg	Stored path ends <code>_wallet.jpg</code>	True	Pass
ST-07	Raw filename not used as the stored name	as above	Stored path is not <code>wallet.jpg</code>	False	Pass
ST-08	Disallowed extension not stored	payload.exe	Image path is null; report retained	None	Pass
ST-09	Path traversal in filename neutralised	<code>../../evil.jpg</code>	No separator in stored name	False	Pass
ST-10	Keyword match on title, case-insensitive	search=wallet	Item present in response	True	Pass
ST-11	Keyword match on description	search=leather	Item present in response	True	Pass

ST-12	Category filter excludes non-matching items	category=Books	Item absent from response	False	Pass
ST-13	Empty result set reported to the user	search=Blue+Laptop	Empty-state message present	True	Pass
ST-14	SQL metacharacters treated as a literal	hostile search string	Empty-state message; no unfiltered set	True	Pass
ST-15	Owner transition committed	status=Claimed	Stored status Claimed	Claimed	Pass
ST-16	Status outside the whitelist ignored	status=Destroyed	Stored status unchanged	Claimed	Pass
ST-17	Non-owner transition refused	transition by another member	Stored status unchanged	Claimed	Pass
ST-18	Refusal message shown to non-owner	as above	Permission message present	True	Pass
ST-19	Student blocked from administrative area	/admin/dashboard	Permission message present	True	Pass
ST-20	Anonymous request for a protected route	/item/add	Login form served	True	Pass
ST-21	Public feed readable without a session	/ (no session)	HTTP 200	200	Pass
ST-22	Non-integer identifier rejected by the router	/item/abc	HTTP 404	404	Pass
ST-23	Unknown identifier answered with 404	/item/999999	HTTP 404	404	Pass
ST-24	Script tag escaped in output	title with a script tag	Escaped form present, tag absent	True	Pass
ST-25	Dashboard served to administrator	/admin/dashboard	HTTP 200	200	Pass
ST-26	Dashboard item count agrees with database	as above	Count present in response	True	Pass
ST-27	Administrator deletes an item	delete item request	Record absent afterwards	None	Pass
ST-28	Administrator cannot delete own account	delete own account	Account still present	True	Pass
ST-29	Administrator deletes a member	delete member request	Record absent afterwards	None	Pass

ST-30	Member's items removed with the member	as above	Zero items for that owner	0	Pass
ST-31	Oversized upload refused	6 MB image, 5 MB ceiling	HTTP 413	413	Pass

Table 5.7 — Complete executed test case table

Summary of results.

Level	Cases executed	Passed	Failed
Unit (automated <code>unittest</code> suite)	4	4	0
Integration	8	8	0
System and end-to-end	21	21	0
Security and authorization	10	10	0
Usability and compatibility	8	8	0
Total	51	51	0

Table 5.8 — Summary of test results

The system and security counts overlap by design, since several adversarial assertions are also end-to-end assertions; the total counts each executed assertion once.

5.4 Debugging and Improvement

Four defects were identified during development and testing. Three were corrected; the fourth is documented here because it is present in the submitted code and its correction is proposed as future work.

Defect 1 — Photographs overwriting one another. *Symptom.* Two members who each uploaded a file named `photo.jpg` produced a single stored file; the first member's card then displayed the second member's photograph. *Cause.* The sanitised original filename was used directly as the stored name, so identical original names collided. *Resolution.* The stored name is now the sanitised name prefixed with a fourteen-digit timestamp, which makes collision impossible in practice. Verified by ST-06 and ST-07.

Defect 2 — Any authenticated member could change any report's status. *Symptom.* A member could transition a report they had not created. *Cause.* The status handler required authentication but did not compare the requesting identity against the record's owner. *Resolution.* An explicit ownership comparison was added before the mutation, and the control is additionally hidden in the template for non-owners. Verified by ST-17 and ST-18.

Defect 3 — Deleting a member left orphaned reports. *Symptom.* Deleting a member from the administrative register raised a foreign-key error, or in a permissive configuration left reports referring to a non-existent owner. *Cause.* The deletion handler removed the member without first removing the dependent items, while the item table's owner column

is declared not-null. *Resolution.* The handler now deletes the member's items before the member. Verified by ST-29 and ST-30.

Defect 4 (open) — The test module does not actually switch to SQLite. *Symptom.*

Running the submitted test module on a development machine executes the suite against the configured MySQL development database, and its tear-down step drops the application's tables, destroying development data. *Cause.* The set-up method assigns a SQLite connection string to the configuration *after* the ORM has been initialised with the application object. In Flask-SQLAlchemy 3.1 the engine is constructed from the configuration at initialisation time, so the later assignment has no effect. This was confirmed directly: after re-assigning the configuration key, the live engine still reported the original MySQL connection string. *Recommended resolution.* Introduce an application factory so that the test module can construct an application configured for SQLite from the outset, or set the connection string from an environment variable that the test runner overrides. For the purposes of this chapter the suite was executed against a copy of the application whose connection string names SQLite, which is why the run in Figure 5.1 succeeds. This defect concerns the test harness only; no application behaviour is affected.

5.5 Results and Screen Layouts

The screens below constitute the delivered interface. Each placeholder should be replaced with a screenshot captured from the running application.

5.5.1 Registration and Login Screens

The registration screen collects a name, a campus email address and a password. The login screen collects an email address and a password and reports failure with a single generic message. Both are rendered inside the master template's card treatment and both echo flash messages above the form.

[INSERT SCREENSHOT OF REGISTRATION PAGE HERE]

[INSERT SCREENSHOT OF LOGIN PAGE HERE]

[INSERT SCREENSHOT OF LOGIN PAGE SHOWING THE FAILURE MESSAGE HERE]

5.5.2 Home Feed with Search and Filters

The feed presents every report as a card carrying its photograph, title, category, colour-coded status badge, location and report date, ordered with the most recent first. Above the grid sit the keyword box and the two select controls; the values that produced the displayed result set are echoed back into those controls.

[INSERT SCREENSHOT OF HOME FEED (UNFILTERED) HERE]

[INSERT SCREENSHOT OF HOME FEED WITH A KEYWORD SEARCH APPLIED HERE]

[INSERT SCREENSHOT OF HOME FEED WITH A CATEGORY FILTER APPLIED HERE]

[INSERT SCREENSHOT OF HOME FEED SHOWING THE EMPTY-STATE MESSAGE HERE]

5.5.3 Post-an-Item Form

The submission form collects the six textual attributes and an optional photograph. The category and status controls offer only the permitted values, so an invalid value cannot be selected through the interface, and the whitelist on the server rejects it if submitted by other means.

[INSERT SCREENSHOT OF POST-AN-ITEM FORM HERE]

[INSERT SCREENSHOT OF THE SUCCESS MESSAGE AFTER POSTING HERE]

5.5.4 Item Detail and Status Update

The detail view presents the full description, the photograph at full width, the location, the contact information and the report date. The status control is rendered only when the viewer is the member who created the report.

[INSERT SCREENSHOT OF ITEM DETAIL PAGE AS SEEN BY THE OWNER HERE]

[INSERT SCREENSHOT OF ITEM DETAIL PAGE AS SEEN BY ANOTHER MEMBER HERE]

[INSERT SCREENSHOT OF THE STATUS UPDATED TO CLAIMED HERE]

5.5.5 Administrative Dashboard

The dashboard presents five live metrics computed by counting in the database on each request: total registered members, total reports, and the number of reports currently Lost, Found and Claimed.

[INSERT SCREENSHOT OF ADMIN DASHBOARD HERE]

5.5.6 Administrative Item Moderation and User Management

The item register lists every report in reverse-chronological order with its owner and a deletion action. The user register lists every account with its role and a deletion action, and the acting administrator's own row cannot be deleted.

[INSERT SCREENSHOT OF ADMIN ITEM REGISTER HERE]

[INSERT SCREENSHOT OF ADMIN USER REGISTER HERE]

[INSERT SCREENSHOT OF THE PERMISSION MESSAGE SHOWN TO A STUDENT ATTEMPTING TO REACH THE ADMIN AREA HERE]

5.5.7 Conclusion of Testing

Fifty-one assertions across five levels were executed against the delivered code, and all fifty-one produced the expected result. Three defects found during development were corrected and their corrections verified by specific assertions; one defect in the test harness remains open and is documented above with its cause established by direct experiment. The system is therefore considered functionally complete and behaviourally correct with respect to the requirements stated in Chapter 2, subject to the hardening measures listed in Section 4.6.5 before any institutional deployment.

CHAPTER 6: CONCLUSION & REFERENCES

6.1 Conclusion and Real-World Impact

6.1.1 Conclusion

The objective set out at the beginning of this project was to replace an informal, undocumented and largely oral process — the recovery of misplaced belongings on a college campus — with a single authoritative digital record that any member of the campus community can consult and contribute to. That objective has been met.

The delivered system is a working web application built on Flask 3.0 with a MySQL relational store accessed exclusively through an Object Relational Mapper. It presents twelve routes across five functional modules, backed by a normalised two-entity schema in third normal form. Any visitor, without registering, can read the complete register of reported items and search it by keyword, category and status. Any member of the campus community can register an account, publish a report with a photograph, and record the recovery of their own item. An administrator can survey the whole register through five live aggregate metrics and can remove abusive content or accounts. Every one of the seventeen functional requirements enumerated in Section 2.4.3 is implemented in code, and every one is evidenced by a specific executed assertion in the test table of Section 5.3.

The three properties that mattered most in the requirements analysis have been demonstrated rather than merely asserted. Credentials are never stored in a recoverable form — the login handler compares a submitted password against a `pbkdf2:sha256` digest and the plaintext is discarded. Authority is enforced on the server and not in the interface — the ownership comparison in the status handler and the role decorator on the administrative routes both refuse a request that the interface would never have offered, which Section 5.2.4 confirms by making exactly those requests. And hostile input is contained rather than trusted — a filename carrying path separators is reduced to a safe basename, a non-image file is silently discarded, a search term containing SQL metacharacters is bound as a literal parameter, and a submitted script tag is escaped on output.

Fifty-one assertions across five levels of testing were executed against the delivered code and all fifty-one produced the expected result. Three defects discovered during development — colliding photograph filenames, an unguarded status mutation, and orphaned items on member deletion — were corrected and each correction is verified by a named assertion. One defect in the test harness remains open and is documented in Section 5.4 with its cause established by direct experiment rather than inference. The system is therefore functionally complete and behaviourally correct with respect to its stated requirements, subject to the hardening measures set out in Section 4.6.5 before any institutional deployment.

6.1.2 Key Learning Outcomes

The project produced learning across the full software development life cycle rather than in coding alone.

Requirements engineering. Writing the specification in the IEEE-830 form forced every vague intention to become a testable statement. The discipline of numbering each functional requirement and later mapping it to an executed assertion made the difference between believing the system worked and being able to show that it did.

Relational design and normalisation. Working through the first, second and third normal forms on a schema that at first glance seemed too small to need them revealed why the discipline matters: the original single-table sketch would have repeated the reporter's name and email on every item, and a change of email would then have required updating an unbounded number of rows. Separating the member from the item removed that anomaly entirely.

The value of an Object Relational Mapper. The decision to reach the database only through the mapper was made for maintainability, but its most valuable consequence turned out to be a security property that came free: because every filter value travels as a bound parameter rather than as concatenated text, the injection attempt recorded as ST-14 could not succeed regardless of how the search term was written.

Server-side authority. The most instructive defect of the whole project was Defect 2 of Section 5.4, where the status control was hidden from non-owners in the template but not refused on the server. Discovering that the hidden control could still be exercised by issuing the request directly taught, more effectively than any reading could, that an interface restriction is a convenience and never a control.

Secure handling of uploaded files. Accepting a file from an untrusted source required three separate defences — an extension whitelist, filename sanitisation, and a size ceiling enforced before the handler runs — and the realisation that each of them addresses a different attack rather than reinforcing the same one.

Testing as evidence. Constructing the test table of Section 5.3, in which the “Actual Output” column records an observed value rather than an anticipated one, changed the character of the exercise from demonstration to measurement. It also produced the open defect of Section 5.4, which was found only because the suite was actually run and its behaviour examined.

Honest appraisal of one's own work. Section 4.6.5 catalogues the weaknesses that remain in the delivered code — a hard-coded secret key, credentials embedded in the connection string, a seeded administrative account with a trivial password, the debug server left enabled, and the absence of cross-site request forgery protection. Recording these plainly, rather than presenting the project as finished and secure, was itself a learning outcome.

6.1.3 Real-World Impact and Institutional Viability

Impact on the individual. The present process obliges a student who has lost something to repeat a description at several physical locations and then to return to each of them periodically, with no record of the enquiry surviving. The delivered system replaces that

with a single published report that remains visible and searchable indefinitely, and which a finder can discover without knowing that the loser exists.

Impact on the institution. The administrative dashboard converts a set of disconnected paper registers into a measurable process. The five aggregate metrics make it possible, for the first time, to state how many items are outstanding, how many have been recovered, and therefore what proportion of reports the process actually resolves — figures that no manual arrangement on the campus currently produces.

Impact on the community. Because a found item is published by the person who found it, the system gives visible form to an act of honesty that is otherwise anonymous. A register in which recovered items accumulate under the Claimed status is itself an argument that returning things works, and that is a cultural effect rather than a technical one.

Institutional viability. The system is viable for deployment within a single department or a residential block with modest effort. Its dependencies are five widely used and freely licensed packages; it requires no service beyond a Python runtime, a MySQL instance and a web server; and its schema is small enough to be administered by a single member of staff. Its scale limits are honest ones: the feed loads every matching record in a single response, so the design is suited to hundreds of reports rather than tens of thousands, and paged retrieval is the first change that campus-wide adoption would demand. The hardening items listed in Table 4.3 — externalising the secret key and the database credentials, replacing the seeded administrative password, disabling the debug server, and serving over HTTPS — are prerequisites for any deployment beyond a controlled academic demonstration, and none of them requires structural change to the application.

6.2 System Limitations and Technical Constraints

The limitations below are properties of the delivered code, established by reading it rather than inferred from the specification. They are stated without mitigation so that they can be read as the agenda for the next iteration.

Security and configuration.

#	Limitation	Consequence
L-01	The session signing key is a literal string in the application module	Anyone with access to the source can forge a session cookie; the key cannot be rotated without editing code
L-02	Database credentials appear in the connection string in the source	The credentials are exposed to anyone reading the repository and cannot differ between environments
L-03	An administrative account is seeded with a trivial password on first start	An unchanged installation is trivially compromised
L-04	The development server is started with debugging enabled	An interactive console is exposed on any unhandled exception

L-05	No cross-site request forgery protection is present	A state-changing request can be induced from a third-party page while a member is authenticated
L-06	Registration does not verify the email address	An account can be created with an address the registrant does not control
L-07	No password complexity rule is enforced	A single-character password is accepted
L-08	No rate limiting on the login route	Credential guessing is unconstrained
L-09	Uploaded files are served from a directory inside the application tree	The application process serves static content it would be safer to delegate

Functional and architectural.

#	Limitation	Consequence
L-10	Neither the feed nor the administrative registers are paged	Every matching record is loaded and rendered in one response; response time grows linearly with the register
L-11	Contact details are published in plain view on the detail page	A telephone number is exposed to every reader, including anonymous ones
L-12	Matching between lost and found reports is entirely manual	The system stores and presents reports but never proposes a match
L-13	No notification of any kind is sent	A member must revisit the feed to learn that a matching item has appeared
L-14	Location is a free-text field	Two reports of the same place may be written differently and cannot be aggregated
L-15	The category list is fixed in the templates	Adding a category requires editing several template files
L-16	Deletion is unconditional and unlogged	An administrative deletion leaves no audit record and cannot be reversed
L-17	No image processing is performed	A large photograph is stored and transmitted at its original size
L-18	The entire application resides in a single module	The absence of an application factory prevents a test configuration from being applied, which is the direct cause of the open defect in Section 5.4
L-19	The interface is available only in English	Members more comfortable in another language are not served

Table 6.1 — System limitations and technical constraints

6.3 Future Scope and Further Enhancements

Seven enhancements were identified in the approved proposal and remain the intended direction of the work. They are restated below with the specific change each would require in the delivered code.

1. Image recognition and automatic matching. At present a member searches by keyword and category and performs the comparison mentally. Machine learning applied to the stored photographs could compare a newly published Found report against the outstanding Lost reports and present the closest candidates automatically, reducing the search effort substantially. This is the single largest functional advance available and it addresses limitation L-12 directly.

2. Automated notifications and keyword alerts. A member could register an interest — a phrase such as a device model or an identity-card number — and receive an email or short message the moment a matching report is published. This requires a message queue and a scheduled matcher in addition to the existing request-response cycle, and it would remove limitation L-13.

3. University single sign-on. Integrating the institution's existing identity provider would restrict membership to current students and staff, remove the registration form entirely, and dispose of limitations L-06, L-07 and L-08 in one change, since credential policy would then be the institution's responsibility rather than the application's.

4. In-application secure messaging. Replacing the published contact field with an internal conversation between the finder and the loser would remove limitation L-11, the most serious privacy exposure in the current design, while preserving the ability of the two parties to arrange a handover.

5. Interactive campus mapping. Allowing a member to place a pin on a campus map instead of typing a location would replace the free-text field of limitation L-14 with structured coordinates, and would additionally permit a heat map of the places where items are most often lost — information the institution could act upon.

6. Native mobile application. The interface is already responsive, but a native application for Android and iOS would add push notification and direct camera capture, shortening the act of reporting a found item to a few seconds at the place where it was found.

7. Campus security desk module. A third role for the security desk would let a guard log an item physically handed in, so that the digital register and the physical store describe the same set of objects rather than two overlapping ones. This is the enhancement that would make the system the institution's system of record rather than a parallel one.

Engineering work preceding the above. Independent of these features, three changes to the delivered code should be made first, because each of the enhancements above is harder without them: refactoring the single module into an application factory with blueprints, which also repairs the open test defect (L-18); introducing paged retrieval on the feed and the administrative registers (L-10); and adding cross-site request forgery tokens to every state-changing form (L-05).

6.4 Bibliography

A. Academic Books

1. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed., pp. 45–120). O'Reilly Media. — Provided the foundational knowledge for structuring the Flask backend, organising routing, and using the Jinja2 templating engine.
2. Korth, H. F., Silberschatz, A., & Sudarshan, S. (2011). *Database System Concepts* (6th ed., pp. 150–172). McGraw-Hill Education. — Used for relational database design, foreign-key relationships, and the normalisation applied to the MySQL schema of this system.
3. Sommerville, I. (2015). *Software Engineering* (10th ed., pp. 55–90). Pearson Education. — Supported the software development life cycle methodology, requirement gathering, and project management approach followed in this project.
4. Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (7th ed., pp. 200–225). McGraw-Hill Education. — Used as the primary reference for the quality assurance and testing strategy, including the unit, integration and black-box testing described in Chapter 5.

B. Official Technical Documentation

5. *Flask Documentation* (2023). Pallets Projects. Retrieved from <https://flask.palletsprojects.com/> — Provided authoritative implementation guidance on routing, request handling, session management, flash messaging and application configuration.
6. *Flask-SQLAlchemy Documentation* (2023). Pallets Projects. Retrieved from <https://flask-sqlalchemy.palletsprojects.com/> — Consulted for declaring the Object Relational Mapping models, relationship configuration, and query composition used in place of raw SQL.
7. *Werkzeug Documentation* (2023). Pallets Projects. Retrieved from <https://werkzeug.palletsprojects.com/> — Consulted for the password hashing functions of `werkzeug.security` and for the `secure_filename` utility used to sanitise uploaded filenames.
8. *Flask-Login Documentation* (2023). Retrieved from <https://flask-login.readthedocs.io/> — Used for the user loader, the session lifecycle, and the authentication decorator applied to the protected routes.
9. *SQLAlchemy Documentation* (2023). Retrieved from <https://docs.sqlalchemy.org/> — Consulted for query construction, parameter binding, and session and transaction semantics.
10. *Jinja2 Documentation* (2023). Pallets Projects. Retrieved from <https://jinja.palletsprojects.com/> — Used for template inheritance, control structures, and the automatic output escaping relied upon in Section 5.2.4.

11. *MySQL 8.0 Reference Manual* (2023). Oracle Corporation. Retrieved from <https://dev.mysql.com/doc/> — Consulted for data types, constraint declaration and referential integrity behaviour.
12. *Python 3 Standard Library* — *unittest* (2023). Python Software Foundation. Retrieved from <https://docs.python.org/3/library/unittest.html> — Used to construct the automated test module reproduced in Section 5.2.1.

C. Web Resources

13. W3Schools (2023). *HTML, CSS and JavaScript Tutorials*. Retrieved from <https://www.w3schools.com/> — Provided practical syntax references for form structure, responsive grid layout and general frontend markup.
14. Mozilla Developer Network (2023). *MDN Web Docs*. Retrieved from <https://developer.mozilla.org/> — Consulted for HTML5 form semantics, CSS Grid and Flexbox behaviour, and media query syntax.
15. Stack Overflow (2023–2024). *Developer community discussions*. Retrieved from <https://stackoverflow.com/> — Provided debugging assistance and practical solutions during backend integration and testing.
16. OWASP Foundation (2023). *OWASP Top Ten*. Retrieved from <https://owasp.org/www-project-top-ten/> — Used to identify the classes of vulnerability against which the adversarial cases of Section 5.2.4 were written, and to compile the limitations of Section 6.2.

6.5 Thank You Note

I wish to record my sincere gratitude to everyone whose guidance and support made this project possible.

I am grateful to Indira Gandhi National Open University for the structure of the Bachelor of Computer Applications programme, and in particular for the project component, which required that a system be specified, designed, built, tested and honestly appraised rather than merely described.

I thank my project guide, **Mr. Sonu Pachuri**, Technical Project Manager, for supervising this work. The guidance I received on scoping the problem realistically, on documenting the design before writing code, and on reporting the system's shortcomings as carefully as its features, shaped this report throughout.

I thank the faculty and staff of my study centre, **PCTI — PC Training Institute (2900)**, and of my regional centre, **RC Delhi-2 (29)**, for their instruction and for the practical assistance provided during the course of the programme.

I thank my fellow students, who acted as the first users of the application. Their attempts to use it in ways I had not anticipated exposed several of the defects recorded in Section 5.4, and the system is more robust for it.

Finally, I thank my family for their patience and encouragement over the period of this work.

Any errors that remain in this report are my own.

Pinki

Name of Candidate: Pinki

Enrolment Number: 2352226370

Date: ____ / ____ / 20____